Huawei AI Certification Training

HCIA-AI

Machine Learning Lab Guide

ISSUE: 4.0



HUAWEI TECHNOLOGIES CO., LTD

Copyright © Huawei Technologies Co., Ltd. 2025. All rights reserved.

No part of this document may be reproduced or transmitted in any form or by any means without prior written consent of Huawei Technologies Co., Ltd.

Trademarks and Permissions

 and other Huawei trademarks are trademarks of Huawei Technologies Co., Ltd.

All other trademarks and trade names mentioned in this document are the property of their respective holders.

Notice

The purchased products, services and features are stipulated by the contract made between Huawei and the customer. All or part of the products, services and features described in this document may not be within the purchase scope or the usage scope. Unless otherwise specified in the contract, all statements, information, and recommendations in this document are provided "AS IS" without warranties, guarantees or representations of any kind, either express or implied.

The information in this document is subject to change without notice. Every effort has been made in the preparation of this document to ensure accuracy of the contents, but all statements, information, and recommendations in this document do not constitute a warranty of any kind, express or implied.

Huawei Technologies Co., Ltd.

Address: Huawei Industrial Base Bantian, Longgang Shenzhen 518129 People's Republic of China

Website: https://e.huawei.com

Huawei Certification System

Huawei Certification is an integral part of the company's Platform + Ecosystem strategy. It supports the new Information and Communications Technology (ICT) infrastructure that features Cloud-Pipe-Device synergy and covers all the ICT domains, including certification for the ICT architecture and its applications, as well as certification for cloud service and platform.

The certification system offers three levels: Huawei Certified ICT Associate (HCIA), Huawei Certified ICT Professional (HCIP), and Huawei Certified ICT Expert (HCIE), to meet the learning and advancement requirements of ICT practitioners.

Huawei certifications cover all ICT fields and adapt to the industry trend of ICT convergence. With its leading talent development system and certification standards, Huawei is committed to fostering new ICT talent in the digital era, and building a healthy ICT talent ecosystem.

The Huawei HCIA-AI V4.0 certification system is intended to train and certify engineers who are capable of designing, developing, and innovating AI products and solutions by using machine learning and deep learning algorithms.

With HCIA-AI V4.0 certification, you are proved to:

Understand the history of AI development.

Master algorithms related to traditional machine learning and deep learning.

Be capable of building, training, and deploying neural networks by using development framework PyTorch.

Be competent in sales, marketing, product manager, project management, technical support, and other AI positions.

The Huawei certification system introduces you to the industry and market, helps you in innovation, and enables you to stand atop the AI frontiers.


HCIE
Huawei Certified ICT Expert
HCIP
Huawei Certified ICT Professional
HCIA
Huawei Certified ICT Associate
Project Management
Cloud Platform & Services
Basic Software & Hardware
ICT Infrastructure
Cloud Platform & Services Domain
Cloud Computing
Cloud Service
Big Data
GaussDB
IoT
Basic Software & Hardware Domain
openEuler
openGauss
HarmonyOS
AI
IT Infrastructure Domain
Computing
Storage
Collaboration
Intelligent Vision
Digital Power
MDC
Datacom Domain
Datacom
WLAN
Security
Optical Domain
Transmission
Access
Wireless Domain
5G
LTE

About This Document

Overview

This document applies to candidates who are preparing for the HCIA-AI exam and others who want to learn basic machine learning knowledge.

Description

This lab guide comprises the following five experiments:

- Experiment 1 - Linear regression. The Python tool package scikit-learn is used to implement a simple linear regression algorithm.
- Experiment 2 - Linear regression expansion. The Python basic tool package numpy is used to implement algorithms such as linear regression and gradient descent from scratch.
- Experiment 3 - Logistic regression. The logistic regression algorithm in the tool package is used to implement simple classification.
- Experiment 4 - Decision tree. Trainees will construct a decision tree to predict the weather, and visualize the decision tree.
- Experiment 5 - K-means clustering algorithm.

Background Knowledge Required

This course is for Huawei's professional certification. Trainees are expected to meet the following requirements:

- Have basic knowledge of Python programming.
- Have basic math skills, including linear algebra and probability theory.

Lab Environment Overview

The lab environment is Python 3.7. For details about how to set up the environment, see the Environment Setup Guide.

Datasets used in this lab can be obtained at https://certification-data.obs.cn-north-4.myhuaweicloud.com/ENG/HCIA-AI/V4.0/Dataset.zip

Contents

About This Document 3
Overview 3
Description 3
Background Knowledge Required 3
Lab Environment Overview 3
1 Implementation of Common Machine Learning Algorithms 5
1.1 Introduction 5
1.1.1 About This Lab 5
1.1.2 Objectives 5
1.2 Code Implementation 5
1.2.1 Linear Regression 5
1.2.2 Linear Regression Implementation (Expansion Experiment) 7
1.2.3 Logistic Regression 10
1.2.4 Decision Tree 12
1.2.5 K-means Algorithm Implementation 14
1.3 Question 18
1.4 Summary 18

1 Implementation of Common Machine Learning Algorithms

1.1 Introduction

1.1.1 About This Lab

This lab introduces common machine learning algorithms to help you better understand their functions and usages. Specifically, it will explain how to build a linear regression algorithm from scratch and implement the decision tree and K-means clustering based on scikit-learn.

1.1.2 Objectives

- Build a linear regression algorithm from scratch.
- Master the use of classification and regression algorithms.
- Master the use of the V clustering algorithm.
- Master the implementation process of machine learning algorithms.

1.2 Code Implementation

1.2.1 Linear Regression

Step 1 Import dependent packages.

Input:

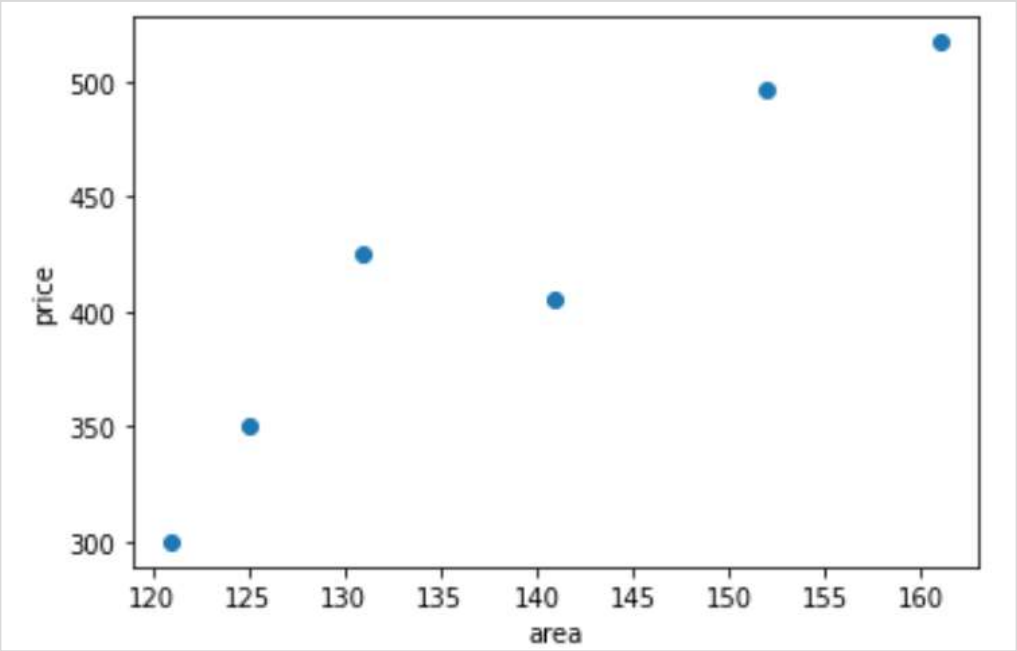
```
from sklearn.linear_model import LinearRegression # Import the linear regression model
import matplotlib.pyplot as plt # The plotting library
import numpy as np
```

Step 2 Build and visualize a house price dataset.

Input:

```
x = np.array([121, 125, 131, 141, 152, 161]).reshape(-1,1) # x denotes the house area as a feature.
y = np.array([300, 350, 425, 405,496,517]) # y denotes the house price.
plt.scatter(x,y)
plt.xlabel("area") # X axis indicates the area.
plt.ylabel("price") # Y axis indicates the price.
plt.show()
```

Output:



Step 3 Train the model.

Input:

```
lr = LinearRegression() # Encapsulate the linear regression model into an object.
lr.fit(x,y) # Train the model on the dataset.
```

Step 4 Visualize the model.

Input:

```
w = lr.coef_# Slope of the model
b = lr.intercept_# Intercept of the model
print('Slope: ',w)
print('Intercept: ',b)
```

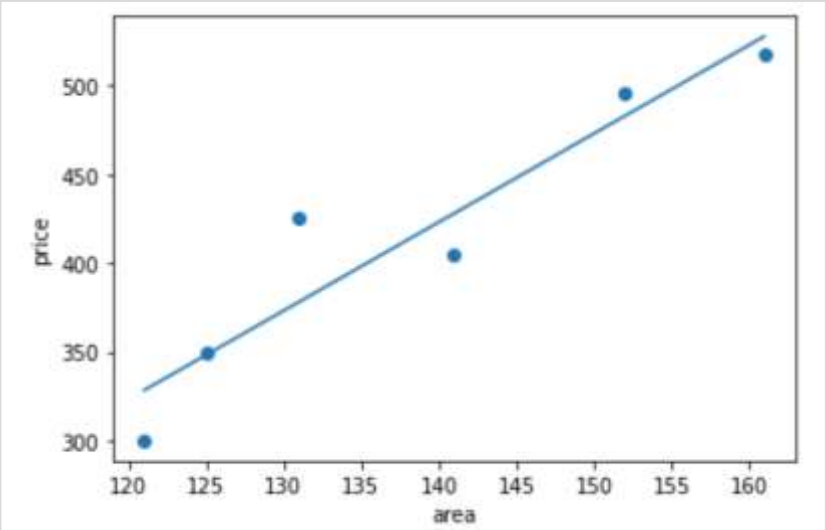
Output:

```
Slope: [4.98467124]
Intercept: -274.8769665187576
```

Input:

```
plt.scatter(x,y)
plt.xlabel("area") # X axis indicates the area.
plt.ylabel("price") # Y axis indicates the price.
plt.plot([x[0],x[-1]],[x[0]*w+b,x[-1]*w+b])
```

Output:



Step 5 Start a prediction task using the model.

Input:

```
testX = np.array([[130]])# A test sample with an area of 130
lr.predict(testX)
```

Output:

```
Array([373.13029447]) # The model predicts the house price of the sample.
```

1.2.2 Linear Regression Implementation (Expansion Experiment)

This experiment uses the **lr2_data.txt** dataset, which contains simulated house area and price data. To obtain the dataset, see the Lab Environment Setup.

Step 1 Import dependencies.

Input:

```
import numpy as np
import matplotlib.pyplot as plt
```

Step 2 Define the function for calculating gradients.

Input:

```
def generate_gradient(X, theta, y):
    sample_count = X.shape[0]
    # Calculate the gradient based on the matrix 1/m ∑(((h(x^i)-y^i)) x_j^i)
    return (1./sample_count)*X.T.dot(X.dot(theta)-y)
```

Step 3 Define the function for reading datasets.

Input:

```
def get_training_data(file_path):
    orig_data = np.loadtxt(file_path,skiprows=1) # Ignore the title in the first row of the dataset.
    cols = orig_data.shape[1]
    return (orig_data, orig_data[:, :cols - 1], orig_data[:, cols-1:])
```

Step 4 Define the function for initializing parameters.

Input:

```
# Initialize the θ array.
def init_theta(feature_count):
    return np.ones(feature_count).reshape(feature_count, 1)
```

Step 5 Define the function for implementing gradient descent.

Input:

```
def gradient_descending(X, y, theta, alpha):
    Jthetas= []  # Record the change trend of the cost function J(θ) to confirm the gradient descent is correct.
    # Calculate the loss function, which is equal to the square of the difference between the actual value and the predicted value: (y^i-h(x^i))^2
    Jtheta = (X.dot(theta)-y).T.dot(X.dot(theta)-y)
    index = 0
    gradient = generate_gradient(X, theta, y) # Calculate the gradient.
    while not np.all(np.absolute(gradient) <= 1e-5):  # End the calculation when the gradient is less than 0.00001.
        theta = theta - alpha * gradient
        gradient = generate_gradient(X, theta, y) # Calculate the new gradient.
        # Calculate the loss function, which is equal to the square of the difference between the actual value and the predicted value: (y^i-h(x^i))^2
        Jtheta = (X.dot(theta)-y).T.dot(X.dot(theta)-y)
        if (index+1) % 10 == 0:
            Jthetas.append((index, Jtheta[0]))  # Record the result every 10 calculations.
        index += 1
    return theta,Jthetas
```

Step 6 Define the function for visualizing the change curve of the loss function.

Input:

```
# Plot the loss function change curve.
def showJTheta(diff_value):
    p_x = []
    p_y = []
    for (index, sum) in diff_value:
        p_x.append(index)
        p_y.append(sum)
    plt.plot(p_x, p_y, color='b')
    plt.xlabel('steps')
    plt.ylabel('loss funtion')
    plt.title('step - loss function curve')
    plt.show()
```

Step 7 Define the function for visualizing data points and the fitted curve.

Input:

```
# Plot the actual data points and the fitted curve.
```

```
def showlinercurve(theta, sample_training_set):
    x, y = sample_training_set[:, 1], sample_training_set[:, 2]
    z = theta[0] + theta[1] * x
    plt.scatter(x, y, color='b', marker='x',label="sample data")
    plt.plot(x, z, 'r', color="r",label="regression curve")
    plt.xlabel('x')
    plt.ylabel('y')
    plt.title('liner regression curve')
    plt.legend()
    plt.show()
```

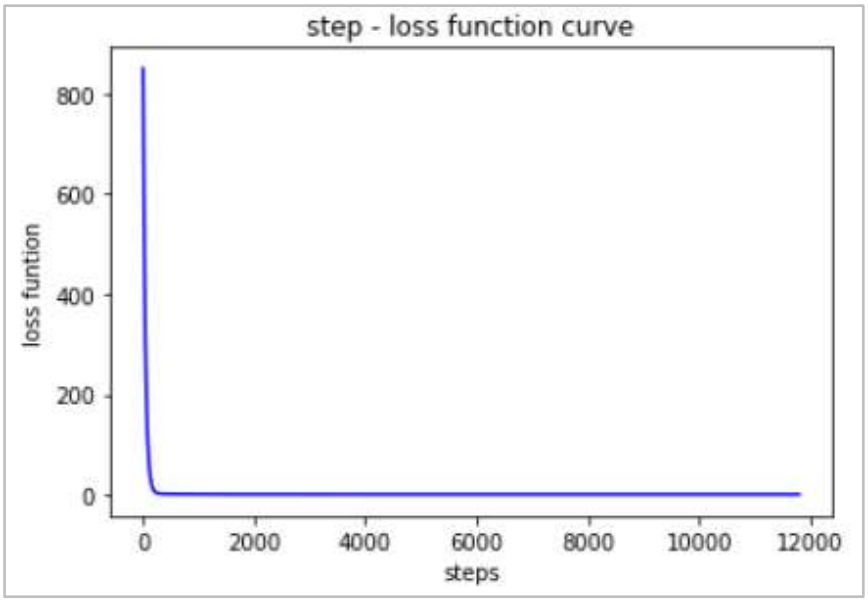
Step 8 Plot the final results.

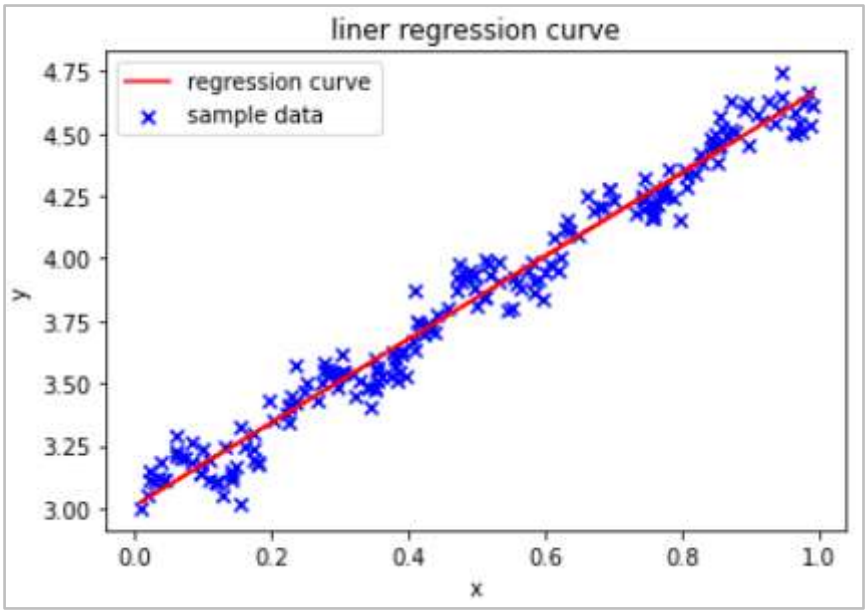
Input:

```
# Read the dataset.
training_data_include_y, training_x, y = get_training_data("./ML/02/lr2_data.txt")
# Obtain the numbers of samples and features, respectively.
sample_count, feature_count = training_x.shape
# Define the learning step α.
alpha = 0.01
# Initialize θ.
theta = init_theta(feature_count)
# Obtain the final parameter θ and cost.
result_theta,Jthetas = gradient_descending(training_x, y, theta, alpha)
# Display the parameter.
print("w:{}".format(result_theta[0][0]),"b:{}".format(result_theta[1][0]))
showJTheta(Jthetas)
showlinercurve(result_theta, training_data_include_y)
```

Output (ignore warnings):

```
w:3.0076279423997594 b:1.668677412281192
```





1.2.3 Logistic Regression

This experiment uses a custom dataset of house rent and area. The dataset is defined during the initial phase of the experiment.

Step 1 Import dependencies.

Input:

```
# Import StandardScaler from sklearn.preprocessing.
from sklearn.preprocessing import StandardScaler
# Import LogisticRegression from sklearn.linear_model.
from sklearn.linear_model import LogisticRegression
```

Step 2 Define the dataset.

Input:

```
# Each item in X denotes the rent and area.
# y indicates whether to rent the room (0: no; 1: yes).
X=[[2200,15],[2750,20],[5000,40],[4000,20],[3300,20],[2000,10],[2500,12],[12000,80],
    [2880,10],[2300,15],[1500,10],[3000,8],[2000,14],[2000,10],[2150,8],[3400,20],
    [5000,20],[4000,10],[3300,15],[2000,12],[2500,14],[10000,100],[3150,10],
    [2950,15],[1500,5],[3000,18],[8000,12],[2220,14],[6000,100],[3050,10]
  ]

y=[1,1,0,0,1,1,1,1,0,1,1,0,1,1,0,1,0,0,0,1,1,1,0,1,0,1,0,1,1,0]
```

Step 3 Preprocess data.

Standardize data to ensure that the variance of feature data in each dimension is 1 and the mean is 0. In this way, the prediction result is not dominated by large feature values of some dimensions.

Input:

```
ss = StandardScaler()
```

```
X_train = ss.fit_transform(X)
```

Display the standardized data.

Input:

```
print(X_train)
```

Output:

```
[[-0.60583897 -0.29313058]
 [-0.37682768 -0.09050576]
 [ 0.56003671  0.71999355]
 [ 0.14365254 -0.09050576]
 [-0.14781638 -0.09050576]
 [-0.68911581 -0.49575541]
 [-0.48092372 -0.41470548]
 [ 3.47472592  2.34099218]
 [-0.32269773 -0.49575541]
 [-0.56420055 -0.29313058]
 [-0.89730789 -0.49575541]
 [-0.27273163 -0.57680534]
 [-0.68911581 -0.33365555]
 [-0.68911581 -0.49575541]
 [-0.62665818 -0.57680534]
 [-0.10617796 -0.09050576]
 [ 0.56003671 -0.09050576]
 [ 0.14365254 -0.49575541]
 [-0.14781638 -0.29313058]
 [-0.68911581 -0.41470548]
 [-0.48092372 -0.33365555]
 [ 2.64195758  3.15149149]
 [-0.21027401 -0.49575541]
 [-0.29355084 -0.29313058]
 [-0.89730789 -0.69838024]
 [-0.27273163 -0.17155569]
 [ 1.80918923 -0.41470548]
 [-0.59751129 -0.33365555]
 [ 0.97642089  3.15149149]
 [-0.25191242 -0.49575541]]
```

Step 4 Fit the data.

Input:

```
# Use the fit method of LogisticRegression to train model parameters.
lr = LogisticRegression()
lr.fit(X_train, y)
```

Output:

```
LogisticRegression()
```

Step 5 Predict the data.

Input:

```
testX = [[2000,8]]
X_test = ss.transform(testX)
print("Value to be predicted: ",X_test)
label = lr.predict(X_test)
print("predicted label = ", label)
# Output the predicted probability.
prob = lr.predict_proba(X_test)
print("probability = ",prob)
```

Output:

```
Value to be predicted: [[-0.68911581 -0.57680534]]
predicted label =    [1]
probability =    [[0.41886952 0.58113048]]
```

1.2.4 Decision Tree

This experiment uses the **tennis.txt** dataset, which contains 14 samples. Each sample contains weather-related features and whether it is suitable for tennis.

Step 1 Import dependencies.

Input:

```
import pandas as pd
import numpy as np
from sklearn import tree
import pydotplus
```

Step 2 Define the function for generating a decision tree.

Input:

```
# Generate a decision tree.
def createTree(trainingData):
    data = trainingData.iloc[:, :-1]    # Feature matrix
    labels = trainingData.iloc[:, -1]    # Labels
    trainedTree = tree.DecisionTreeClassifier(criterion="entropy")    # Decision tree classifier
    trainedTree.fit(data, labels)    # Train the model.
    return trainedTree
```

Step 3 Define the function for saving the generated tree diagram.

Input:

```
def showtree2pdf(trainedTree,finename):
    dot_data = tree.export_graphviz(trainedTree, out_file=None) # Export the tree in Graphviz format.
    graph = pydotplus.graph_from_dot_data(dot_data)
    graph.write_pdf(finename)    # Save the tree diagram to the local machine in PDF format.
```

Step 4 Define the function for generating vectorized data.

In the function, **pd.Categorical(list).codes** obtains the sequence number list corresponding to the original data, so as to convert the categorical information into numeric information.

Input:

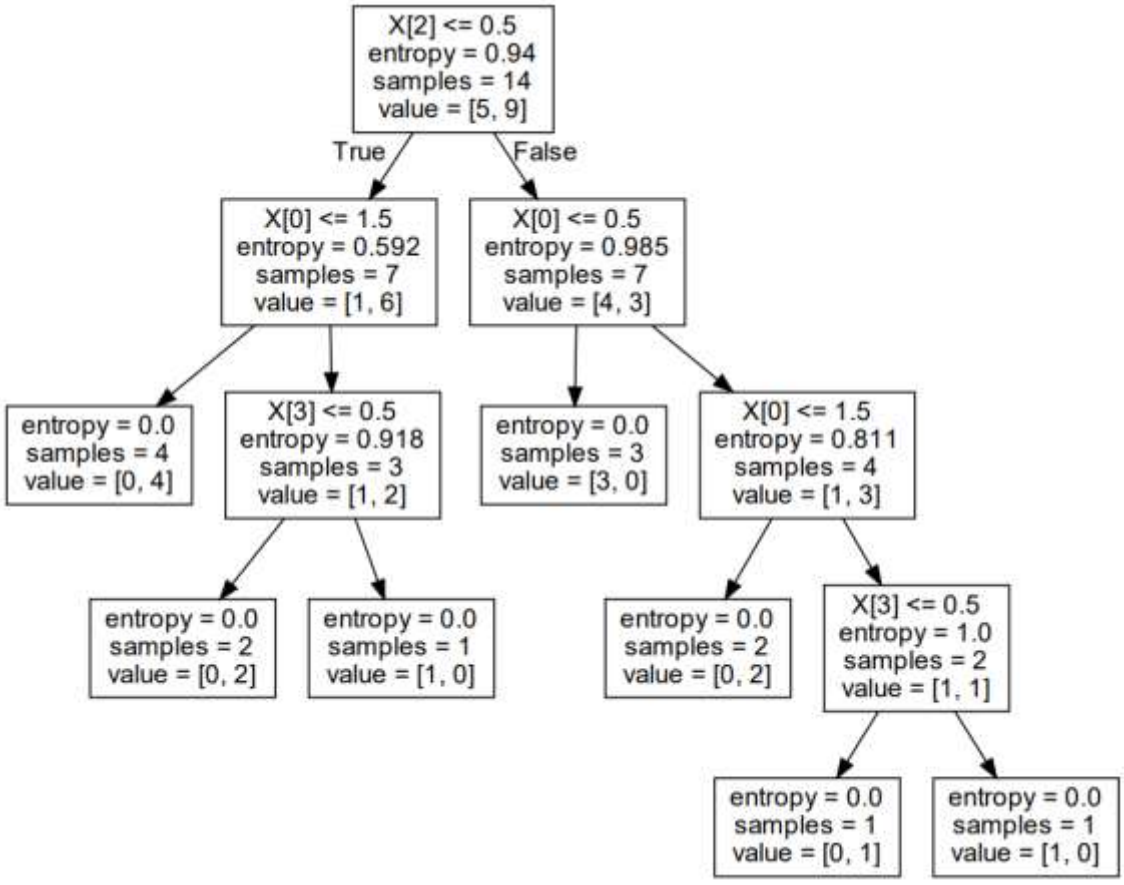
```
def data2vectoc(data):
    names = data.columns[:-1]
    for i in names:
        col = pd.Categorical(data[i])
        data[i] = col.codes
    return data
```

Step 5 Invoke the function for prediction.

Input:

```
data = pd.read_table("./ML/tennis.txt",header=None,sep='\t') # Read training data.
trainingvec=data2vectoc(data) # Vectorize data.
decisionTree=createTree(trainingvec) # Create a decision tree.
showtree2pdf(decisionTree,"tennis.pdf") # Plot the decision tree.
```

A decision tree diagram named **tennis.pdf** is generated on the local machine.



The file content is a visualized display of the decision tree. In the diagram, **X[2]** is the third feature variable (humidity); **X[0]** is the first feature variable (weather); **X[3]** is the fourth feature variable (wind); **entropy** is the entropy value of the node; and **samples** is the number of samples in the node, for example, 14 in the first node (root node) indicates the number of samples in the training set; and **value** indicates the numbers of samples of different types, for example, in the root node, 5 indicates the number of "no" samples, and 9 indicates the number of "yes" samples.

Predict a new sample. Input:

```
testVec = [0,0,1,1] # Weather is sunny, temperature is low, humidity is high, and wind is strong.
print(decisionTree.predict(np.array(testVec).reshape(1,-1))) # Predict.
```

Output:

```
['N']
```

1.2.5 K-means Algorithm Implementation

Step 1 Import dependencies.

make_blobs is used to generate the dataset required for this experiment, **matplotlib** is used for data visualization, and **KMeans** is used for fitting data based on the K-means algorithm.

Input:

```
from sklearn.datasets import make_blobs
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
```

Step 2 Generate the dataset.

make_blobs() is used to generate a dataset for a clustering task by returning the generated dataset and labels. Note: the K-means algorithm is often used to process unlabeled datasets in production. The generated dataset contains labels only to facilitate understanding.

Input:

```
X, y = make_blobs(n_samples=500,n_features=2,centers=4,random_state=1)
```

n_samples is the number of samples, **n_features** is the number of features of each sample (the dimension of data), **centers** is the number of categories, and **random_state** is the seed for the random number generator. The seed ensures that data generated each time is the same.

Display the dimension information of the generated dataset.

Input:

```
Print("Dimension of X is {}".format(X.shape))
Print("Dimension of y is {}".format(y.shape))
```

Output:

```
Dimension of X is (500, 2)
Dimension of y is (500,).
```

The dataset contains 500 sample points, and each sample point contains two features.

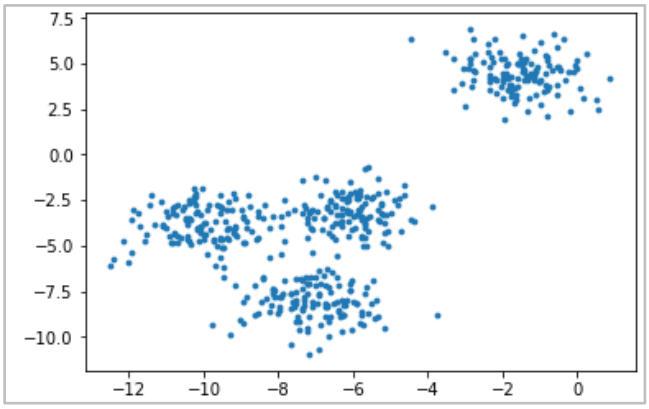
Step 3 Draw scatter graphs.

First, directly draw a scatter graph of the dataset. The labels generated during dataset generation are not considered for now. The graph contains four different clusters.

Input:

```
fig, ax1 = plt.subplots(1)
ax1.scatter(X[:, 0], X[:, 1]
                ,marker='o'   # Set the shape of the point to circle.
                ,s=8   # Set the size of the point.
               )
plt.show()
```

Output:



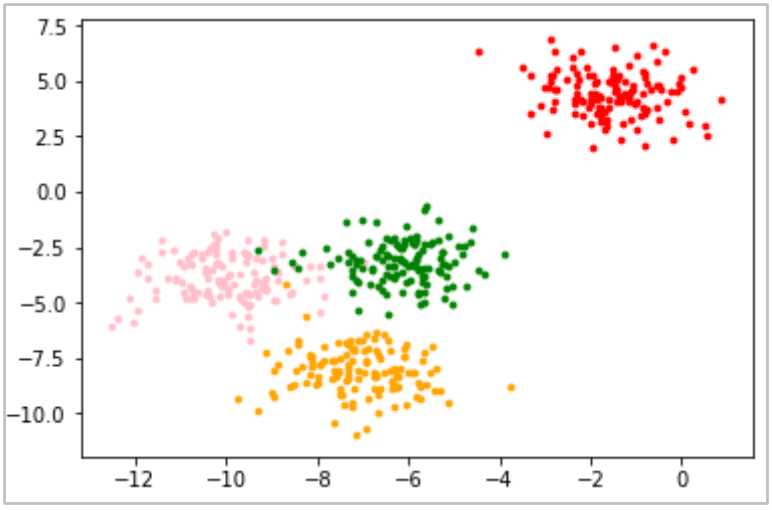
Next, draw a scatter graph where points in different colors correspond to the generated labels.

Input:

```
color = ["red","pink","orange","green"]
fig, ax1 = plt.subplots(1)

for i in range(4):
    ax1.scatter(X[y==i, 0], X[y==i, 1]    # Draw the color based on the label.
                ,marker='o'    # Set the shape of the point to circle.
                ,s=8    # Set the size of the point.
                ,c=color[i]
               )
plt.show()
```

Output:



Step 4 Perform k-means clustering.

Use **sklearn.cluster.KMeans** provided by scikit-learn to implement K-means clustering. First, aggregate the data samples into three types.

Input:

```
n_clusters = 3
cluster1 = KMeans(n_clusters=n_clusters,random_state=3).fit(X)
```

View the label of each sample point after clustering.

Input:

```
y_pred1 = cluster1.labels_
print(y_pred1)
```

Output:

```
[0 0 2 1 2 1 2 2 2 2 0 0 2 1 2 0 2 0 1 2 2 2 2 1 2 2 1 1 2 2 0 1 2 0 2 0 2
 2 0 2 2 2 2 1 2 2 0 2 2 2 1 1 1 2 2 2 2 0 2 2 2 2 2 1 1 2 2 1 2 0 2 2 2 2 0 2 2 0
 2 2 0 2 2 2 2 1 1 2 1 1 2 2 1 2 2 1 2 2 1 0 2 2 1 0 0 2 0 1 1 0 1 2 1 2 2 1 1 2 2
 0 1 2 1 2 1 2 1 2 2 2 0 0 2 2 2 1 0 0 2 1 2 2 2 2 2 0 1 2 1 1 2 0 2 1 1 1 2 2
 0 0 2 2 1 0 1 2 2 2 2 2 2 2 2 2 1 0 0 0 2 1 0 2 2 2 0 1 2 2 2 2 0 2 2 2 1 0 0
 2 2 0 0 2 1 1 0 0 2 1 2 0 0 1 0 2 1 2 2 0 2 2 0 2 2 2 2 2 0 2 2 2 1 2 1 2 0
 2 2 2 2 2 1 2 1 0 2 0 2 1 1 2 0 1 0 2 2 2 0 0 0 0 2 2 0 2 2 1 1 2 2 1 2 2 2
 1 2 1 2 2 1 2 0 0 2 2 2 2 1 1 2 1 2 0 1 0 1 0 0 1 0 1 1 2 2 2 2 2 2 2 0 1
 0 0 0 2 2 2 0 2 0 0 2 0 0 2 1 0 2 2 1 1 2 0 1 1 2 0 1 1 2 2 1 2 2 0 0 1 2
 0 2 1 1 2 2 2 0 2 1 1 2 1 1 1 1 0 0 2 1 2 2 0 1 2 1 2 1 2 2 2 1 2 2 0 1 0
 0 0 0 0 0 2 0 1 0 1 1 2 1 2 2 2 0 1 2 1 2 0 2 2 0 2 2 1 1 0 2 2 1 2 2 0 0
 2 0 2 2 0 2 0 2 1 0 1 2 2 1 2 2 1 0 2 1 1 2 2 2 2 0 1 0 2 1 0 0 0 2 1 2 0
 2 2 2 2 0 2 2 2 2 2 2 0 2 2 0 2 1 2 1 2 2 2 1 1 1 2 2 2 0 2 1 2 0 1 0 1 0
 2 1 1 0 2 2 0 2 2 2 0 2 1 2 2 0 0 0 2]
```

Print the centroid of each cluster.

Input:

```
centroid1 = cluster1.cluster_centers_
print(centroid1)
```

Output:

```
 [[-7.09306648 -8.10994454]
 [-1.54234022  4.43517599]
 [-8.0862351  -3.5179868 ]]
```

Visualize the clustering result.

Input:

```
color = ["red","pink","orange","gray"]

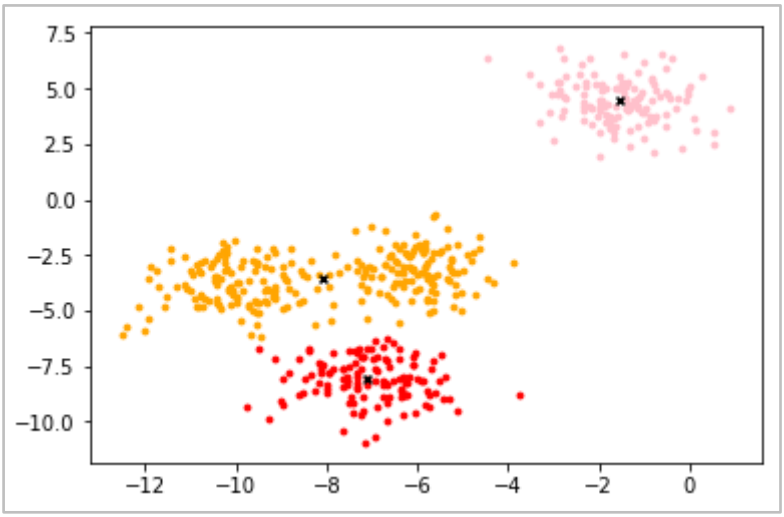
fig, ax1 = plt.subplots(1)

for i in range(n_clusters):
    ax1.scatter(X[y_pred1==i, 0], X[y_pred1==i, 1]
                ,marker='o' # Set the shape of the point to circle.
                ,s=8 # Set the size of the point.
                ,c=color[i]
               )

ax1.scatter(centroid1[:,0],centroid1[:,1]
            ,marker="x"
            ,s=15
            ,c="black")
```

```
plt.show()
```

Output:



The graph shows that the data samples are aggregated into three clusters, and the centroid of each cluster is represented by **x**.

Step 5 Perform k-means clustering again.

Next, perform the preceding steps to use k-means clustering to aggregate data samples into four types.

Input:

```
n_clusters = 4
cluster2 = KMeans(n_clusters=n_clusters,random_state=0).fit(X)
y_pred2 = cluster2.labels_
centroid2 = cluster2.cluster_centers_
print("Centroid: {}".format(centroid2))
```

Output:

```
Centroid: [[ -6.08459039   -3.17305983]
 [ -1.54234022    4.43517599]
 [ -7.09306648   -8.10994454]
 [-10.00969056   -3.84944007]]
```

Visualize the clustering result.

Input:

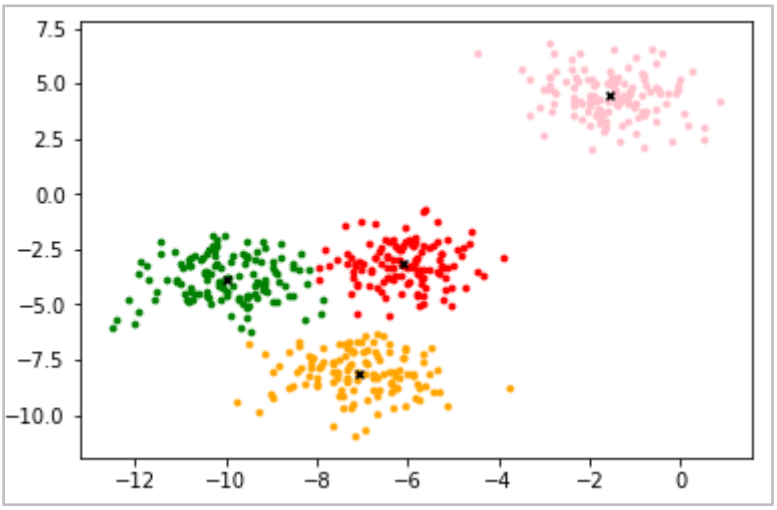
```
color = ["red","pink","orange","green"]

fig, ax1 = plt.subplots(1)

for i in range(n_clusters):
    ax1.scatter(X[y_pred2==i, 0], X[y_pred2==i, 1]
                ,marker='o' # Set the shape of the point to circle.
                ,s=8 # Set the size of the point.
                ,c=color[i]
               )
```

```
ax1.scatter(centroid2[:,0],centroid2[:,1]
            ,marker="x"
            ,s=15
            ,c="black")
plt.show()
```

Output:



The graph shows that the data samples are aggregated into four clusters, and the centroid of each cluster is represented by **x**. Compare the scatter graphs generated based on the original data and data aggregated into four types. It can be seen that many sample points are aggregated into wrong clusters.

1.3 Question

How to implement K-means from scratch using Python?

1.4 Summary

This lab introduces the machine learning implementation process, including data import, segmentation, standardization, as well as models and hyperparameters. Common machine learning algorithms are implemented based on scikit-learn to help you better understand how to build and use machine learning models.

Huawei Certification-AI Series Training

HCIA-AI

Deep Learning and AI Development Framework

Lab Guide

Issue: 4.0



HUAWEI TECHNOLOGIES CO., LTD

Copyright © 2025 HUAWEI TECHNOLOGIES CO., LTD. All Rights Reserved.

No part of this document may be reproduced or transmitted in any form or by any means without prior written consent of Huawei Technologies Co., Ltd.

Trademark Statement


and other Huawei trademarks are trademarks of Huawei Technologies Co., Ltd.

All other trademarks and registered trademarks mentioned in this document are the property of their respective holders.

Notice

The purchased products, services and features are stipulated by the commercial contract made between Huawei and the customer. All or partial products, services and features described in this document may not be within the purchased scope or the usage scope. Unless otherwise specified in the contract, all statements, information, and recommendations in this document are provided "AS IS" without warranties, guarantees or representations of any kind, either express or implied.

The information in this document is subject to change without notice. Every effort has been made in the preparation of this document to ensure accuracy of the contents, but all statements, information, and recommendations in this document do not constitute a warranty of any kind, express or implied.

Huawei Technologies Co., Ltd.

Address:	Huawei Industrial Base Bantian, Longgang, Shenzhen 518129 People's Republic of China
Website:	https://e.huawei.com

Huawei Certification System

Huawei Certification is an integral part of the company's Platform + Ecosystem strategy. It supports the new Information and Communications Technology (ICT) infrastructure that features Cloud-Pipe-Device synergy and covers all the ICT domains, including certification for the ICT architecture and its applications, as well as certification for cloud service and platform.

The certification system offers three levels: Huawei Certified ICT Associate (HCIA), Huawei Certified ICT Professional (HCIP), and Huawei Certified ICT Expert (HCIE), to meet the learning and advancement requirements of ICT practitioners.

Huawei certifications cover all ICT fields and adapt to the industry trend of ICT convergence. With its leading talent development system and certification standards, Huawei is committed to fostering new ICT talent in the digital era, and building a healthy ICT talent ecosystem.

The Huawei HCIA-AI V4.0 certification system is intended to train and certify engineers who are capable of designing, developing, and innovating AI products and solutions by using machine learning and deep learning algorithms.

With HCIA-AI V4.0 certification, you are proved to:
Understand the history of AI development.
Master algorithms related to traditional machine learning and deep learning.
Be capable of building, training, and deploying neural networks by using development framework PyTorch.
Be competent in sales, marketing, product manager, project management, technical support, and other AI positions.

The Huawei certification system introduces you to the industry and market, helps you in innovation, and enables you to stand atop the AI frontiers.


HCIE
Huawei Certified ICT Expert
HCIP
Huawei Certified ICT Professional
HCIA
Huawei Certified ICT Associate
Project Management
Cloud Platform & Services
Basic Software & Hardware
ICT Infrastructure
Cloud Platform & Services Domain
Basic Software & Hardware Domain
IT Infrastructure Domain
Datacom Domain
Optical Domain
Wireless Domain
Cloud Computing
Cloud Service
Big Data
GaussDB
IoT
openEuler
openGauss
HarmonyOS
AI
Computing
Storage
Collaboration
Intelligent Vision
Digital Power
MDC
Datacom
WLAN
Security
Transmission
Access
5G
LTE

About This Document

Overview

This document is applicable to the candidates who are preparing for the HCIA-AI exam or readers who want to understand AI basics and basics of PyTorch programming.

Description

This lab guide introduces the following one experiment:

- Use Python and basic third-party libraries to implement a three-layer fully-connected neural network, including the network structure, gradient calculation, and parameter optimization. Use different optimizers for training and compare the loss function changes. This experiment provides reference for trainees to understand the overall process of deep learning and the basic implementation of each phase.

Link to the data and code files used in the experiment:

https://certification-data.obs.cn-north-4.myhuaweicloud.com/ENG/HCIA-AI/V4.0/Dataset.zip

Background Knowledge Required

This document is for Huawei's basic certification. To better understand this document, ensure that you meet the following requirements:

- Have basic knowledge about Python.

Contents

About This Document 2
Overview 2
Description 2
Background Knowledge Required 2
1 Building a Fully-Connected Neural Network from Scratch 4
1.1 Experiment Introduction 4
1.1.1 About This Experiment 4
1.1.2 File Parsing 4
1.1.3 Experiment Process 5
1.2 Training Code Parsing 6
1.2.1 Main Program 6
1.2.2 Loading the Dataset 7
1.2.3 Building the Network Structure 10
1.2.4 Starting Training 13
1.2.5 Drawing the Graph 13
1.2.6 Experiment Result 14
1.3 Questions 14

1 Building a Fully-Connected Neural Network from Scratch

1.1 Experiment Introduction

1.1.1 About This Experiment

This experiment aims to use the Python programming language and basic libraries such as NumPy to build a multi-layer fully-connected neural network, and use the MNIST dataset for training and testing to recognize handwritten digital images.

The purpose of the experiment is to build, train, and test neural networks without using any deep learning framework. By gradually implementing each function, you can gain a deep understanding of the implementation principle and training and testing process of neural networks.

The MNIST dataset is a widely used handwritten digit recognition dataset. It contains a large number of handwritten digit images which are used to train and test various image processing and recognition technologies, especially handwritten digit recognition.

The MNIST dataset consists of handwritten images of 10 digits from 0 to 9, and is divided into a training set and a test set. The training set contains 60,000 samples, and the test set contains 10,000 samples. Each sample is a 28 x 28 pixel grayscale image with pixel values ranging from 0 (white) to 255 (black). These images were extracted from a project of the National Institute of Standards and Technology (NIST). The purpose of this project is to collect a large amount of handwritten digital data for the research and development of optical character recognition (OCR) systems.

1.1.2 File Parsing

Decompress the file after downloading the code and dataset files. The file directories are as follows:

```
|— MNIST
|    |— functions.py
|    |— gradient.py
|    |— layers.py
|    |— load_mnist.py
|    |— multi_layer_net.py
|    |— train_mnist.py
|    |— optimizer.py
|    |— util.py
|    |— t10k-images-idx3-ubyte
|    |— t10k-labels-idx1-ubyte
|    |— train-images-idx3-ubyte
```

| |— train-labels-idx1-ubyte

Data files:

train-images-idx3-ubyte: image data of the training set, including 60,000 28 x 28 pixel grayscale images.

train-labels-idx1-ubyte: image labels of the training set, including 60,000 digital labels.

t10k-images-idx3-ubyte: image data of the test set, including 10,000 28 x 28 pixel grayscale images.

t10k-labels-idx1-ubyte: image labels of the test set, including 10,000 digital labels.

mnist.pkl: serialized by the Python pickle module and contains the training set and validation set of the MNIST dataset.

train_mnist.py: code entry for the experiment, and the main training program.

load_mnist.py: contains implementation of the data loading function.

multi_layer_net.py: implementation of a multi-layer neural network.

layers.py: implementation of basic layer functions.

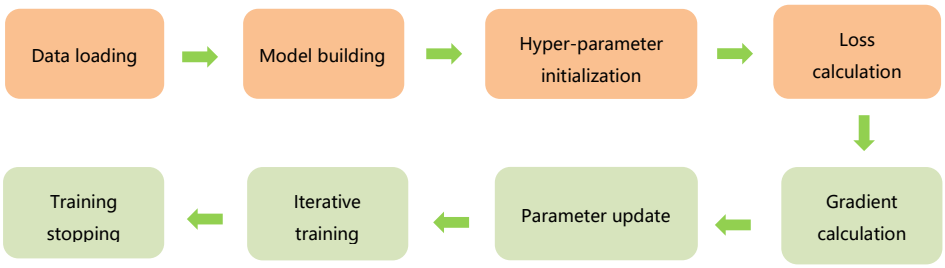
gradient.py: calculates the implementation of the gradient function.

optimizer.py: contains implementation of multiple optimizers.

util.py: contains implementation of the data conversion function.

functions.py: contains implementation of multiple activation functions.

1.1.3 Experiment Process



1.2 Training Code Parsing

1.2.1 Main Program

train_mnist.py is the main program of the project. The code is as follows:

```
# coding: utf-8
import matplotlib.pyplot as plt
from load_mnist import load_mnist
from util import smooth_curve
from multi_layer_net import MultiLayerNet
from optimizer import *


# 0: Read the MNIST data==========.
(x_train, t_train), (x_test, t_test) = load_mnist(normalize=True)

train_size = x_train.shape[0]
batch_size = 128
max_iterations = 1000


# 1: Set the experiment==========
optimizers = {} # Create a dictionary for storing optimizers.
optimizers['SGD'] = SGD()
optimizers['Momentum'] = Momentum()
optimizers['AdaGrad'] = AdaGrad()
optimizers['Adam'] = Adam()


networks = {} # Create a dictionary for storing network structures.
train_loss = {} # Create a dictionary for storing loss functions.
for key in optimizers.keys():
    networks[key] = MultiLayerNet(
        input_size=784, hidden_size_list=[100, 100, 100, 100],
        output_size=10)
    train_loss[key] = []


# 2: Start training==========
for i in range(max_iterations):
    batch_mask = np.random.choice(train_size, batch_size)
    x_batch = x_train[batch_mask]
    t_batch = t_train[batch_mask]

    for key in optimizers.keys():
        grads = networks[key].gradient(x_batch, t_batch)
        optimizers[key].update(networks[key].params, grads)

        loss = networks[key].loss(x_batch, t_batch)
        train_loss[key].append(loss)

    if i % 100 == 0:
        print( "===========" + "iteration:" + str(i) + "===========")
        for key in optimizers.keys():
```

```
            loss = networks[key].loss(x_batch, t_batch)
            print(key + ":" + str(loss))


# 3: Draw a graph==========
markers = {"SGD": "o", "Momentum": "x", "AdaGrad": "s", "Adam": "D"}
x = np.arange(max_iterations)
for key in optimizers.keys():
    plt.plot(x, smooth_curve(train_loss[key]), marker=markers[key], markevery=100, label=key)
plt.xlabel("iterations")
plt.ylabel("loss")
plt.ylim(0, 1)
plt.legend()
plt.show()
```

1.2.2 Loading the Dataset

The compressed package of the MNIST dataset has been downloaded. Use the load_mnist function to load the dataset. This function loads the dataset to the pickle file. To read the dataset, you only need to load the pickle file.

```
# Read the MNIST data.
(x_train, t_train), (x_test, t_test) = load_mnist(normalize=True)

train_size = x_train.shape[0]    # Obtain the number of training images.
batch_size = 128                 # Set batch_size.
max_iterations = 1000            # Set the maximum number of iterations.
```

Analyze the load_mnist function.

Open the **load_mnist.py** file.

The **load_mnist.py** file declares the following variables:

```
# key_file stores the name of the dataset package as a dictionary.
key_file = {
    'train_img':'train-images-idx3-ubyte.gz',
    'train_label':'train-labels-idx1-ubyte.gz',
    'test_img':'t10k-images-idx3-ubyte.gz',
    'test_label':'t10k-labels-idx1-ubyte.gz'
}

# The dataset_dir variable is used to obtain the absolute path of the folder where the current code file is
located.
dataset_dir = os.path.dirname(os.path.abspath(__file__))

# save_file declares the path of the .pkl file through character string concatenation.
save_file = dataset_dir + "/mnist.pkl"
```

The body of the load_mnist function is as follows:

```
def load_mnist(normalize=True, flatten=True, one_hot_label=False):
    """Read the MNIST dataset.
    Parameters
    ----------
    normalize: normalizes the pixel value of the image to a value within the range from 0.0 to 1.0.
```

```
    one_hot_label :
        When one_hot_label is set to True, the label is returned as a one-hot array.
        A one-hot array is similar to [0,0,1,0,0,0,0,0,0,0].
    flatten: indicates whether to expand the image into a one-dimensional array.

    Returns
    -------
    (training image, training label), (test image, test label)
"""
# Check whether the pickle file for storing the dataset exists. If the file does not exist, use the compressed dataset to create the file.
    if not os.path.exists(save_file):
        init_mnist()
    # Use the pickle file to load the dataset.
    with open(save_file, 'rb') as f:
        dataset = pickle.load(f)

    """Preprocess the dataset"""
    if normalize:
        for key in ('train_img', 'test_img'):
            dataset[key] = dataset[key].astype(np.float32) # Convert to the np.float32 data type.
            dataset[key] /= 255.0 # Convert the pixel value to a value within the range of 0 to 1.

    if one_hot_label:     # Convert the label to a one-hot array.
        dataset['train_label'] = _change_one_hot_label(dataset['train_label'])
        dataset['test_label'] = _change_one_hot_label(dataset['test_label'])

    if not flatten:     # Check whether the data needs to be flattened. If the data needs to be flattened, the (784,) format is retained. Otherwise, the data is converted into (28, 28).
         for key in ('train_img', 'test_img'):
            dataset[key] = dataset[key].reshape(-1, 1, 28, 28)

    return (dataset['train_img'], dataset['train_label']), (dataset['test_img'], dataset['test_label'])
```

Analyze other functions called in the load_mnist function body.

init_mnist function:

```
def init_mnist():
    dataset = _convert_numpy () # Convert the dataset into the NumPy format.
    print("Creating pickle file ...")
    with open(save_file, 'wb') as f:       # Open the .pkl file in binary writeable mode.
        pickle.dump (dataset, f, -1) # Write the dataset into the .pkl file. After the operation is complete, the file is automatically closed.
print("Done!")
```

_convert_numpy function:

```
def _convert_numpy():
dataset = {} # Create an empty dictionary.
# Save the dataset in the dictionary by name.
    dataset['train_img'] =    _load_img(key_file['train_img'])
    dataset['train_label'] = _load_label(key_file['train_label'])
    dataset['test_img'] = _load_img(key_file['test_img'])
dataset['test_label'] = _load_label(key_file['test_label'])
```

```
return dataset
```

_load_label and _load_img functions:

```
# Load the label.
def _load_label(file_name):
    file_path = dataset_dir + "/" + file_name    # Absolute path of the file to which the label is written.

    print("Converting " + file_name + " to NumPy Array ...")
with gzip.open(file_path, 'rb') as f:     # Open the compressed dataset.
        """
        np.frombuffer():
        This is a function in the NumPy library and is used to create a NumPy array from the byte buffer.
        The first parameter is the buffer. In this example, it reads the file content through f.read ().
        f.read():
        f is an opened file object.
        f.read () is a file object method. It reads all the remaining content of a file and returns the content in
bytes.
        np.uint8:
        Specifies that the data in the buffer read by the np.frombuffer () function should be interpreted as
uint8.
        offset=8:
        This parameter specifies the number of bytes at the beginning of the buffer that should be skipped
before data parsing.
        It specifies that the function should skip the first eight bytes of the buffer and start creating an array
from the ninth byte.
        """
        labels = np.frombuffer(f.read(), np.uint8, offset=8)
    print("Done")

    return labels
# The structure of the _load_img function is the same as that of the _load_label function. The _load_img
function is used to load data (images).
def _load_img(file_name):
    file_path = dataset_dir + "/" + file_name

    print("Converting " + file_name + " to NumPy Array ...")
    with gzip.open(file_path, 'rb') as f:
        data = np.frombuffer(f.read(), np.uint8, offset=16)
    data = data.reshape(-1, 784)
    print("Done")

return data
```

_change_one_hot_label function:

```
def _change_one_hot_label(X):
    T = np.zeros((X.size, 10))     # Initialize the all-zero matrix T with the shape of (X.size, 10).
    for idx, row in enumerate(T):
        row[X[idx]] = 1 # Set the corresponding label position in one-hot to 1.
    # T = np.eye(10)[X]     #The preceding code can be replaced with this row.
return T
```

1.2.3 Building the Network Structure

```
# 1. Set multiple optimizers and analyze the loss changes when different optimizers are used.
optimizers = {}
optimizers['SGD'] = SGD()
optimizers['Momentum'] = Momentum()
optimizers['AdaGrad'] = AdaGrad()
optimizers['Adam'] = Adam()

#    Build the network model.
networks = {} # Save the network model.
train_loss = {} # Create a dictionary to store the loss values of different optimizers.
for key in optimizers.keys():
      networks[key] = MultiLayerNet(
            input_size=784, hidden_size_list=[100, 100, 100, 100],
            output_size=10)
train_loss[key] = []
```

Parse the MultiLayerNet class.

```
class MultiLayerNet:
      """Fully-connected multi-layer neural network

      Parameters
      ----------
      input_size: input size (784 for MNIST)
      hidden_size_list: list of the number of neurons at the hidden layer (for example, [100, 100, 100])
      output_size: output size (10 for MNIST)
      activation : 'relu' or 'sigmoid'
      weight_init_std: standard deviation of the weight (for example, 0.01).
            Set the initial value of he when relu or he is specified.
            Set the initial value of xavier when sigmoid or xavier is specified.
      weight_decay_lambda: strength of weight decay (L2 norm)
      """
      def __init__(self, input_size, hidden_size_list, output_size,
                        activation='relu', weight_init_std='relu', weight_decay_lambda=0):
          #Configure the network.
            self.input_size = input_size
            self.output_size = output_size
            self.hidden_size_list = hidden_size_list
            self.hidden_layer_num = len(hidden_size_list)
            self.weight_decay_lambda = weight_decay_lambda
            self.params = {}

            # Initialize the weight.
            self.__init_weight(weight_init_std)

            # Generate the layers.
            activation_layer = {'sigmoid': Sigmoid, 'relu': Relu}
            self.layers = OrderedDict()
            for idx in range(1, self.hidden_layer_num+1):
                  self.layers['Affine' + str(idx)] = Affine(self.params['W' + str(idx)],
                                                                  self.params['b' + str(idx)])
                  self.layers['Activation_function' + str(idx)] = activation_layer[activation]()
```

```
        idx = self.hidden_layer_num + 1
        self.layers['Affine' + str(idx)] = Affine(self.params['W' + str(idx)],
            self.params['b' + str(idx)])

        self.last_layer = SoftmaxWithLoss()

    def __init_weight(self, weight_init_std):
        """Set the initial weight.

        Parameters
        ----------
        weight_init_std: standard deviation of the weight (for example, 0.01).
            Set the initial value of he when relu or he is specified.
            Set the initial value of xavier when sigmoid or xavier is specified.
        """
        all_size_list = [self.input_size] + self.hidden_size_list + [self.output_size]
        for idx in range(1, len(all_size_list)):
            scale = weight_init_std
            if str(weight_init_std).lower() in ('relu', 'he'):
                scale = np.sqrt(2.0 / all_size_list[idx - 1])  # Recommended initial value when ReLU is
used.
            elif str(weight_init_std).lower() in ('sigmoid', 'xavier'):
                scale = np.sqrt(1.0 / all_size_list[idx - 1])  # Recommended initial value when sigmoid is
used.

            self.params['W' + str(idx)] = scale * np.random.randn(all_size_list[idx-1], all_size_list[idx])
            self.params['b' + str(idx)] = np.zeros(all_size_list[idx])

    def predict(self, x):
        for layer in self.layers.values():
            x = layer.forward(x)

        return x

    def loss(self, x, t):
        """Function for calculating the loss
        Parameters
        ----------
        x: input data
        t: correct label

        Returns
        -------
        Value of the loss function
        """
        y = self.predict(x)

        weight_decay = 0
        for idx in range(1, self.hidden_layer_num + 2):
            W = self.params['W' + str(idx)]
            weight_decay += 0.5 * self.weight_decay_lambda * np.sum(W ** 2)

        return self.last_layer.forward(y, t) + weight_decay
```

```
    def accuracy(self, x, t):
        y = self.predict(x)
        y = np.argmax(y, axis=1)
        if t.ndim != 1 : t = np.argmax(t, axis=1)

        accuracy = np.sum(y == t) / float(x.shape[0])
        return accuracy

    def numerical_gradient(self, x, t):
        """Gradient calculation (numerical differentiation)
        Parameters
        ----------
        x: input data
        t: correct label

        Returns
        -------
        Dictionary variable with the gradient of each layer.
            grads['W1'], grads['W2']...: weight of each layer
            grads['b1'], grads['b2']...: offset of each layer
        """
        loss_W = lambda W: self.loss(x, t)
        grads = {}
        for idx in range(1, self.hidden_layer_num+2):
            grads['W' + str(idx)] = numerical_gradient(loss_W, self.params['W' + str(idx)])
            grads['b' + str(idx)] = numerical_gradient(loss_W, self.params['b' + str(idx)])

        return grads

    def gradient(self, x, t):
        """Gradient calculation (error backpropagation method)
        Parameters
        ----------
        x: input data
        t: teacher label

        Returns
        -------
        Dictionary variable with the gradient of each layer.
            grads['W1'], grads['W2']...: weight of each layer
            grads['b1'], grads['b2']...: offset of each layer
        """
        # forward
        self.loss(x, t)

        # backward
        dout = 1
        dout = self.last_layer.backward(dout)

        layers = list(self.layers.values())
        layers.reverse()
        for layer in layers:
            dout = layer.backward(dout)

        # Settings
```

```
        grads = {}
        for idx in range(1, self.hidden_layer_num+2):
            grads['W' + str(idx)] = self.layers['Affine' + str(idx)].dW + self.weight_decay_lambda *
self.layers['Affine' + str(idx)].W
            grads['b' + str(idx)] = self.layers['Affine' + str(idx)].db

        return grads
```

1.2.4 Starting Training

```
# 2: Start training==========
for i in range(max_iterations):
    batch_mask = np.random.choice(train_size, batch_size) # Generate a random number.
    x_batch = x_train[batch_mask]     # Use a random number as the subscript and select the training data.
    t_batch = t_train[batch_mask]     # Use a random number as the subscript and select the
corresponding label.

  # Parameter optimization
    for key in optimizers.keys():
        grads = networks[key].gradient(x_batch, t_batch)
        optimizers[key].update(networks[key].params, grads)

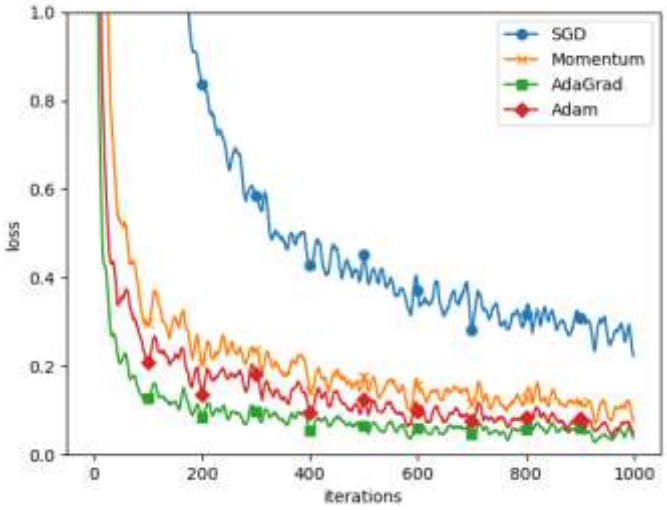
        loss = networks[key].loss(x_batch, t_batch)
        train_loss[key].append(loss)

    if i % 100 == 0:
        print( "===========" + "iteration:" + str(i) + "===========")
        for key in optimizers.keys():
            loss = networks[key].loss(x_batch, t_batch)
            print(key + ":" + str(loss))
```

1.2.5 Drawing the Graph

```
markers = {"SGD": "o", "Momentum": "x", "AdaGrad": "s", "Adam": "D"}
x = np.arange(max_iterations)
for key in optimizers.keys():
    plt.plot(x, smooth_curve(train_loss[key]), marker=markers[key], markevery=100, label=key)
plt.xlabel("iterations")
plt.ylabel("loss")
plt.ylim(0, 1)
plt.legend()
plt.show()
```

1.2.6 Experiment Result



1.3 Questions

1. How do I change the learning rate in the code? Use different learning rate setting policies and observe the loss curve.
2. Add a test module to the code to generate the test accuracy in different phases.

Huawei Certification-AI Series Training

HCIA-AI

Deep Learning and AI Development Framework

Lab Guide

Issue: 4.0



HUAWEI TECHNOLOGIES CO., LTD

Copyright © 2025 HUAWEI TECHNOLOGIES CO., LTD. All Rights Reserved.

No part of this document may be reproduced or transmitted in any form or by any means without prior written consent of Huawei Technologies Co., Ltd.

Trademark Statement


and other Huawei trademarks are trademarks of Huawei Technologies Co., Ltd.

All other trademarks and registered trademarks mentioned in this document are the property of their respective holders.

Notice

The purchased products, services and features are stipulated by the commercial contract made between Huawei and the customer. All or partial products, services and features described in this document may not be within the purchased scope or the usage scope. Unless otherwise specified in the contract, all statements, information, and recommendations in this document are provided "AS IS" without warranties, guarantees or representations of any kind, either express or implied.

The information in this document is subject to change without notice. Every effort has been made in the preparation of this document to ensure accuracy of the contents, but all statements, information, and recommendations in this document do not constitute a warranty of any kind, express or implied.

Huawei Technologies Co., Ltd.

Address:	Huawei Industrial Base Bantian, Longgang, Shenzhen 518129 People's Republic of China
Website:	https://e.huawei.com

Huawei Certification System

Huawei Certification is an integral part of the company's Platform + Ecosystem strategy. It supports the new Information and Communications Technology (ICT) infrastructure that features Cloud-Pipe-Device synergy and covers all the ICT domains, including certification for the ICT architecture and its applications, as well as certification for cloud service and platform.

The certification system offers three levels: Huawei Certified ICT Associate (HCIA), Huawei Certified ICT Professional (HCIP), and Huawei Certified ICT Expert (HCIE), to meet the learning and advancement requirements of ICT practitioners.

Huawei certifications cover all ICT fields and adapt to the industry trend of ICT convergence. With its leading talent development system and certification standards, Huawei is committed to fostering new ICT talent in the digital era, and building a healthy ICT talent ecosystem.

The Huawei HCIA-AI V4.0 certification system is intended to train and certify engineers who are capable of designing, developing, and innovating AI products and solutions by using machine learning and deep learning algorithms.

With HCIA-AI V4.0 certification, you are proved to:
Understand the history of AI development.
Master algorithms related to traditional machine learning and deep learning.
Be capable of building, training, and deploying neural networks by using development framework PyTorch.
Be competent in sales, marketing, product manager, project management, technical support, and other AI positions.

The Huawei certification system introduces you to the industry and market, helps you in innovation, and enables you to stand atop the AI frontiers.


HCIE
Huawei Certified ICT Expert
HCIP
Huawei Certified ICT Professional
HCIA
Huawei Certified ICT Associate
Project Management
Cloud Platform & Services
Basic Software & Hardware
ICT Infrastructure
Cloud Platform & Services Domain
Basic Software & Hardware Domain
IT Infrastructure Domain
Datacom Domain
Optical Domain
Wireless Domain
Cloud Computing
Cloud Service
Big Data
GaussDB
IoT
openEuler
openGauss
HarmonyOS
AI
Computing
Storage
Collaboration
Intelligent Vision
Digital Power
MDC
Datacom
WLAN
Security
Transmission
Access
5G
LTE

About This Document

Overview

This document is applicable to the candidates who are preparing for the HCIA-AI exam or readers who want to understand AI basics and basics of PyTorch programming.

Description

This lab guide introduces the following one experiments:

- Experiment 1 describes how to use PyTorch. At the end of the experiment, you will be able to use PyTorch for image classification and text classification.

Link to the data and code files used in the experiment:

https://certification-data.obs.cn-north-4.myhuaweicloud.com/ENG/HCIA-AI/V4.0/Dataset.zip

Background Knowledge Required

This document is for Huawei's basic certification. To better understand this document, ensure that you meet the following requirements:

- Have basic knowledge about Python.

Contents

About This Document 2
Overview 2
Description 2
Background Knowledge Required 2
1 Building a Fully-Connected Neural Network from Scratch ..... 错误!未定义书签。
1.1 Experiment Introduction ..... **错误!未定义书签。**
1.1.1 About This Experiment ..... **错误!未定义书签。**
1.1.2 File Parsing ..... **错误!未定义书签。**
1.1.3 Experiment Process ..... **错误!未定义书签。**
1.2 Training Code Parsing ..... **错误!未定义书签。**
1.2.1 Main Program ..... **错误!未定义书签。**
1.2.2 Loading the Dataset ..... **错误!未定义书签。**
1.2.3 Building the Network Structure ..... **错误!未定义书签。**
1.2.4 Starting Training ..... **错误!未定义书签。**
1.2.5 Drawing the Graph ..... **错误!未定义书签。**
1.2.6 Experiment Result ..... **错误!未定义书签。**
1.3 Questions ..... **错误!未定义书签。**
2 PyTorch 4
2.1 Experiment Introduction 4
2.1.1 About This Experiment 4
2.1.2 Experiment Objectives 4
2.1.3 Experiment Environment 4
2.2 PyTorch Basics 4
2.2.1 Creating a Tensor 4
2.2.2 Tensor Attributes 6
2.2.3 Tensor Method 6
2.2.4 Loading the Dataset 7
2.3 LeNet 11
2.4 ResNet-50 Image Classification 14
2.5 Questions 26

1 PyTorch

1.1 Experiment Introduction

1.1.1 About This Experiment

This experiment introduces the tensor data structure of PyTorch. By performing operations on tensors, you can understand the basic PyTorch syntax. After learning about basic operations, you can use PyTorch to build the LeNet and ResNet network structures to implement image classification.

1.1.2 Experiment Objectives

- To understand how to create a PyTorch tensor
- To understand the attributes and methods of PyTorch tensors
- To learn to build PyTorch networks

1.1.3 Experiment Environment

The PyTorch framework is required for this experiment. The recommended version is 2.0 or later. You can conduct the experiment on your PC or log in to Huawei Cloud and purchase ModelArts.

1.2 PyTorch Basics

1.2.1 Creating a Tensor

Tensors of different dimensions represent different data. A 0-dimensional tensor represents a scalar, a 1-dimensional tensor represents a vector, a 2-dimensional tensor represents a matrix, and a 3-dimensional tensor may represent three RGB channels of a color image, and the like.

Step 1 Randomly initialize a tensor.

Code:

```
import torch
x = torch.rand(4, 3)
print(x)
```

Output:

```
tensor([[0.2052, 0.2865, 0.9629],
        [0.3342, 0.8409, 0.0419],
        [0.8176, 0.9898, 0.8704],
        [0.0259, 0.0501, 0.3116]])
```

Step 2 Create a tensor with different types of data.

Code:

```
import numpy as np
print(torch.Tensor(np.array([[1, 2], [3, 4]])) # Use an array to create a tensor.
print(torch.Tensor([True,False])) # Use bool data to create a tensor.
print(torch.Tensor((1,2,3,4,5))) # Use a tuple to create a tensor.
print(torch.Tensor([i for i in range(10)])) # Use a list to create a tensor.
```

Output:

```
tensor([[1., 2.],
        [3., 4.]])
tensor([1., 0.])
tensor([1., 2., 3., 4., 5.])
tensor([0., 1., 2., 3., 4., 5., 6., 7., 8., 9.])
```

Step 3 Build different types of tensors.

Code:

```
print (torch.FloatTensor (2,3).type ()) # Build a 2 x 3 tensor of the float type.
print (torch.DoubleTensor (2,3).type ()) # Build a 2 x 3 tensor of the double type.
print (torch.HalfTensor (2,3).type ()) # Build a 2 x 3 tensor of the HalfTenso type.
print (torch.ByteTensor (2,3).type ()) # Build a 2 x 3 tensor of the byte type.
print (torch.CharTensor (2,3).type ()) # Build a 2 x 3 tensor of the char type.
print (torch.ShortTensor (2,3).type ()) # Build a 2 x 3 tensor of the short type.
print (torch.IntTensor (2,3).type ()) # Build a 2 x 3 tensor of the Int type.
print (torch.LongTensor (2,3).type ()) # Build a 2 x 3 tensor of the long type.
```

Output:

```
torch.FloatTensor
torch.DoubleTensor
torch.HalfTensor
torch.ByteTensor
torch.CharTensor
torch.ShortTensor
torch.IntTensor
torch.LongTensor
```

Step 4 Create special tensors.

Code:

```
import torch
x = torch.zeros(4, 3, dtype=torch.long)
print (x) # All-zero tensor
x = torch.ones(4, 3, dtype=torch.long)
print (x) # All-1 tensor
x = x.new_ones(4, 3, dtype=torch.double)
print(x)
x = torch.randn_like(x, dtype=torch.float)
```

```
print(x) #
```

Output:

```
tensor([[0, 0, 0],
        [0, 0, 0],
        [0, 0, 0],
        [0, 0, 0]])
tensor([[1, 1, 1],
        [1, 1, 1],
        [1, 1, 1],
        [1, 1, 1]])
tensor([[1., 1., 1.],
        [1., 1., 1.],
        [1., 1., 1.],
        [1., 1., 1.]], dtype=torch.float64)
tensor([[-0.8838,    0.8029, -1.9473],
        [-0.7147, -1.0430, -0.3002],
        [ 1.2838,    1.9056, -1.6879],
        [-0.8628,    1.6469,    0.6585]])
```

1.2.2 Tensor Attributes

Tensor attributes include shape and data type (dtype).

- The tensor shape is a tuple.
- The tensor dtype is a data type of PyTorch.

Code:

```
x = torch.Tensor(np.array([[1, 2], [3, 4]]))
print (x.shape) # Obtain the shape.
print(x.size())
print (x.dim ()) # Number of dimensions
print (x.device) # Hardware
print (x.dtype) # Data type
```

Output:

```
torch.Size([2, 2])
torch.Size([2, 2])
2
cpu
torch.float32
```

1.2.3 Tensor Method

Code:

```
x = torch.Tensor(np.array([[1, 2], [3, 4]]))

print (x.reshape (4,1)) # reshape
x = x.unsqueeze (2) # Add a dimension.
print(x.shape)
x = x.squeeze (2) # Compress dimensions.
```

```
print(x.shape)

print (x.flatten ()) # Flatten to one dimension.
x.to (device) # Move to the specified device.
```

Output:

```
tensor([[1.],
        [2.],
        [3.],
        [4.]])
torch.Size([2, 2, 1])
torch.Size([2, 2])
tensor([1., 2., 3., 4.])
```

1.2.4 Loading the Dataset

Step 1 Load the MNIST dataset.

Before loading the dataset, download the MNIST dataset from http://yann.lecun.com/exdb/mnist/ and store the training and test files in the MNIST folder. If you do not download the dataset, PyTorch automatically downloads it. Ensure that the network connection is normal.

Code:

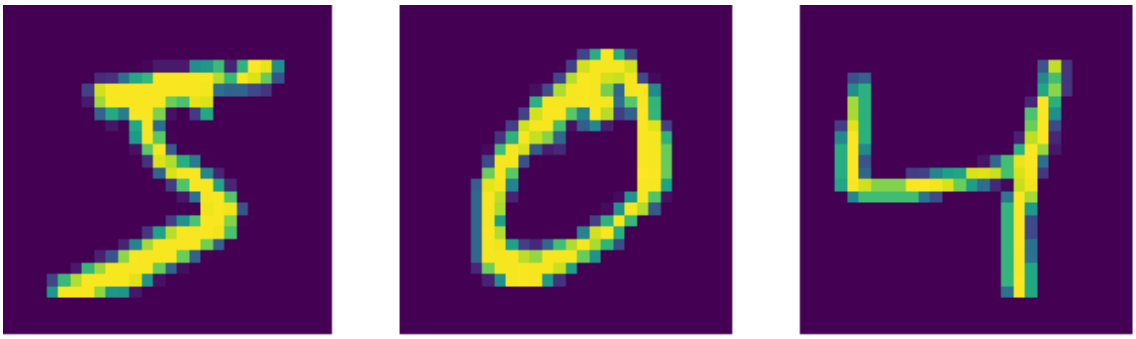
```
import matplotlib.pyplot as plt
from torchvision import datasets, transforms


# Read images from the MNIST dataset.
train_dataset = datasets.MNIST(root='./MNIST',train=True,download=True,transform=transforms.ToTensor())
# View the image and set the image size.
plt.figure(figsize=(8,8))
i = 1

# Print three subplots of the training set.
for dic in train_dataset.train_data[:3]:
    plt.subplot(3,3,i)
    plt.imshow(dic)
    plt.axis('off')
    i +=1
plt.show()
```

Output:

Figure 1-1 Samples of the MNIST dataset



Step 2 Use ImageFolder to load the dataset.

The flower dataset used in the ResNet experiment is loaded in this experiment.

```
train = torchvision.datasets.ImageFolder("./flower_photos_train/") # Load the data.

# Check the dataset attributes.
print ("Dataset types")
print(train.classes)
print(train.class_to_idx)
print ("Dataset samples")
print(train.samples[0])
print ("Image data format")
print(train.extensions[0])
```

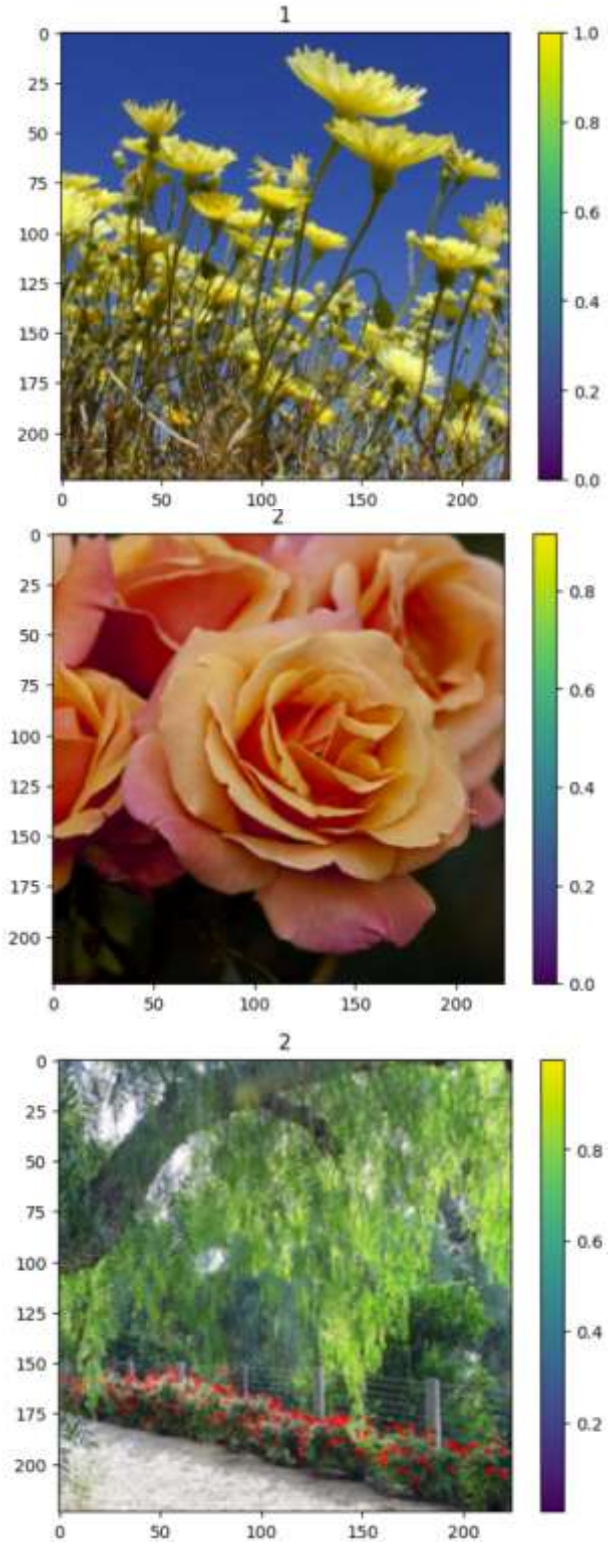
Output:

```
Dataset types
['daisy', 'dandelion', 'roses', 'sunflowers', 'tulips']
{'daisy': 0, 'dandelion': 1, 'roses': 2, 'sunflowers': 3, 'tulips': 4}
Dataset samples
('./flower_photos_train/daisy\\10140303196_b88d3d6cec.jpg', 0)
Image data format
.jpg
```

Visualize the data.

```
# Display the loaded image data.
for i in range(5):
    plt.figure()
    random.seed = 2
    num = random.randint(0,3000)
    plt.imshow(train[num][0].permute(1,2,0)) # train[num] returns a tuple containing data and the category number
corresponding to the label.
    plt.title(train[num][1])
    plt.colorbar()
    plt.grid(False)
plt.show()
```

Output:



Step 3 Customize the dataset.

In PyTorch, a custom dataset class usually needs to inherit the torch.utils.data.Dataset class and implement the __len__ and __getitem__ methods. The two methods are used to return the number of samples in the dataset and load and preprocess the samples of a given index.

```
import os
import torch
from torch.utils.data import Dataset, DataLoader
from PIL import Image
from torchvision import transforms
# Pseudocode
class CustomDataset(Dataset):
    def __init__(self, image_dir, label_file, transform=None):
        """
            image_dir (string): directory for storing images.
            label_file (string): path of the label file. Each row corresponds to a label.
            transform (callable, optional): optional image conversion operation, which is used for data
augmentation or preprocessing. The default value is None.
        """
        self.image_dir = image_dir # Image directory
        self.labels = self._load_labels(label_file) # Load labels.
        self.image_filenames = sorted(os.listdir(image_dir)) # Obtain the image file name list and sorts the files.
        self.transform = transform # Image transformation

    def _load_labels(self, label_file):
        """
        Load labels from a file.
        Args:
            label_file (string): path of the label file
        Returns:
            list: Label list. Each element is an integer.
        """
        with open(label_file, 'r') as f:
            labels = [int(line.strip()) for line in f]    # Read the file, convert each row into an integer, and store the
integers in the list.
        return labels

    def __len__(self):
        """
        Return the number of samples in the dataset.
        Returns: number of samples
        """
        return len(self.image_filenames)    # Return the length of the image file name list.

    def __getitem__(self, idx):
        """
        Obtain samples in the dataset based on indexes.
        Args:
            idx (int): sample index
        Returns:
            tuple: a tuple containing images and labels
        """
        img_name = os.path.join(self.image_dir, self.image_filenames[idx])    # Build the full path of the image file.
        image = Image.open(img_name).convert('RGB')    # Open the image file and convert it to the RGB mode.
        label = self.labels[idx] # Obtain the corresponding label.

        if self.transform:
            image = self.transform(image) # Transform the image if the transformation operation is allowed.

        # Return the tuple of the images and labels.
```

```
            return image, label

if __name__ == "__main__":
    # Define the image transformation operation.
    transform = transforms.Compose([
        # Data processing
        pass
    ])

    # Image and label file paths
    image_dir = 'path/to/images' # Replace it with your image file path.
    label_file = 'path/to/labels.txt'    # Replace it with your label file path.

    # Create the dataset instance.
    dataset = CustomDataset(image_dir=image_dir, label_file=label_file, transform=transform)

    # Create the data loader.
    dataloader = DataLoader(dataset, batch_size=32, shuffle=True, num_workers=4)    # Set the batch size to 32, shuffle the data sequence, and use four working processes.
```

1.3 LeNet

Use PyTorch to implement the LeNet network.

Step 1 Import the toolkit.

```
import torch
import torch.nn as nn
import torch.optim as optim
import torchvision
import torchvision.transforms as transforms
import os
import matplotlib.pyplot as plt
```

Step 2 Define the data mode and use it to load data.

The loaded dataset is CIFAR-10, which contains RGB color images of 10 categories: airplane, automobile, bird, cat, deer, dog, frog, horse, ship, and truck. Each category has 6000 images. There is a total of 60,000 images in the dataset, 50,000 of which are used for training and 10,000 used for testing. The size of each image is 32 x 32 pixels.

```
# Data preprocessing
transform = transforms.Compose(
    [transforms.ToTensor(),
     transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))])

# Download and load the CIFAR-10 dataset.
trainset = torchvision.datasets.CIFAR10(root='./data', train=True, download=True, transform=transform)
trainloader = torch.utils.data.DataLoader(trainset, batch_size=64, shuffle=True, num_workers=1)

testset = torchvision.datasets.CIFAR10(root='./data', train=False, download=True, transform=transform)
testloader = torch.utils.data.DataLoader(testset, batch_size=64, shuffle=False, num_workers=1)
```

Step 3 Display the data.

```
# Display the images.
dataiter = iter(trainloader)
images, labels = next(dataiter)
# Create a grid to display images.
imshow = torchvision.utils.make_grid(images)

# Convert to a NumPy array and remove normalization.
imshow = imshow.numpy().transpose((1, 2, 0))

# Show images.
plt.imshow(imshow)
plt.show()
```

Step 4 Define and instantiate a network structure class.

```
# Define the LeNet network structure.
class LeNet(nn.Module):
    def __init__(self):
        super(LeNet, self).__init__()    # Call the constructor of the parent class nn.Module.
        # For the first convolutional layer, the number of input channels is 3, the number of output channels is 6, and the size of the convolution kernel is 5 x 5.
        self.conv1 = nn.Conv2d(3, 6, kernel_size=5)
        # For the second convolutional layer, the number of input channels is 6, the number of output channels is 16, and the size of the convolution kernel is 5 x 5.
        self.conv2 = nn.Conv2d(6, 16, kernel_size=5)
        # For the first fully-connected layer, the number of input features is 16 x 5 x 5, and the number of output features is 120.
        self.fc1 = nn.Linear(16*5*5, 120)
        # For the second fully-connected layer, the number of input features is 120, and the number of output features is 84.
        self.fc2 = nn.Linear(120, 84)
        # For the third fully-connected layer (output layer), the number of input features is 84, and the number of output features is 10.
        self.fc3 = nn.Linear(84, 10)

    def forward(self, x):
        x = torch.tanh(self.conv1(x))
        # Perform 2 x 2 max pooling on x.
        x = torch.max_pool2d(x, 2)
        # x passes through the second convolutional layer, and then applies the tanh activation function.
        x = torch.tanh(self.conv2(x))
        # Perform 2 x 2 max pooling on x.
        x = torch.max_pool2d(x, 2)
        # Flatten x into a one-dimensional vector and input it to the fully-connected layer. The size is (batch_size, 16 x 5 x 5).
        x = x.view(-1, 16*5*5)
        x = torch.tanh(self.fc1(x))
        # x passes through the second fully-connected layer and then applies the tanh activation function.
        x = torch.tanh(self.fc2(x))
        # x passes through the third fully-connected layer (output layer). Generally, the activation function is not used here because the output value may be used for tasks such as classification.
        x = self.fc3(x)
        return x
```

```
# Instantiate the LeNet network.
net = LeNet()
net.parameters # View the model parameters.
```

Output:

```
<bound method Module.parameters of LeNet(
  (conv1): Conv2d(3, 6, kernel_size=(5, 5), stride=(1, 1))
  (conv2): Conv2d(6, 16, kernel_size=(5, 5), stride=(1, 1))
  (fc1): Linear(in_features=400, out_features=120, bias=True)
  (fc2): Linear(in_features=120, out_features=84, bias=True)
  (fc3): Linear(in_features=84, out_features=10, bias=True)
)>
```

Step 5 Define the loss function and optimizer.

```
criterion = nn.CrossEntropyLoss()
optimizer = optim.SGD(net.parameters(), lr=0.001, momentum=0.9)
```

Step 6 Define the training process.

```
# Check whether there are trained parameters to load.
if os.path.isfile("cifar10.pth"):
net.load_state_dict(torch.load("cifar10.pth"))
# Store the loss value.
loss_history = []

# Train the network.
for epoch in range(50): # 50 epochs
    running_loss = 0.0
    for i, data in enumerate(trainloader, 0):
        inputs, labels = data

        optimizer.zero_grad()

        outputs = net(inputs)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()

        running_loss += loss.item()
        if i % 200 == 199:
            print(f'[Epoch: {epoch + 1}, Batch: {i + 1}] loss: {running_loss / 200:.3f}')
            running_loss = 0.0

        # Record the loss value.
        loss_history.append(loss.item())

print('Finished Training')
```

Step 7 Visualize the loss decrease process.

```
# Draw the loss curve.
```

```
plt.plot(loss_history)
plt.xlabel('Batch')
plt.ylabel('Loss')
plt.title('Loss over Training Batches')
plt.show()
```

Step 8 Use the test set to test the model.

```
# Test the performance of the network with the test set.
correct = 0
total = 0
with torch.no_grad():
    for data in testloader:
        images, labels = data
        outputs = net(images)
        _, predicted = torch.max(outputs.data, 1)
        total += labels.size(0)
        correct += (predicted == labels).sum().item()

print(f'Accuracy of the network on the 10000 test images: {100 * correct / total:.2f}%')
```

Output:

```
Accuracy of the network on the 10000 test images: 75.97%
```

1.4 ResNet-50 Image Classification

The purpose of this experiment is ResNet-50 image classification, a typical case of deep learning. The process is as follows:

- Process the required dataset. The flower image dataset is used here.
- Define a network. Here we need to build a ResNet-50 model.
- Define the loss function and optimizer.
- Load the dataset and perform training. After the training is complete, use the test set for evaluation.

Step 1 Prepare for the experiment.

Before you start, check whether PyTorch is correctly installed.

In addition, you shall have basic mathematical knowledge such as Python coding basics, probability, and matrix.

Recommended environment:

Version: PyTorch 2.5

Programming language: Python 3 (≥ 3.7)

Step 2 Prepare data.

A flower image dataset is used in this example. It is an open-source dataset that includes five types of flowers: daisy (633 images), dandelion (898 images), roses (641 images), sunflowers (699 images), and tulips (799 images). The 3670 images with a size of about 230 MB are stored in five directories.

To test the model after it is deployed, the dataset is divided into two parts: flower_photos_train and flower_photos_test.

The directory structure is as follows (in the flowers folder of the dataset):

flower_photos_train

├— daisy

├— dandelion

├— roses

├— sunflowers

├— tulips

├— LICENSE.txt

flower_photos_test

├— daisy

├— dandelion

├— roses

├— sunflowers

├— tulips

├— LICENSE.txt

Download the data file.

```
!pip install download
```

```
from download import download
url_train =
b"aHR0cHM6Ly9hc2NlbmQtcHJvZmVzc2lvbmFsLWNvbnN0cnVjdGlvbi1kYXRhc2V0Lm9icy5teWh1YXdlaWNsb3VkLmNvbS9kZWVwLWxlYXJuaW5nL2Zsb3dlcl9waG90b3NfdHJhaW4uemlw"
url_test =
b"aHR0cHM6Ly9hc2NlbmQtcHJvZmVzc2lvbmFsLWNvbnN0cnVjdGlvbi1kYXRhc2V0Lm9icy5teWh1YXdlaWNsb3VkLmNvbS9kZWVwLWxlYXJuaW5nL2Zsb3dlcl9waG90b3NfdGVzdC56aXA="
path_train = download(base64.b64decode(url_train).decode(), "./", kind="zip", replace=True, verbose=False)
path_train = download(base64.b64decode(url_test).decode(), "./", kind="zip", replace=True, verbose=False)
```

Step 3 Import the Python libraries and modules and configure running information.

Import the Python libraries before start.

```
import torch
import torchvision
import torchvision.transforms as transforms
import torch.nn as nn
import torch.nn.functional as F
```

```
import torch.optim as optim
import matplotlib.pyplot as plt
from easydict import EasyDict as edict
import random
```

Step 4 Define the constant dictionary.

```
cfg = edict({
    'data_path': './flower_photos_train', # Path of the training dataset
    'test_path':'flowers/flower_photos_train',        #Path of the test dataset
    'data_size': 3616,
    'HEIGHT': 224, # Image height
    'WIDTH': 224, # Image width
    '_R_MEAN': 123.68, # Mean of CIFAR10
    '_G_MEAN': 116.78,
    '_B_MEAN': 103.94,
    '_R_STD': 1, # User-defined standard deviation
    '_G_STD': 1,
    '_B_STD':1,
    '_RESIZE_SIDE_MIN': 256, # Minimum resize value of image enhancement
    '_RESIZE_SIDE_MAX': 512,

    'batch_size': 1, # Batch size
    'num_class': 5, # Classification
    'epoch_size': 5, # Number of training times
    'num_workers':1,
    'device':"cpu", # cpu, xpu, etc.

    'prefix': ' renet50.pth',    # Model name

})
```

Step 5 Read and process the data.

Define the data processing method.

```
# Use transforms.Compose to combine multiple processing modes.
transform_train = transforms.Compose([
    transforms.RandomHorizontalFlip(),

    transforms.ToTensor(),
    transforms.Normalize(mean=[cfg._R_MEAN, cfg._G_MEAN, cfg._B_MEAN], std=[cfg._R_STD, cfg._G_STD,
cfg._B_STD]),
    transforms.Resize(cfg._RESIZE_SIDE_MIN),
    transforms.CenterCrop((cfg.HEIGHT, cfg.WIDTH)),

])

transform_test = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5)),
])
```

Load the data.

```
train = torchvision.datasets.ImageFolder(cfg. data_path, transform=transform_train)
trainloader = torch.utils.data.DataLoader(train, batch_size=cfg.batch_size, shuffle=True,
num_workers=cfg.num_workers)
```

Step 6 Define the network structure.

```
import torch.nn as nn

# Define the basic block, which is mainly used for ResNet-18 and ResNet-34.
class BasicBlock(nn.Module):
    expansion = 1 # Multiple for expansion of the number of output channels

    def __init__(self, in_channels, out_channels, stride=1):
        super(BasicBlock, self).__init__()

        # Residual function
        self.residual_function = nn.Sequential(
            nn.Conv2d(in_channels, out_channels, kernel_size=3, stride=stride, padding=1, bias=False),    # The
first convolutional layer
            nn.BatchNorm2d(out_channels),    # The first batch normalization layer
            nn.ReLU (inplace=True), # Activation function
            nn.Conv2d(out_channels, out_channels * BasicBlock.expansion, kernel_size=3, padding=1, bias=False),
# The second convolutional layer
            nn.BatchNorm2d(out_channels * BasicBlock.expansion)    # The second batch normalization layer
        )

        # Shortcut, which is used to adjust the dimensions and stride
        self.shortcut = nn.Sequential()

        # If the dimensions do not match, use 1x1 convolution to adjust the dimensions of the shortcut.
        if stride != 1 or in_channels != BasicBlock.expansion * out_channels:
            self.shortcut = nn.Sequential(
                nn.Conv2d(in_channels, out_channels * BasicBlock.expansion, kernel_size=1, stride=stride,
bias=False),
                nn.BatchNorm2d(out_channels * BasicBlock.expansion)
            )

    def forward(self, x):
        # Residual connection and activation function
        return nn.ReLU(inplace=True)(self.residual_function(x) + self.shortcut(x))

# Define the bottleneck block, which is mainly used for ResNet-50, ResNet-101, and ResNet-152.
class BottleNeck(nn.Module):
    expansion = 4 # Multiple for expansion of the number of output channels

    def __init__(self, in_channels, out_channels, stride=1):
        super(BottleNeck, self).__init__()

        # Residual function
        self.residual_function = nn.Sequential(
            nn.Conv2d(in_channels, out_channels, kernel_size=1, bias=False),    # The first 1x1 convolutional layer
for reducing dimensions
            nn.BatchNorm2d(out_channels),    # The first batch normalization layer
            nn.ReLU (inplace=True), # Activation function
```

```
                nn.Conv2d(out_channels, out_channels, stride=stride, kernel_size=3, padding=1, bias=False),    # 3x3
convolutional layer
                nn.BatchNorm2d(out_channels),    # The second batch normalization layer
                nn.ReLU (inplace=True), # Activation function
                nn.Conv2d(out_channels, out_channels * BottleNeck.expansion, kernel_size=1, bias=False),    # The
second 1x1 convolutional layer for adding dimensions
                nn.BatchNorm2d(out_channels * BottleNeck.expansion)    # The third batch normalization layer
            )

            # Shortcut, which is used to adjust the dimensions and stride
            self.shortcut = nn.Sequential()

            # If the dimensions do not match, use 1x1 convolution to adjust the dimensions of the shortcut.
            if stride != 1 or in_channels != out_channels * BottleNeck.expansion:
                self.shortcut = nn.Sequential(
                    nn.Conv2d(in_channels, out_channels * BottleNeck.expansion, stride=stride, kernel_size=1,
bias=False),
                    nn.BatchNorm2d(out_channels * BottleNeck.expansion)
                )

        def forward(self, x):
            # Residual connection and activation function
            return nn.ReLU(inplace=True)(self.residual_function(x) + self.shortcut(x))

# Define the ResNet network.
class ResNet(nn.Module):

        def __init__(self, block, num_block, num_classes=100):
            super(ResNet, self).__init__()

            self.in_channels = 64 # Initial number of channels

            # The first convolutional layer
            self.conv1 = nn.Sequential(
                nn.Conv2d(3, 64, kernel_size=3, padding=1, bias=False),
                nn.BatchNorm2d(64),
                nn.ReLU(inplace=True)
            )

            # The number of layers varies depending on the ResNet version.
            self.conv2_x = self._make_layer(block, 64, num_block[0], 1)
            self.conv3_x = self._make_layer(block, 128, num_block[1], 2)
            self.conv4_x = self._make_layer(block, 256, num_block[2], 2)
            self.conv5_x = self._make_layer(block, 512, num_block[3], 2)

            # Global average pooling layer
            self.avg_pool = nn.AdaptiveAvgPool2d((1, 1))

            # Fully-connected layer. Set the number of output classes as required.
            self.fc = nn.Linear(512 * block.expansion, num_classes)

        def _make_layer(self, block, out_channels, num_blocks, stride):
            # Generate a sequence of layers.
            strides = [stride] + [1] * (num_blocks - 1)    # Set the stride of each block to 1 except the first block.
            layers = []
```

```
            for stride in strides:
                layers.append(block(self.in_channels, out_channels, stride))    # Add a block.
                self.in_channels = out_channels * block.expansion    # Update the number of channels.

            return nn.Sequential(*layers)    # Return the layer sequence.

        def forward(self, x):
            # Forward propagation
            output = self.conv1(x)
            output = self.conv2_x(output)
            output = self.conv3_x(output)
            output = self.conv4_x(output)
            output = self.conv5_x(output)
            output = self.avg_pool(output)    # Global average pooling
            output = output.view(output.size(0), -1)    # Flatten into a one-dimensional vector.
            output = self.fc (output) # Fully-connected layer

            return output
```

Define the RseNet structure under different parameters (number of layers) based on the defined class.

```
# Define ResNets of different versions.
def resnet18():
    """Return the ResNet-18 model""
    return ResNet(BasicBlock, [2, 2, 2, 2])

def resnet34():
    """Return the ResNet-34 model"""
    return ResNet(BasicBlock, [3, 4, 6, 3])

def resnet50():
    """Return the ResNet-50 model""
    return ResNet(BottleNeck, [3, 4, 6, 3], num_classes=cfg.num_class) # Change the number of classes.

def resnet101():
    """Return the ResNet-101 model"""
    return ResNet(BottleNeck, [3, 4, 23, 3])

def resnet152():
    """Return the ResNet-152 model"""
return ResNet(BottleNeck, [3, 8, 36, 3])
```

ResNet-50 is used in the subsequent experiment.

Step 7 Define the loss function and optimizer, and initialize the network.

```
import os

net = resnet50().to(cfg.device)
if os.path.isfile(cfg.prefix):
    net.load_state_dict(torch.load(cfg.prefix))


criterion = nn.CrossEntropyLoss()
```

```
optimizer = optim.SGD(net.parameters(), lr=0.1, momentum=0.9, weight_decay=0.0001)
scheduler = optim.lr_scheduler.ReduceLROnPlateau(optimizer, factor = 0.1, patience=5)
```

Step 8 Define the training process.

```
for epoch in range(cfg.epoch_size):
    losses = []
    running_loss = 0
    for i, inp in enumerate(trainloader):
        inputs, labels = inp
        inputs, labels = inputs.to(cfg.device), labels.to(cfg.device)
        optimizer.zero_grad()

        outputs = net(inputs)
        loss = criterion(outputs, labels)
        losses.append(loss.item())

        loss.backward()
        optimizer.step()

        running_loss += loss.item()

        if i%200 == 0 and i > 0:
            print(f'Loss [{epoch+1}, {i}](epoch, minibatch): ', running_loss / 100)
            running_loss = 0.0

    avg_loss = sum(losses)/len(losses)
    scheduler.step(avg_loss)

print('Training Done')
```

Step 9 Save the model.

```
torch.save(net.state_dict(),cfg. prefix)
```

Step 10 Test the model.

```
test = torchvision.datasets.ImageFolder(cfg. test_path, transform=transform_test)
testloader = torch.utils.data.DataLoader(test, batch_size=cfg.batch_size, shuffle=True,
num_workers=cfg.num_workers)

correct = 0
total = 0

with torch.no_grad():
    for data in testloader:
        images, labels = data
        images, labels = images.to(cfg.device), labels.to(cfg.device)
        outputs = net(images)

        _, predicted = torch.max(outputs.data, 1)
        total += labels.size(0)
        correct += (predicted == labels).sum().item()
```

```
print('Accuracy: ', 100*(correct/total), '%')
```

1.5 TextCNN Text Classification

Step 1 Create a notebook file and select the PyTorch environment.

Step 2 Import related libraries.

```
import os
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader, Dataset
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
import numpy as np
import re
from collections import Counter
import urllib
import tarfile

# Set a random seed.
torch.manual_seed(42)
```

Step 3 Download the dataset.

```
# Download data.

def download_imdb(data_dir):
    # Create a data directory.
    if not os.path.exists(data_dir):
        os.makedirs(data_dir)

    # Download the IMDb dataset.
    url = "https://ai.stanford.edu/~amaas/data/sentiment/aclImdb_v1.tar.gz"
    file_path = os.path.join(data_dir, "aclImdb_v1.tar.gz")

    if not os.path.exists(file_path):
        print("Downloading IMDB dataset...")
        urllib.request.urlretrieve(url, file_path)
        print("Download complete.")

    # Decompress the file.
    if not os.path.exists(os.path.join(data_dir, "aclImdb")):
        print("Extracting IMDB dataset...")
        with tarfile.open(file_path, "r:gz") as tar:
            tar.extractall(path=data_dir)
        print("Extraction complete.")

# Download and decompress the IMDb dataset.
data_dir = "data/imdb"
download_imdb(data_dir)
```

Step 4 Load data.

```
def load_imdb_data(data_dir):
    reviews = []
    labels = []

    for label_type in ['pos', 'neg']:
        dir_name = os.path.join(data_dir, 'aclImdb', 'train', label_type)
        for fname in os.listdir(dir_name):
            if fname.endswith('.txt'):
                with open(os.path.join(dir_name, fname), 'r', encoding='utf-8') as f:
                    reviews.append(f.read())
                labels.append(1 if label_type == 'pos' else 0)

    return reviews, labels
```

Step 5 Preprocess text and build a vocabulary.

```
# Preprocess text.
def preprocess_text(text):
    text = re.sub(r'[^\w\s]', '', text) # Removal of punctuations
    text = text.lower()    # Conversion to lowercase
    return text

# Build a vocabulary.
def build_vocab(reviews, max_vocab_size=10000):
    word_counts = Counter()
    for review in reviews:
        word_counts.update(review.split())
    vocab = {word: i + 1 for i, (word, _) in enumerate(word_counts.most_common(max_vocab_size))}
    vocab['<PAD>'] = 0    # Addition of padding characters
    return vocab
```

Step 6 Convert the text into index sequences.

Input:

```
# Convert the text into index sequences.
def text_to_sequence(text, vocab):
    return [vocab.get(word, 0) for word in text.split()]

# Pad the sequences.
def pad_sequence(seq, max_len):
    if len(seq) >= max_len:
        return seq[:max_len]
    else:
        return seq + [0] * (max_len - len(seq))
```

Step 7 Define the dataset.

```
class IMDBDataset(Dataset):
    def __init__(self, X, y):
        self.X = X
        self.y = y
```

```
    def __len__(self):
        return len(self.X)

    def __getitem__(self, idx):
        return self.X[idx], self.y[idx]
```

Step 8 Define the TextCNN model.

```
class TextCNN(nn.Module):
    def __init__(self, vocab_size, embed_dim, num_classes, kernel_sizes=[3, 4, 5], num_filters=100):
        super(TextCNN, self).__init__()
        self.embedding = nn.Embedding(vocab_size, embed_dim)
        self.convs = nn.ModuleList([
            nn.Conv2d(1, num_filters, (k, embed_dim)) for k in kernel_sizes
        ])
        self.fc = nn.Linear(len(kernel_sizes) * num_filters, num_classes)
        self.dropout = nn.Dropout(0.5)

    def forward(self, x):
        x = self.embedding(x)  # (batch_size, seq_len, embed_dim)
        x = x.unsqueeze(1)  # (batch_size, 1, seq_len, embed_dim)
        x = [torch.relu(conv(x)).squeeze(3) for conv in self.convs]  # [(batch_size, num_filters, seq_len - kernel_size
+ 1)]
        x = [torch.max_pool1d(i, i.size(2)).squeeze(2) for i in x]  # [(batch_size, num_filters)]
        x = torch.cat(x, 1)  # (batch_size, num_filters * len(kernel_sizes))
        x = self.dropout(x)
        x = self.fc(x)  # (batch_size, num_classes)
        return x
```

Step 9 Define the model training functions.

```
# Define the model training functions.
def train_model(model, train_loader, criterion, optimizer, device, num_epochs=10):
    model.to(device)
    model.train()

    for epoch in range(num_epochs):
        total_loss = 0
        for batch_X, batch_y in train_loader:
            batch_X, batch_y = batch_X.to(device), batch_y.to(device)
            optimizer.zero_grad()
            outputs = model(batch_X)
            loss = criterion(outputs, batch_y)
            loss.backward()
            optimizer.step()
            total_loss += loss.item()

        print(f'Train-Epoch [{epoch+1}/{num_epochs}], Loss: {total_loss/len(train_loader):.4f}')
```

Step 10 Define the test functions.

```
# Define the test model.
def test_model(model, test_loader, device):
```

```
    model.eval()
    correct = 0
    total = 0
    with torch.no_grad():
        for batch_X, batch_y in test_loader:
            batch_X, batch_y = batch_X.to(device), batch_y.to(device)
            outputs = model(batch_X)
            _, predicted = torch.max(outputs.data, 1)
            total += batch_y.size(0)
            correct += (predicted == batch_y).sum().item()

    print(f'Test Accuracy: {100 * correct / total:.2f}%')
```

Step 11 Define the sentiment prediction functions.

```
# Predict sentiments.
def predict_sentiment(text, model, vocab, max_len=200):
    text = preprocess_text(text)
    sequence = text_to_sequence(text, vocab)
    sequence = pad_sequence(sequence, max_len)
    sequence = torch.tensor(sequence, dtype=torch.long).unsqueeze(0).to(device)
    with torch.no_grad():
        output = model(sequence)
        _, predicted = torch.max(output, 1)
    return "pos" if predicted.item() == 1 else "neg"
```

Step 12 Train the model on the main program, and test the model accuracy using the test set, and save the model.

```
#   Main program
if __name__ == "__main__":
    # Download and load data.
    data_dir = "data/imdb"
    download_imdb(data_dir)
    reviews, labels = load_imdb_data(data_dir)

    # Preprocess data.
    reviews = [preprocess_text(review) for review in reviews]
    vocab = build_vocab(reviews)
    sequences = [text_to_sequence(review, vocab) for review in reviews]

    # Pad the sequences and split the dataset.
    max_len = 200
    X = [pad_sequence(seq, max_len) for seq in sequences]
    X = torch.tensor(X, dtype=torch.long)
    y = torch.tensor(labels, dtype=torch.long)
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

    # Create datasets and data loaders.
    train_dataset = IMDBDataset(X_train, y_train)
    test_dataset = IMDBDataset(X_test, y_test)
    train_loader = DataLoader(train_dataset, batch_size=64, shuffle=True)
    test_loader = DataLoader(test_dataset, batch_size=64, shuffle=False)
```

```
# Define the model, loss function, and optimizer.
  vocab_size = len(vocab)
  embed_dim = 100
  num_classes = 2
  model = TextCNN(vocab_size, embed_dim, num_classes)
  criterion = nn.CrossEntropyLoss()
  optimizer = optim.Adam(model.parameters(), lr=0.001)

# Obtain the device.

  device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
  print(f'Using device: {device}')

# Train the model.
  train_model(model, train_loader, criterion, optimizer, device, num_epochs=10)

# Test the model.
  test_model(model, test_loader, device)

# Save the model.
  model_path = "textcnn_model.pth"
  torch.save(model.state_dict(), model_path)
  print(f"Model saved to {model_path}")
```

Output:

```
Using device: cpu
Train-Epoch [1/10], Loss: 0.6857
Train-Epoch [2/10], Loss: 0.5247
Train-Epoch [3/10], Loss: 0.4623
Train-Epoch [4/10], Loss: 0.4034
Train-Epoch [5/10], Loss: 0.3492
Train-Epoch [6/10], Loss: 0.3010
Train-Epoch [7/10], Loss: 0.2539
Train-Epoch [8/10], Loss: 0.2090
Train-Epoch [9/10], Loss: 0.1751
Train-Epoch [10/10], Loss: 0.1385
Test Accuracy: 84.70%
Model saved to textcnn_model.pth
```

Step 13 Test the model.

```
# Test two English texts.
test_texts = [
      "This movie is absolutely fantastic! The acting was superb and the plot was engaging.", # Positive
      "I hated this movie. It was boring and the characters were poorly developed."    # Negative
]
for text in test_texts:
```

```
    sentiment = predict_sentiment(text, model, vocab)
    print(f"Text: {text}\nsentiment: {sentiment}\n")
```

Output:

```
text: This movie is absolutely fantastic! The acting was superb and the plot was engaging.
emotion: pos

text: I hated this movie. It was boring and the characters were poorly developed.
emotion: neg
```

1.6 Questions

- Why the loss does not decrease but increases during LeNet and ResNet training?
- Can the accuracy be further improved if we continue to increase the number of training epochs during CIFAR-10 training of LeNet? If it can or cannot, why?
- How can we further improve network accuracy without changing the network structure?

Huawei AI Certification Tutorials

HCIA-AI

Model Deployment and Application

Lab Guide

ISSUE: 4.0



HUAWEI TECHNOLOGIES CO., LTD

Copyright © 2025 HUAWEI TECHNOLOGIES CO., LTD. All Rights Reserved.

No part of this document may be reproduced or transmitted in any form or by any means without prior written consent of Huawei Technologies Co., Ltd.

Trademark Statement

 and other Huawei trademarks are trademarks of Huawei Technologies Co., Ltd.

All other trademarks and registered trademarks mentioned in this document are the property of their respective holders.

Notice

The purchased products, services and features are stipulated by the commercial contract made between Huawei and the customer. All or partial products, services and features described in this document may not be within the purchased scope or the usage scope. Unless otherwise specified in the contract, all statements, information, and recommendations in this document are provided "AS IS" without warranties, guarantees or representations of any kind, either express or implied.

The information in this document is subject to change without notice. Every effort has been made in the preparation of this document to ensure accuracy of the contents, but all statements, information, and recommendations in this document do not constitute a warranty of any kind, express or implied.

Huawei Technologies Co., Ltd.

Address:	Huawei Industrial Base Bantian, Longgang, Shenzhen 518129 People's Republic of China
Website:	https://e.huawei.com

Huawei Certification System

Huawei Certification is an integral part of the company's Platform + Ecosystem strategy. It supports the new Information and Communications Technology (ICT) infrastructure that features cloud-pipe-device synergy and covers all the ICT domains, including certification for the ICT technical architecture and application, and cloud service and cloud platform.

The certification system offers three levels: Huawei Certified ICT Associate (HCIA), Huawei Certified ICT Professional (HCIP), and Huawei Certified ICT Expert (HCIE), to meet the learning and advancement requirements of ICT practitioners.

Huawei certifications cover all ICT fields and adapt to the industry trend of ICT convergence. With its leading talent development system and certification standards, Huawei is committed to fostering new ICT talent in the digital era, and building a healthy ICT talent ecosystem.

The Huawei HCIA-AI V4.0 certification system is intended to train and certify engineers who are capable of designing, developing, and innovating AI products and solutions by using machine learning, deep learning, and foundation model algorithms.

With HCIA-AI V4.0 certification, you are proved to:
Understand the history of AI development.
Master algorithms related to traditional machine learning, deep learning, and foundation models.
Be capable of building, training, and deploying neural networks with PyTorch development frameworks.
Be competent in AI development engineer positions.

The Huawei certification system introduces you to the industry and market, helps you in innovation, and enables you to stand atop the AI frontiers.


HCIE
Huawei Certified ICT Expert
HCIP
Huawei Certified ICT Professional
HCIA
Huawei Certified ICT Associate
Project Management
Cloud Platform & Services
Basic Software & Hardware
ICT Infrastructure
Cloud Platform & Services Domain
Cloud Computing
Cloud Service
Big Data
GaussDB
IoT
Basic Software & Hardware Domain
openEuler
openGauss
HarmonyOS
AI
IT Infrastructure Domain
Computing
Storage
Collaboration
Intelligent Vision
Digital Power
MDC
Datacom Domain
Datacom
WLAN
Security
Optical Domain
Transmission
Access
Wireless Domain
5G
LTE

About This Document

Overview

This document is intended for trainees who are preparing for the HCIA-AI exam as well as readers who are interested in the AI basics, Large Language Model (LLM) deployment, and API calling.

Description

This document consists of two experiments. Starting from CLI-based LLM deployment, it describes the operations such as CLI dialog, web-based LLM deployment, deployment service status query, and API dialog calling.

- Experiment 1 is CLI-based LLM deployment. You can communicate with LLM through the CLI by configuring basic parameters.
- Experiment 2 is web-based LLM deployment. After the service is successfully deployed, you can use simple code to call APIs to communicate with LLM.

Background Knowledge Required

This is a basic course for Huawei certification. To better understand this document, readers should:

- Have AI basics, Python programming capability, and computer knowledge.

Experiment Environment Overview

Device introduction

The following table lists the recommended device configurations for each HCIA-AI experiment environment.

Device, hardware, and software versions:

Device Name	Hardware Requirement	Software Version
PC	X86 CPU that supports the AVX instruction set.	Operating system: Windows 10 or later, or Ubuntu 22.04 or later Python version: Python 3.x

Contents

About This Document **3**
Overview **3**
Description **3**
Background Knowledge Required **3**
Experiment Environment Overview **3**
1 CLI-based LLM Deployment **5**
1.1 Experiment Introduction 5
1.1.1 About This Experiment 5
1.1.2 Experiment Objectives 5
1.2 Experiment Configuration 5
1.2.1 General Steps 5
1.2.2 Procedure 5
2 Web-based LLM Deployment **11**
2.1 Experiment Introduction 11
2.1.1 About This Experiment 11
2.1.2 Experiment Objectives 11
2.2 Experiment Configuration 11
2.2.1 General Steps 11
2.2.2 Procedure 11
2.3 Question 15

1 CLI-based LLM Deployment

1.1 Experiment Introduction

1.1.1 About This Experiment

In this experiment, llama.cpp is used to deploy the model (project address: https://github.com/ggerganov/llama.cpp). The model file is the quantized DeepSeek-R1-Distill-Qwen-1.5B model (project image path: https://huggingface.co/bartowski/DeepSeek-R1-Distill-Qwen-1.5B-GGUF). llama.cpp is implemented using C++ and does not have any dependency, which greatly simplifies LLM deployment and is suitable for beginners. Requiring fewer resources for inference, the quantized DeepSeek-R1-Distill-Qwen-1.5B model delivers a high inference speed with the CPU on a laptop, lowering hardware requirements. It is the smallest inference model of the DeepSeek-R1 series and is capable of thinking. With a few simple steps, you can talk to LLM in the command line.

1.1.2 Experiment Objectives

- To learn how to use llama.cpp to deploy the LLM.
- To understand the common parameters for deploying the LLM.

1.2 Experiment Configuration

1.2.1 General Steps

- Download the llama.cpp executable file.
- Download the quantized DeepSeek-R1-Distill-Qwen-1.5B model file.
- Configure different running parameters to communicate with the LLM.

1.2.2 Procedure

Note: The procedure for downloading the code file and model file in a Windows operating system is different from that in Ubuntu. Perform operations based on the operating system. The deployment commands are the same.

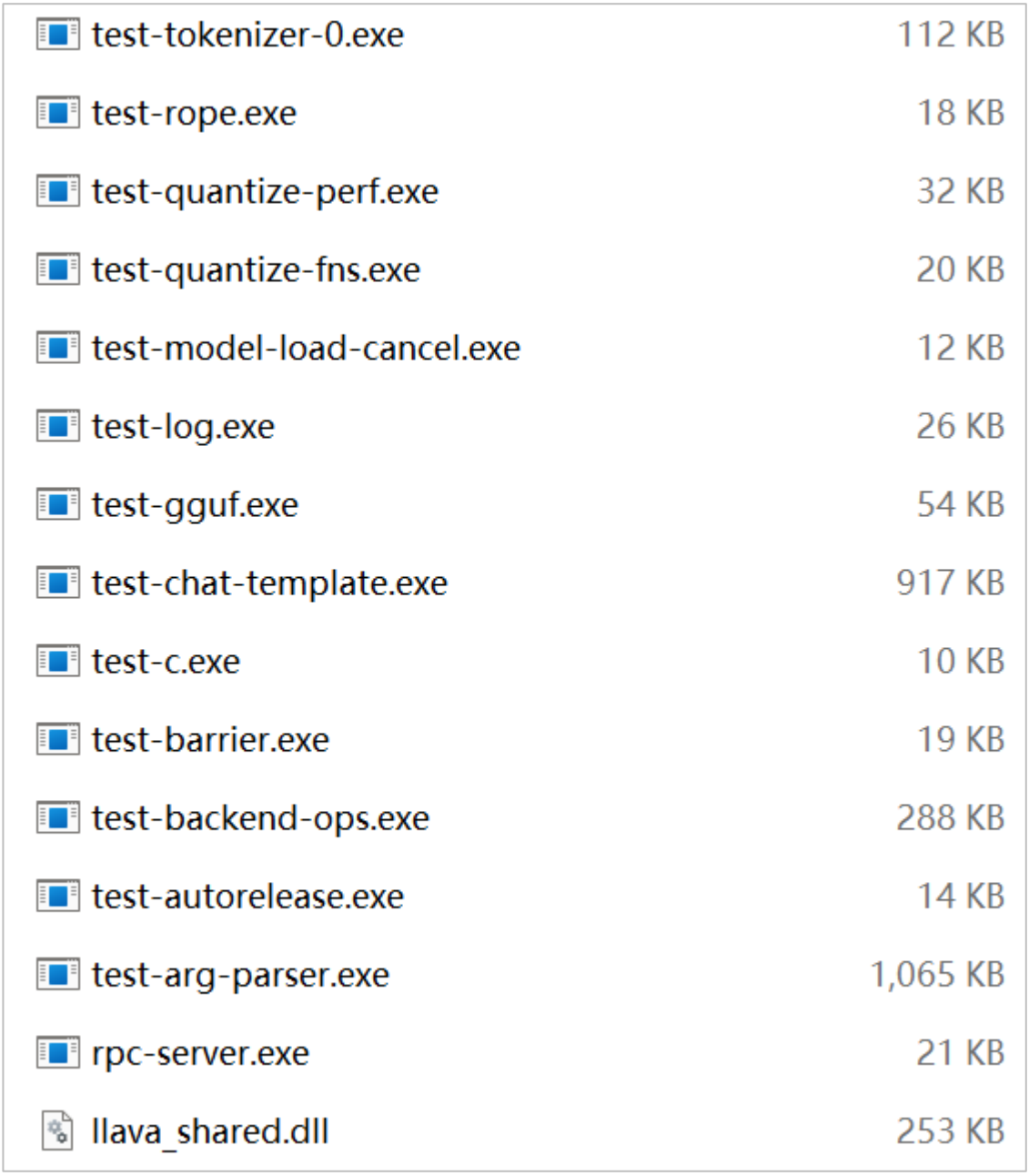
1.2.2.1 Downloading the llama.cpp Executable File

1.2.2.1.1 Windows users

Step 1 Click the following link to download the compressed package:

https://certification-data.obs.cn-north-4.myhuaweicloud.com/CHS/HCIA-AI/V4.0/chapter5/llama_cpp.zip

Step 2 Decompress the package and open the folder, as shown in the following figure.



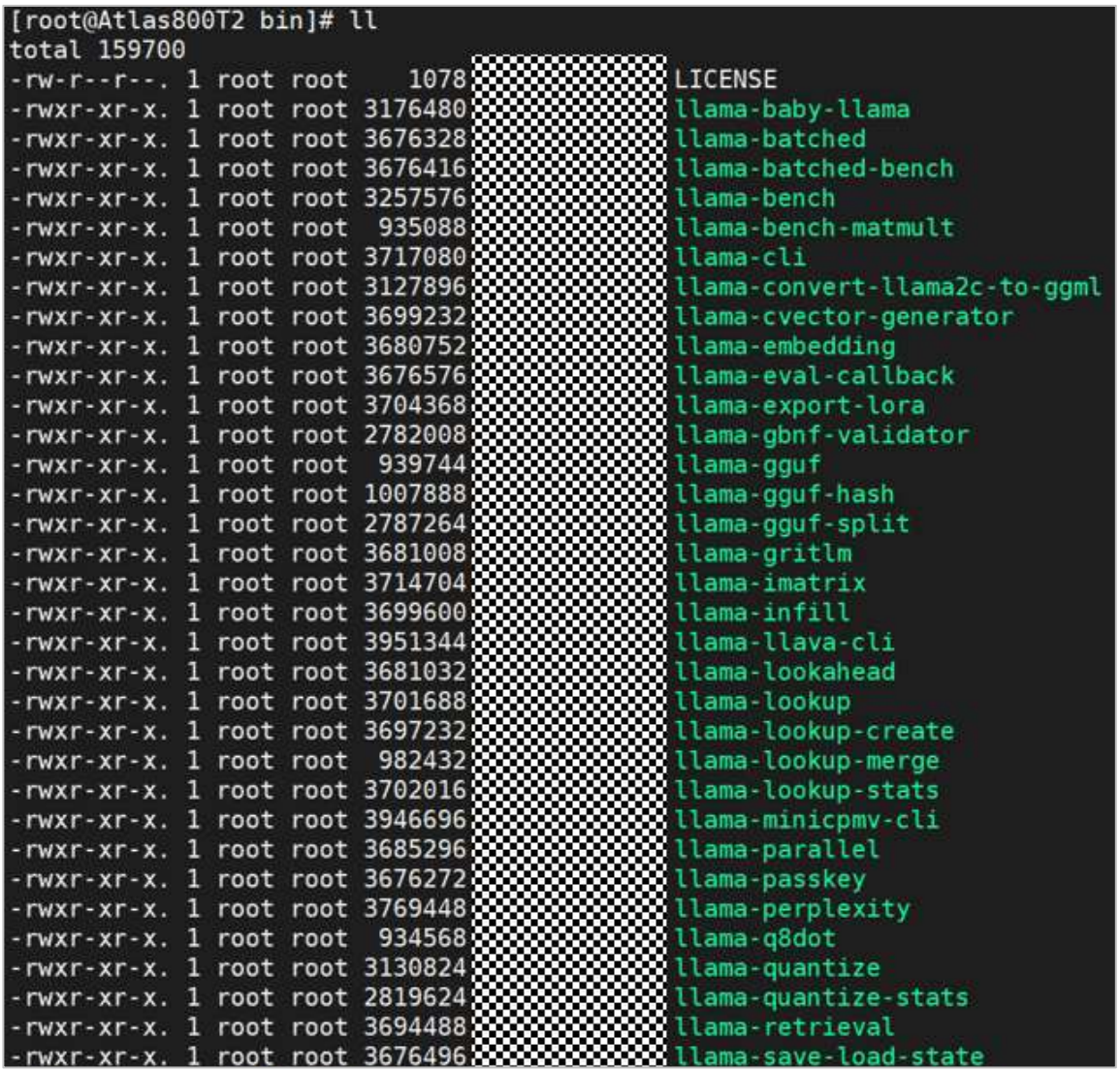
1.2.2.1.2 Ubuntu users

Step 1 Open a terminal window and run the following commands to download and decompress the file:

```
wget
https://certification-data.obs.cn-north-4.myhuaweicloud.com/CHS/HCIA-AI/V4.0/chapter5/llama_cpp_ubuntu.zip
unzip llama_cpp_ubuntu.zip
```

Step 2 Run the following command to open the folder and view the file:

```
cd build/bin && ll
```



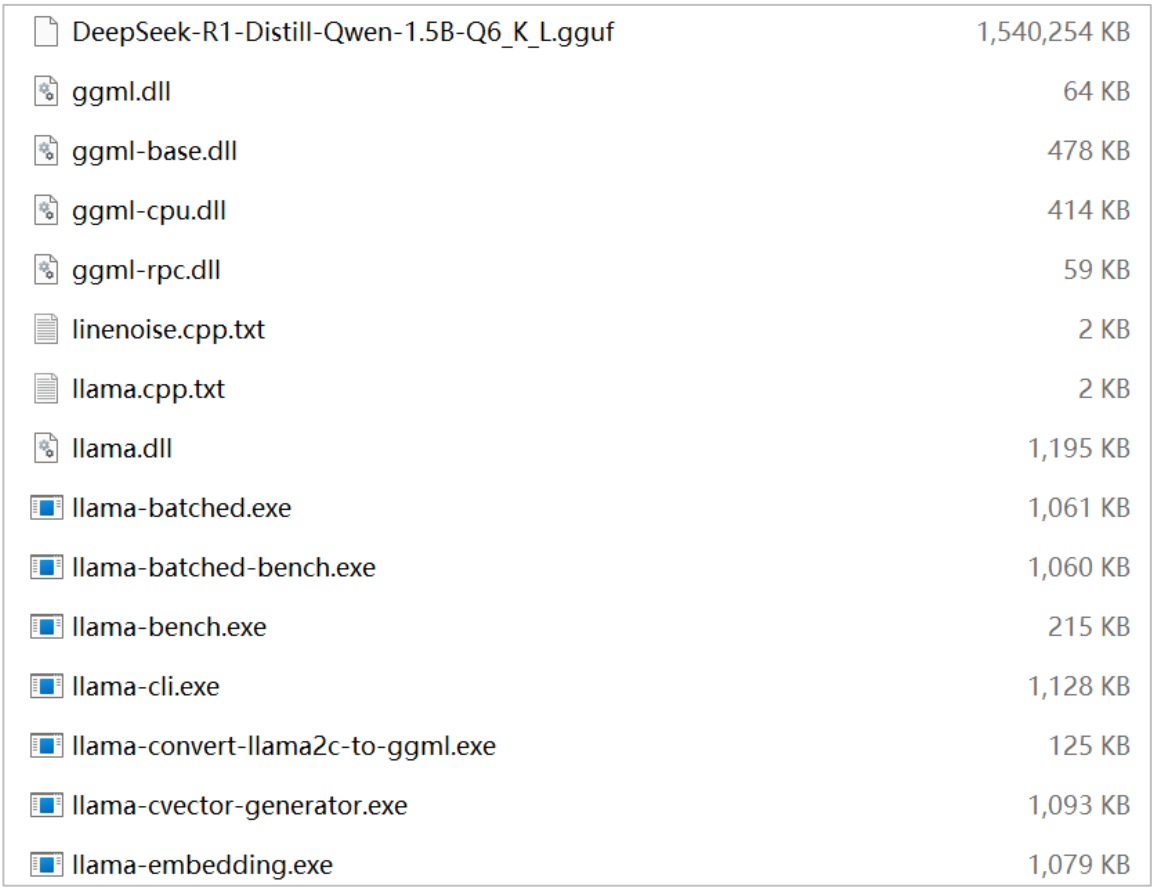
1.2.2.2 Downloading the DeepSeek-R1-Distill-Qwen-1.5B Model File

1.2.2.2.1 Windows users

Step 1 Click the following link to download the model file:

https://huggingface.co/bartowski/DeepSeek-R1-Distill-Qwen-1.5B-GGUF/resolve/main/DeepSeek-R1-Distill-Qwen-1.5B-Q6_K_L.gguf

Step 2 Move the file to the **llama_cpp** folder, as shown in the following figure.



1.2.2.2.2 Ubuntu users

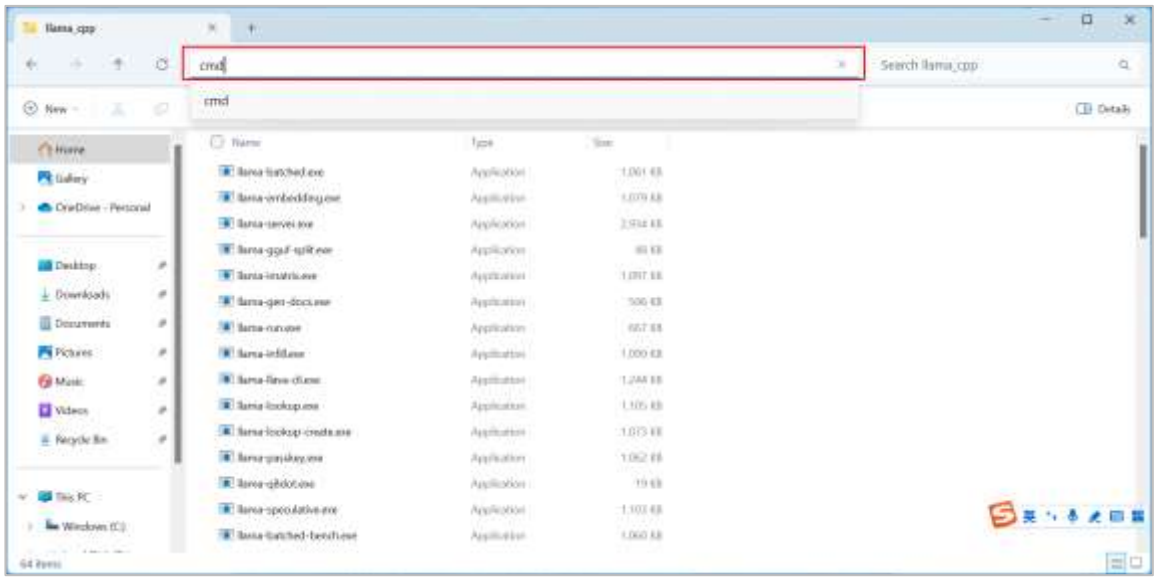
Step 1 Run the following command to download the model file to the code folder:

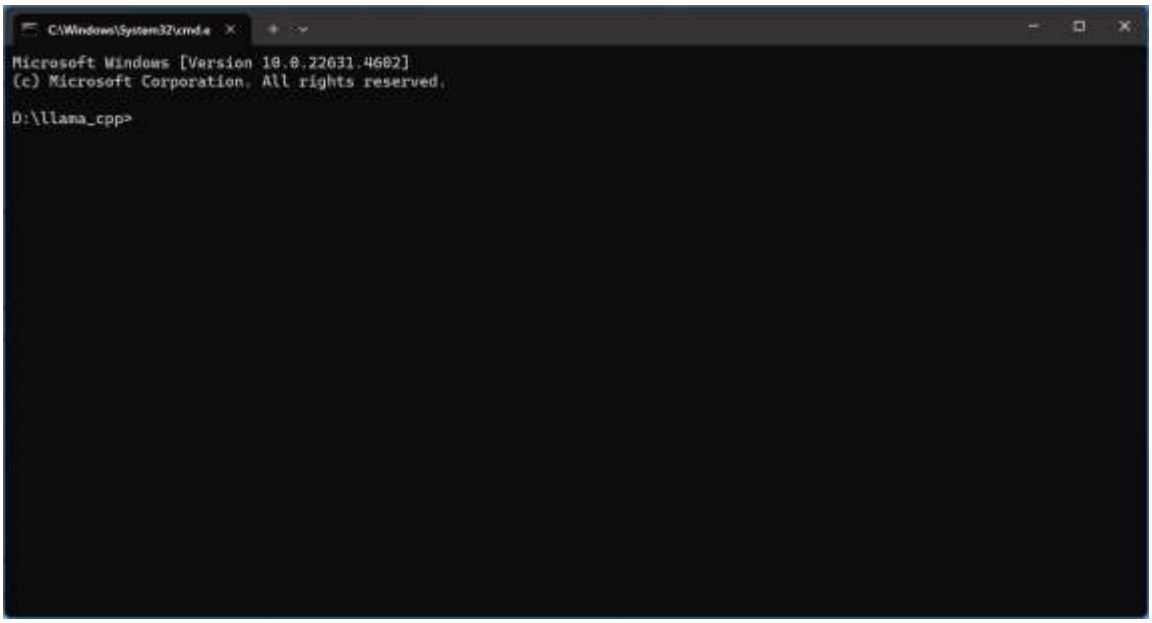
```
wget
https://huggingface.co/bartowski/DeepSeek-R1-Distill-Qwen-1.5B-GGUF/resolve/main/DeepSeek-R1-Distill-Qwen-1.5B-Q6_K_L.gguf
```

1.2.2.3 CLI-based Deployment

Step 1 Click the navigation bar above the folder, enter the following command, and press **Enter**. Ubuntu users can skip this step.

```
cmd
```





Step 2 Run the following command to perform inference for the model:

```
#Dialog mode
llama-cli.exe -m DeepSeek-R1-Distill-Qwen-1.5B-Q6_K_L.gguf --temp 0.6
```

Parameter description:

-m: model path

-cnv: conversation mode (Special tokens, prefixes, and suffixes are not printed, and the default chat template is used. By default, the dialog mode is enabled for the software package provided in this experiment. Adding parameters will lower the model's answer quality, so this parameter is not added to the CLI.)

--chat-template: chat template. If the template does not match the model, the model returns garbled characters or answers non-stop. For details, see https://github.com/ggerganov/llama.cpp/wiki/Templates-supported-by-llama_chat_apply_template

--color: Different colors are used to distinguish user input from model output.

--temp: Adjusts the randomness of the output text. The default value is **0.8**. Set this parameter to **0.6**, which is recommended by DeepSeek.

--repeat-penalty: Controls repetition of the token sequence in the generated text. The default value is **1.0** (disabled).

--top-k: The next token is selected from the k candidates with the highest probability. The default value is **40**.

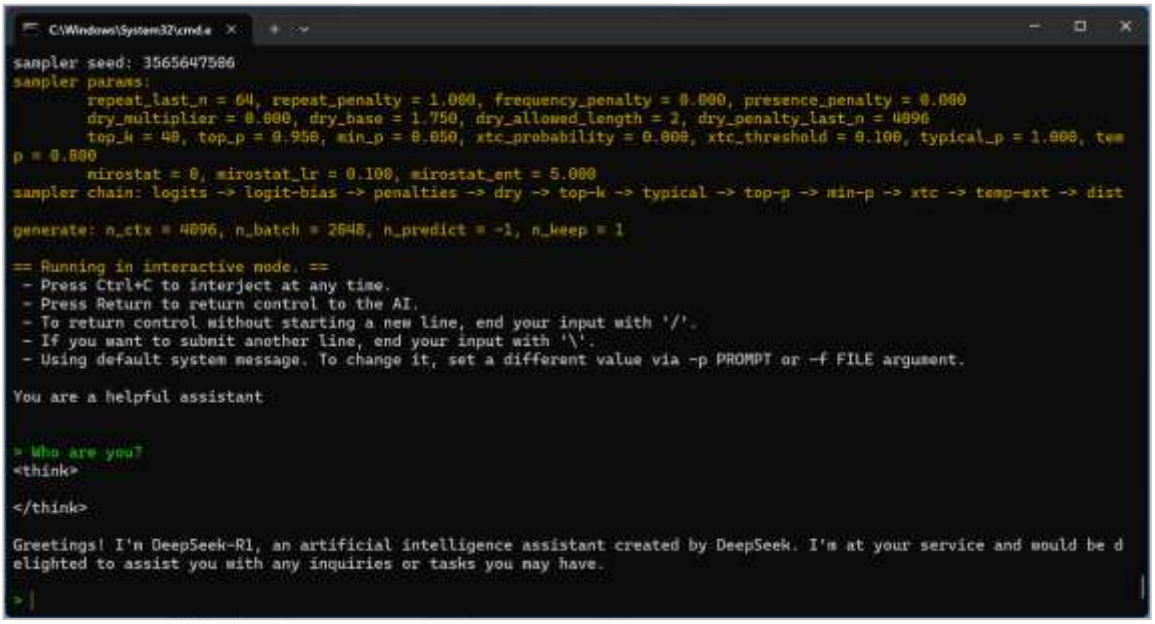
For more details about the parameters, click the following link:

https://github.com/ggerganov/llama.cpp/tree/master/examples/main

Step 3 Dialog with the model. For example, enter "who are you?" and obtain the result.

Step 4 The effects of different parameters are as follows (Text inside <think></think> is the model's thinking process):

```
llama-cli.exe --color -m DeepSeek-R1-Distill-Qwen-1.5B-Q6_K_L.gguf
```



2 Web-based LLM Deployment

2.1 Experiment Introduction

2.1.1 About This Experiment

Different from exercise 1, llama.cpp is used to deploy the web service, and then APIs are called through code to dialog with the model. llama.cpp provides basic API services as well as OpenAI-compatible API services. You can select the calling mode as required. The llama.pp code and model file are the same as those in exercise 1, and the code for calling APIs using Python is added.

2.1.2 Experiment Objectives

- To learn how to write Python code to call APIs
- To learn how to realize structured output of web return values
- To learn how to use llama.cpp to deploy web services.

2.2 Experiment Configuration

2.2.1 General Steps

- Use llama.cpp to deploy the web service.
- Use the library provided in Python to call APIs.
- Use the OpenAI library to call OpenAI-compatible APIs.

2.2.2 Procedure

Step 1 The llama.cpp code and model file are the same as those in experiment 1. Open the CLI and run the following command to deploy the web service:

```
llama-server.exe -m DeepSeek-R1-Distill-Qwen-1.5B-Q6_K_L.gguf -c 2048
```

Parameter description:

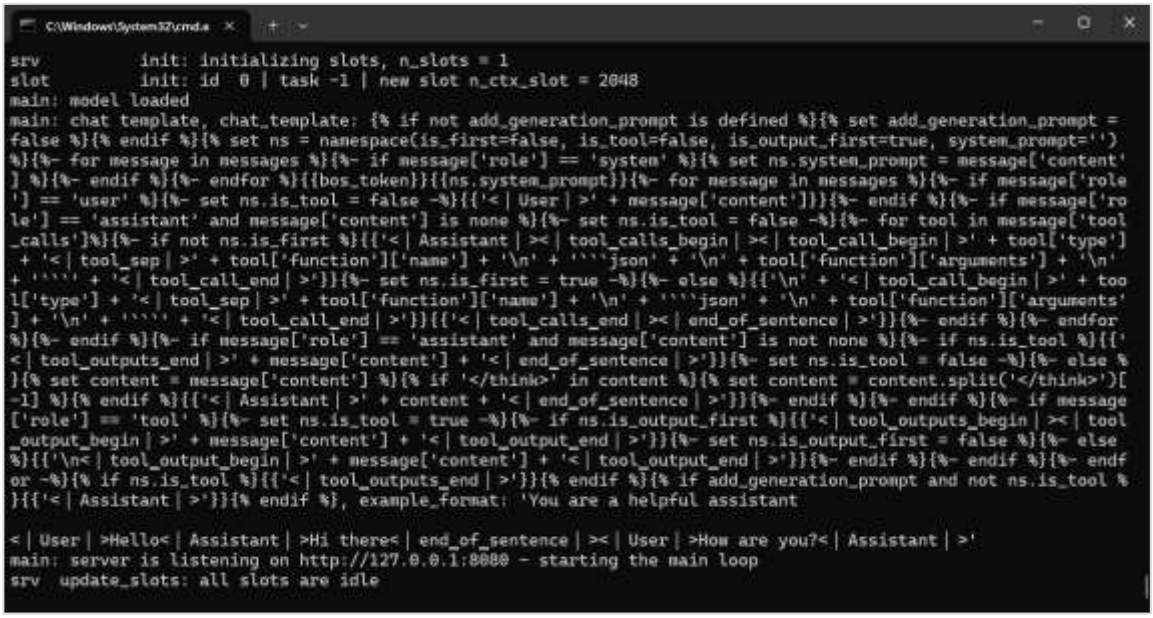
-m: model path

-c: length of the prompt text. The greater length, the more content the model can process. The default value is **4096**, and **0** indicates that the content is loaded from the model.

For more details about the parameters, click the following link:

https://github.com/ggerganov/llama.cpp/tree/master/examples/server

Note: If garbled characters are displayed, run the command in experiment 1 to terminate the program on the same window, and then run the command of the current experiment.



Step 2 Open a Python editor, for example, Jupyter Notebook, on the computer. Click the following link to download the Jupyter Notebook file that contains code. You can also write and run the code by performing the following steps.

https://certification-data.obs.cn-north-4.myhuaweicloud.com/CHS/HCIA-AI/V4.0/chapter5/Web%20API%20InVoking.ipynb

Step 3 Enter the following code to check the web service status. The status code 200 indicates that the web service is normal, and 503 indicates that the web service is abnormal. The string in red is the web service request address. If the web service deployment address is changed in the previous steps, change it here accordingly.

```
import requests
import json

def get_completion():
    response = requests.get(url='http://localhost:8080/health')
return response

print(get_completion())
```

```
import requests
import json

def get_completion():
    response = requests.get(url='http://localhost:8080/health')
    return response

print(get_completion())
```

```
<Response [200]>
```

Step 4 Enter the following code to use the requests library provided by Python to call the original API. When entering prompts, specify whether it is the user's or the model's input. For details about other parameter configuration, click the following link: https://github.com/ggerganov/llama.cpp/tree/master/examples/server#post-completion-given-a-prompt-it-returns-the-predicted-completion.

```
import requests
import json

def get_completion(prompt):
    headers = {'Content-Type': 'application/json'}
    data = {"prompt": prompt}
    response = requests.post(url='http://localhost:8080/completions', headers=headers, data=json.dumps(data))
    return response.json()['content']

print(get_completion('<|User|>Hello<|Assistant|>'))
```



Step 5 Enter the following code to use the requests library provided by Python to call the OpenAI-compatible API. This method is much more convenient. The final output is a list. You can use regular expressions to extract required content.

```
import requests
import json
def get_completion(prompt):
    headers = {'Content-Type': 'application/json','Authorization':'Bearer no-key'}
    data = {
"model": "gpt-3.5-turbo",
"messages": [

{
    "role": "user",
    "content": prompt
}
]
}
    response = requests.post(url='http://localhost:8080/v1/chat/completions', headers=headers,
data=json.dumps(data))
    return response.json()['choices']

print(get_completion('Hello'))
```



```
import requests
import json
def get_completion(prompt):
    headers = {'Content-Type': 'application/json','Authorization':'Bearer no-key'}
    data = {
"model": "gpt-3.5-turbo",
"messages": [
{
    "role": "user",
    "content": prompt
}
]
}
    response = requests.post(url='http://localhost:8080/v1/chat/completions', headers=headers, data=json.dumps(data))
    return response.json()['choices']

print(get_completion('Hello'))
```

```
[{'finish_reason': 'stop', 'index': 0, 'message': {'content': '<think>\n\n</think>\n\nHello! How can I assist you today? 😊', 'tool_calls': None, 'role': 'assistant'}}]
```

Step 6 Enter the following code to use the OpenAI library to call the OpenAI-compatible API. If you are prompted that the OpenAI library cannot be imported, run the following command to install the library. Using this method, you can easily obtain the required output and configure running parameters.

```
#Installation dependencies
pip install openai
```

```
import openai

client = openai.OpenAI(
    base_url="http://localhost:8080/v1", # Service request address for local deployment
    api_key = "sk-no-key-required"
)

completion = client.chat.completions.create(
model="gpt-3.5-turbo",
messages=[

    {"role": "user", "content": "Who are you?"}
]
)

print(completion)
print(completion.choices[0].message)
print(completion.choices[0].message.content)
```

```python
import openai

client = openai.OpenAI(
    base_url="http://localhost:8000/v1", # Service request address for local deployment
    api_key = "sk-no-key-required"
)

completion = client.chat.completions.create(
model="gpt-3.5-turbo",
messages=[
    {"role": "user", "content": "Who are you?"}
]
)

print(completion)
print('-----------------------------------------------------------------------------')
print(completion.choices[0].message)
print('-----------------------------------------------------------------------------')
print(completion.choices[0].message.content)
```

```
ChatCompletion(id='chatcmpl-bTkivFy3lApSOw58OTQOehPHtDZTDkoP', choices=[Choice(finish_reason='stop', index=0, logprobs=None, message=ChatCompletionMesse
ge(content="<think>\n\n</think>\n\nGreetings! I'm DeepSeek-R1, an artificial intelligence assistant created by DeepSeek. I'm at your service and would b
e delighted to assist you with any inquiries or tasks you may have.", refusal=None, role='assistant', audio=None, function_call=None, tool_calls=Non
e))], created=1740038205, model='gpt-3.5-turbo', object='chat.completion', service_tier=None, system_fingerprint='b4689-90e4dba4', usage=CompletionUsage
(completion_tokens=44, prompt_tokens=7, total_tokens=51, completion_tokens_details=None, prompt_tokens_details=None), timings={'prompt_n': 5, 'prompt_m
s': 81.719, 'prompt_per_token_ms': 16.343799999999998, 'prompt_per_second': 61.185281268737095, 'predicted_n': 44, 'predicted_ms': 1835.851, 'predicted_
per_token_ms': 41.72388636363637, 'predicted_per_second': 23.96708665335465})
-----------------------------------------------------------------------------
ChatCompletionMessage(content="<think>\n\n</think>\n\nGreetings! I'm DeepSeek-R1, an artificial intelligence assistant created by DeepSeek. I'm at your
service and would be delighted to assist you with any inquiries or tasks you may have.", refusal=None, role='assistant', audio=None, function_call=None,
tool_calls=None)
-----------------------------------------------------------------------------
<think>

</think>

Greetings! I'm DeepSeek-R1, an artificial intelligence assistant created by DeepSeek. I'm at your service and would be delighted to assist you with any
inquiries or tasks you may have.
```

2.3 Question

When deploying the LLM, which parameters have the greatest impact on the model output?

Huawei AI Certification Tutorials

HCIA-AI

Lab Environment Setup Guide

ISSUE: 4.0



HUAWEI TECHNOLOGIES CO., LTD

Copyright © Huawei Technologies Co., Ltd. 2025. All rights reserved.

No part of this document may be reproduced or transmitted in any form or by any means without prior written consent of Huawei Technologies Co., Ltd.

Trademark Statement


and other Huawei trademarks are trademarks of Huawei Technologies Co., Ltd.

All other trademarks and registered trademarks mentioned in this document are the property of their respective holders.

Notice

The purchased products, services and features are stipulated by the commercial contract made between Huawei and the customer. All or partial products, services and features described in this document may not be within the purchased scope or the usage scope. Unless otherwise specified in the contract, all statements, information, and recommendations in this document are provided "AS IS" without warranties, guarantees or representations of any kind, either express or implied.

The information in this document is subject to change without notice. Every effort has been made in the preparation of this document to ensure accuracy of the contents, but all statements, information, and recommendations in this document do not constitute a warranty of any kind, express or implied.

Huawei Technologies Co., Ltd.

Address:	Huawei Industrial Base Bantian, Longgang, Shenzhen 518129 People's Republic of China
Website:	https://e.huawei.com

Huawei Certification System

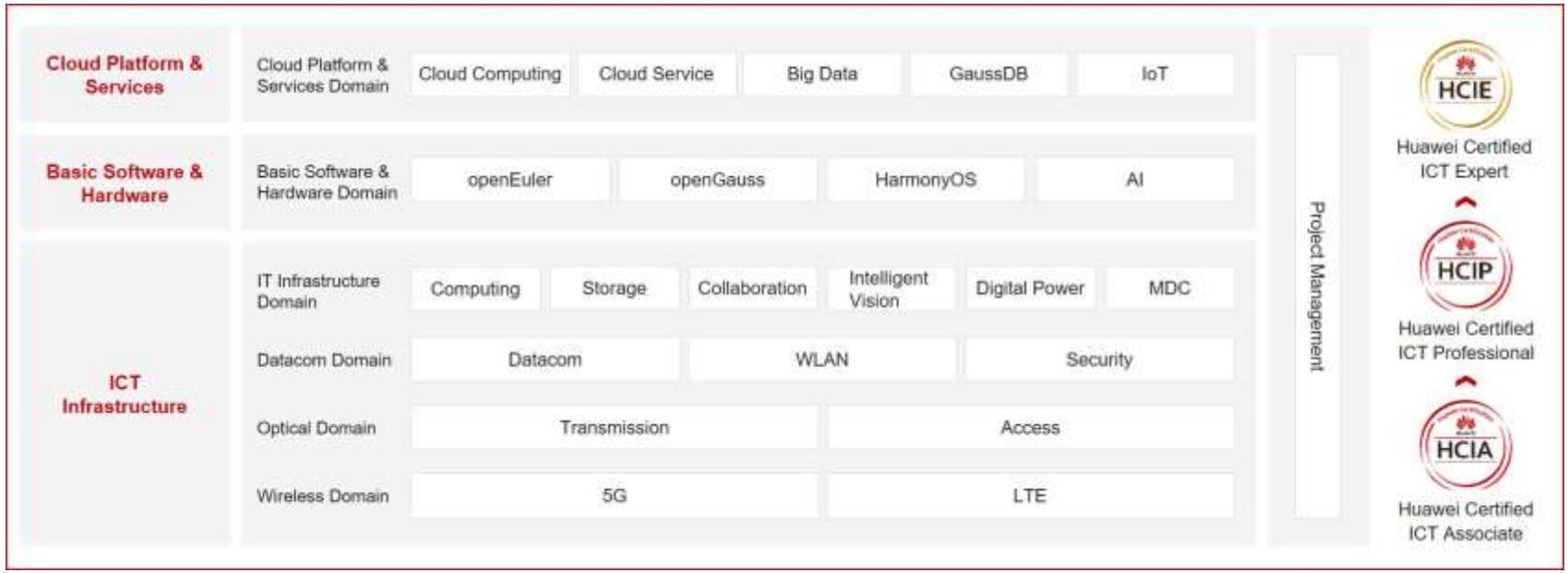
Huawei Certification is an integral part of the company's Platform + Ecosystem strategy. It supports the new Information and Communications Technology (ICT) infrastructure that features cloud-pipe-device synergy and covers all the ICT domains. Huawei Certification consists of three categories: ICT Infrastructure Certification, Basic Software & Hardware Certification, and Cloud Platform & Services Certification.

Huawei offers three levels of certification: Huawei Certified ICT Associate (HCIA), Huawei Certified ICT Professional (HCIP), and Huawei Certified ICT Expert (HCIE).

The Huawei HCIA-AI certification system V4.0 is intended to train and certify engineers who are capable of designing, developing, and innovating AI products and solutions by using machine learning, deep learning, and foundation model algorithms.

With HCIA-AI V4.0 certification, you are proved to:
Understand the history of AI development ; master algorithms related to traditional machine learning, deep learning, and foundation models; be capable of building, training, and deploying neural networks with the PyTorch development frameworks; be competent in AI development engineer positions.

The Huawei certification system introduces you to the industry and market, helps you in innovation, and enables you to stand atop the intelligent vision frontiers.


Cloud Platform & Services
Cloud Platform & Services Domain
Cloud Computing
Cloud Service
Big Data
GaussDB
IoT
Basic Software & Hardware
Basic Software & Hardware Domain
openEuler
openGauss
HarmonyOS
AI
ICT Infrastructure
IT Infrastructure Domain
Computing
Storage
Collaboration
Intelligent Vision
Digital Power
MDC
Datacom Domain
Datacom
WLAN
Security
Optical Domain
Transmission
Access
Wireless Domain
5G
LTE
Project Management
HCIE
Huawei Certified ICT Expert
HCIP
Huawei Certified ICT Professional
HCIA
Huawei Certified ICT Associate

About This Document

Overview

This document is intended for trainees who are preparing for the HCIA-AI exam as well as readers who are interested in the basics of AI and PyTorch programming. Trainees need to configure the following three parts based on the computer environment.

Description

This document consists of the following parts:

- Part 1 describes how to set up the Windows lab environment.
- Part 2 describes how to set up the Ubuntu lab environment.
- Part 3 describes how to use the Huawei Cloud.

Background Knowledge Required

This is a basic course for Huawei certification. To better understand this document, readers should:

- Have basic Python knowledge and be familiar with Python programming basics.

Contents

About This Document 3
Overview 3
Description 3
Background Knowledge Required 3
1 Windows Lab Environment Configuration 2
1.1 Installing Anaconda 2
1.1.1 Introduction to Anaconda 2
1.1.2 Procedure 2
1.2 Introduction to Anaconda Virtual Environment 5
1.2.1 Introduction to Virtual Environment 5
1.2.2 Using CLI to Create Virtual Environments 5
1.2.3 Activating a Virtual Environment 6
1.2.4 Viewing a Virtual Environment 7
1.2.5 Deleting a Virtual Environment 8
1.3 Installing Tool Libraries 8
1.3.1 Installing PyTorch 8
1.3.2 Installing Other Tool Libraries 10
2 Ubuntu Lab Environment Configuration 11
2.1 Installing Anaconda 11
2.1.1 Procedure 11
2.2 1.4 Installing Tool Libraries 14
2.2.1 Installing PyTorch 14
3 Using Huawei Cloud 15
3.1 Overview 15
3.2 Huawei Cloud Account Application and Identity Authentication 15
3.2.1 Procedure 15
3.3 Obtaining the Access Key and Secret Access Key 16
3.3.1 Overview 16
3.3.2 Generating an AK and SK 16
3.4 Purchasing Huawei Cloud Resources-ModelArts 17
3.4.1 ModelArts 17
3.5 Purchasing Huawei Cloud Resources-ECS 20

1 Windows Lab Environment Configuration

1.1 Installing Anaconda

1.1.1 Introduction to Anaconda

Anaconda is a distribution of Python for scientific computing. It supports Linux, macOS, and Windows systems. With its package and environment management functions, Anaconda allows co-existence and switching of different Python versions as well as installation of third-party packages. Anaconda uses the conda command-line tool or pip to manage packages and environments. It also comes with Python and related tools. Anaconda is a Python tool for enterprise-level big data analytics. For Anaconda distribution, more than 250 packages are automatically installed. After installation, you can also install additional open-source packages through PyPI. In addition to data analysis, Anaconda can even be used for big data and AI businesses.

Anaconda provides the following features for developers:

1. Anaconda distribution comes with Python. After installing Anaconda, you do not need to install Python again.

2. With Anaconda, you only need to add virtual environments when different frameworks are required for development. Then, you can work in different environments without worrying about compatibility and other issues. You can also configure environments for special projects to simplify management.

1.1.2 Procedure

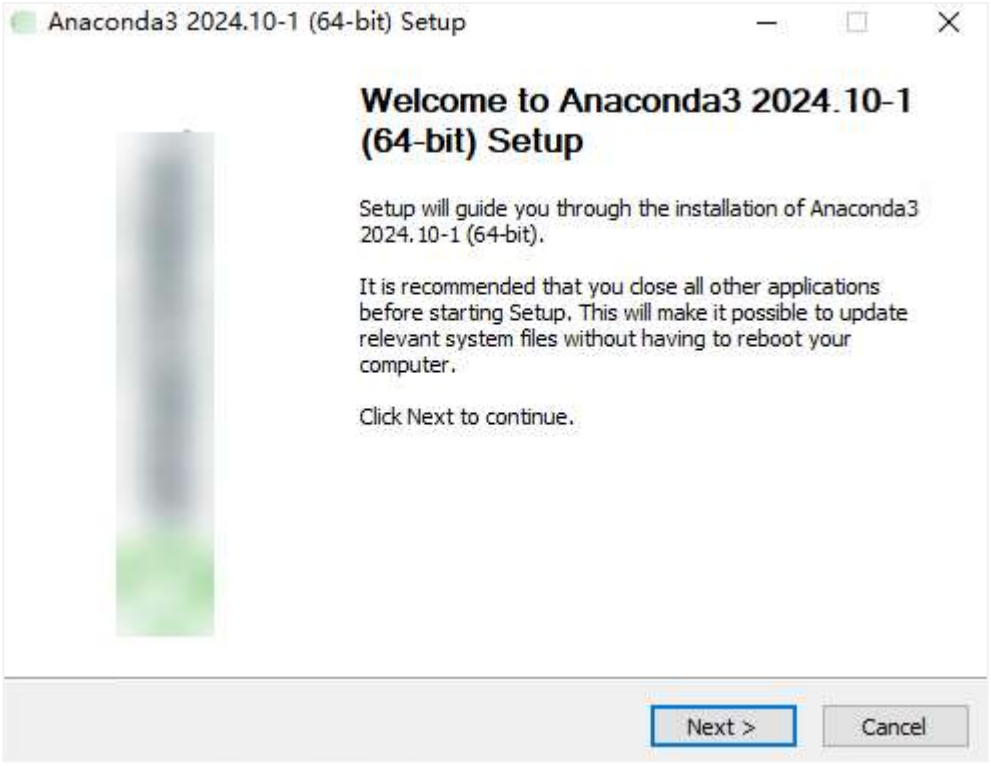
Step 1 Download Anaconda.

Log in to the Anaconda official website https://repo.anaconda.com/archive/ to download the installation package. In this example, the Windows installation package is selected. For 64-bit Windows users, **Anaconda3-20xx.xx-x-Windows-x86_64.exe** is recommended. (This example describes how to install Anaconda. You can also use Miniconda, and the installation procedure is similar. Miniconda is a lightweight version of Anaconda. By default, it contains only Python and Conda. You can use pip or conda to install the Miniconda package.)

Step 2 Install Anaconda.

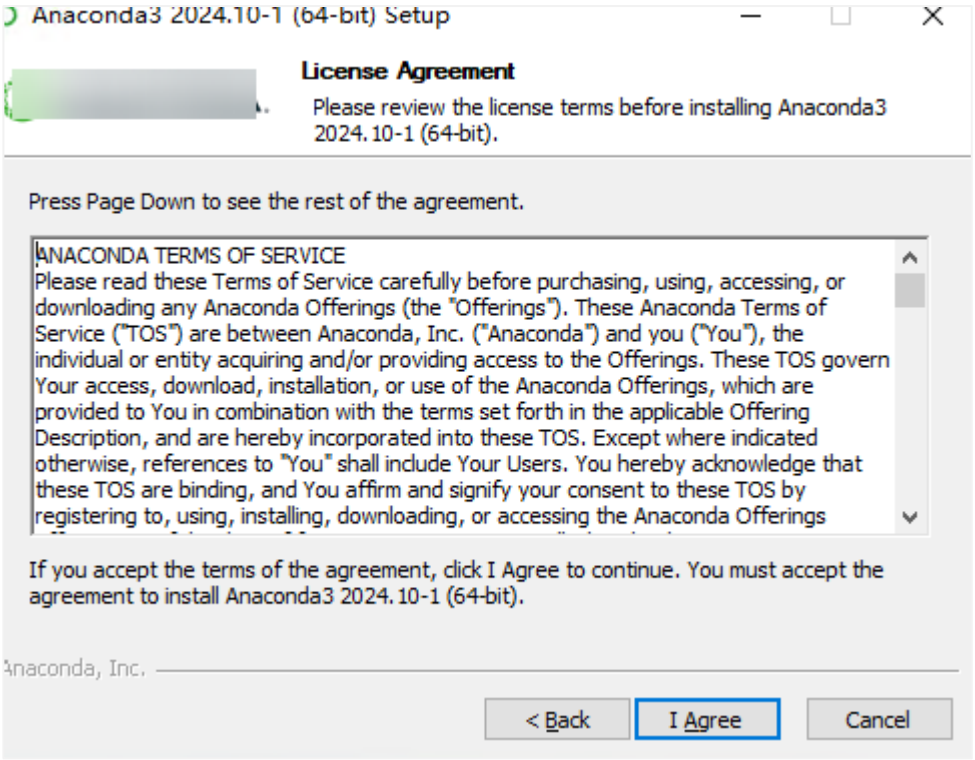
Double-click the **Anaconda3-xxxx.xx-Windows-x86_64.exe** file, and click **Next** on the following page.

Figure 1-1 Anaconda3 installation program



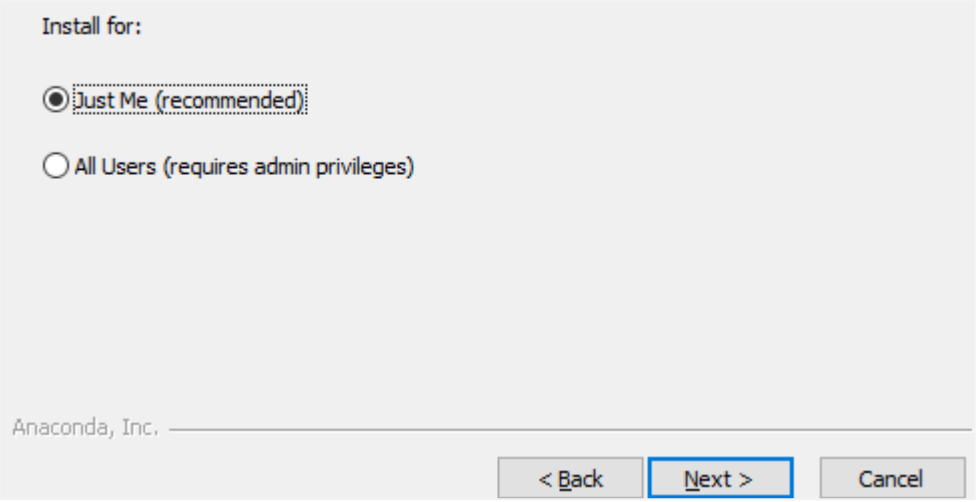
Click **I Agree**.

Figure 1-2 Anaconda3 license agreement



Select **Just me** under **Install for:** and click **Next**.

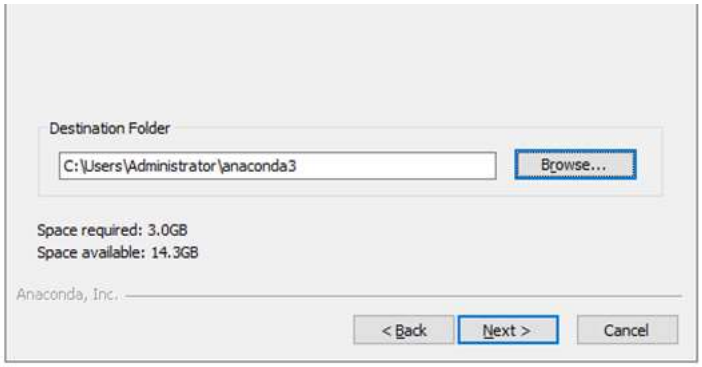
Figure 1-3 Selecting the installation user



Step 3 Set the installation path.

Specify the installation path and click **Next**.

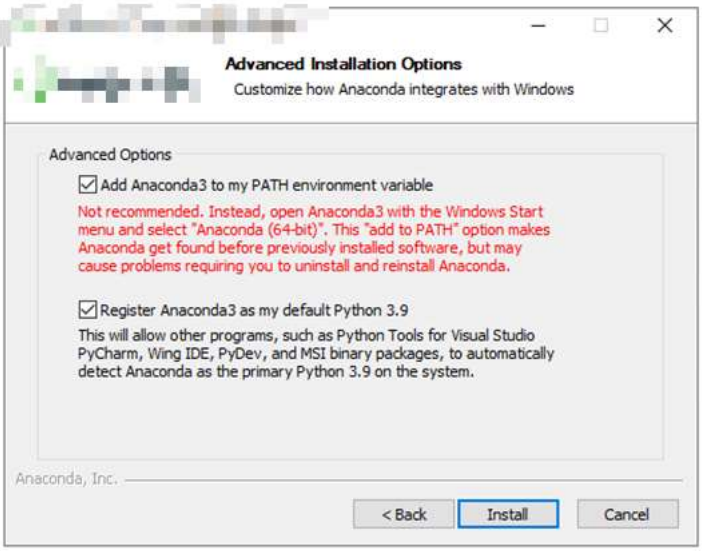
Figure 1-4 Specifying the installation path



Step 4 Configure the environmental variables.

Select **Add Anaconda3 to my PATH environment variable** and **Register Anaconda3 as my default Python 3.9** to reduce subsequent configuration, and click **Install**. (Note: If another Python version is installed on the computer, you are advised to remove it and then install Anaconda. Otherwise, environment conflicts may occur.)

Figure 1-5 Adding Anaconda3 to the environment variable



After the installation is complete, click **Finish**.

1.2 Introduction to Anaconda Virtual Environment

1.2.1 Introduction to Virtual Environment

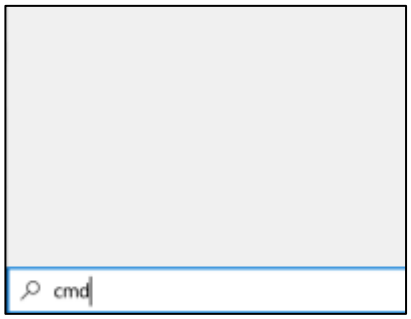
Anaconda is popular because of virtual environments. It allows developers to create unrelated Python environments on the host. When dependencies between different Python modules are complex, or different development framework applications are used, you can use virtual environments to separate environments.

Note: Modules in the virtual environments are independent of each other. If you have installed PyTorch in environment A and want to use PyTorch in environment B, you need to install PyTorch in environment B. The same is required for Spyder and Jupyter Notebook.

1.2.2 Using CLI to Create Virtual Environments

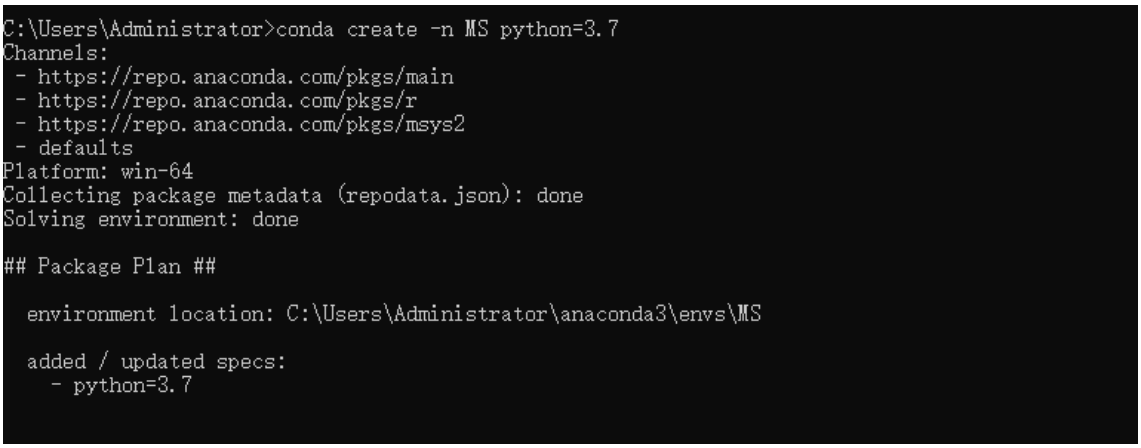
Step 1 Search for **cmd** in the search box on the taskbar to open the CLI.

Figure 1-6 Opening the CLI



Step 2 Run the **conda create –n** *Environment name* **python==***x.x* command. Specify the environment name and Python version as required.

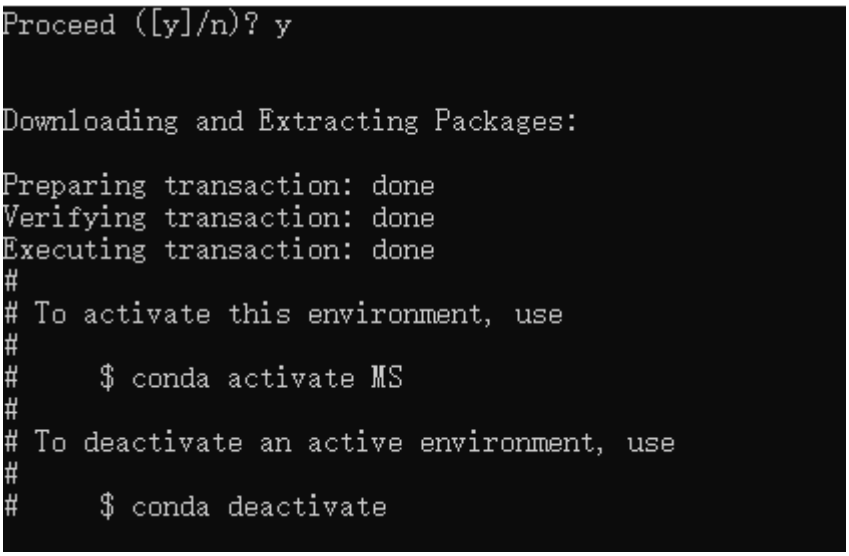
Figure 1-7 Creating a Microsoft virtual environment



Step 3 Enter **y** when you are asked whether to proceed.

Wait until the installation is complete.

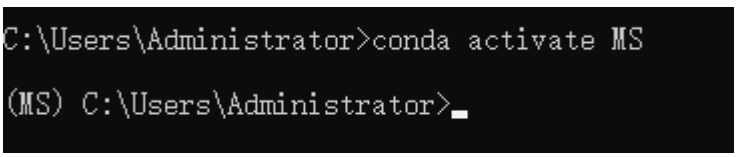
Figure 1-8 Microsoft virtual environment created



1.2.3 Activating a Virtual Environment

Run the **activate** *environment name* command in the CLI to activate the virtual environment. If the name between the brackets ahead the command line is changed, you have started and entered the environment.

Figure 1-9 Activating a Microsoft virtual environment



You can run the **pip** or **conda** command to install modules.

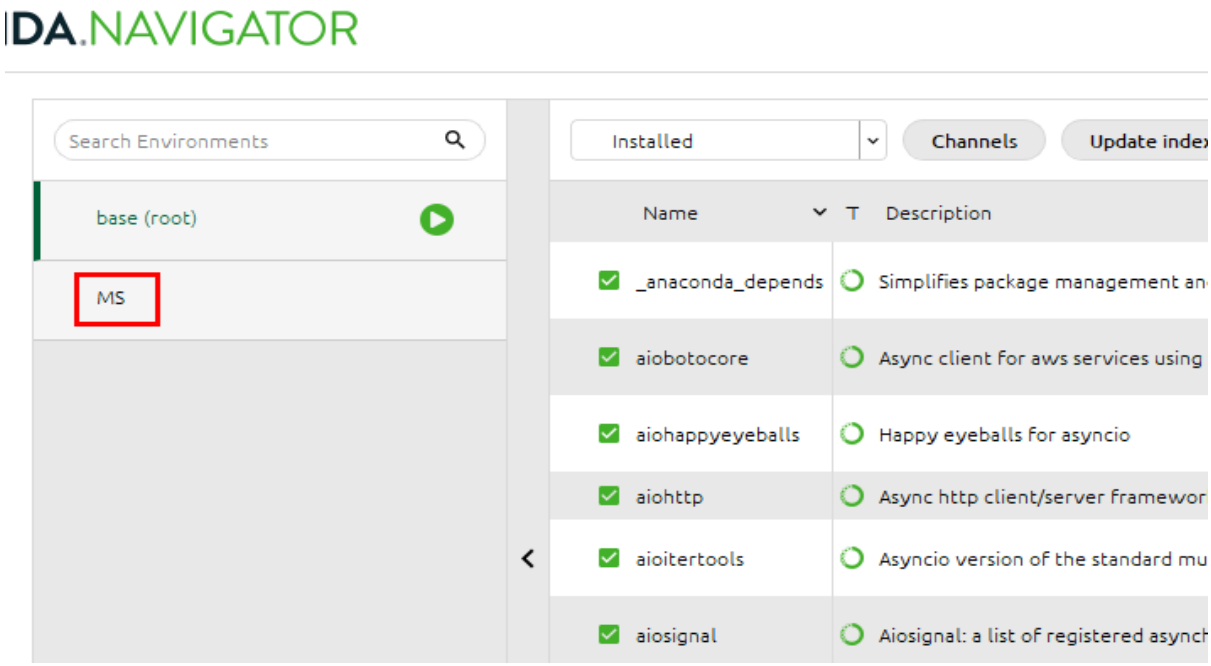
Run the following command to exit the environment:

```
conda deactivate
```

1.2.4 Viewing a Virtual Environment

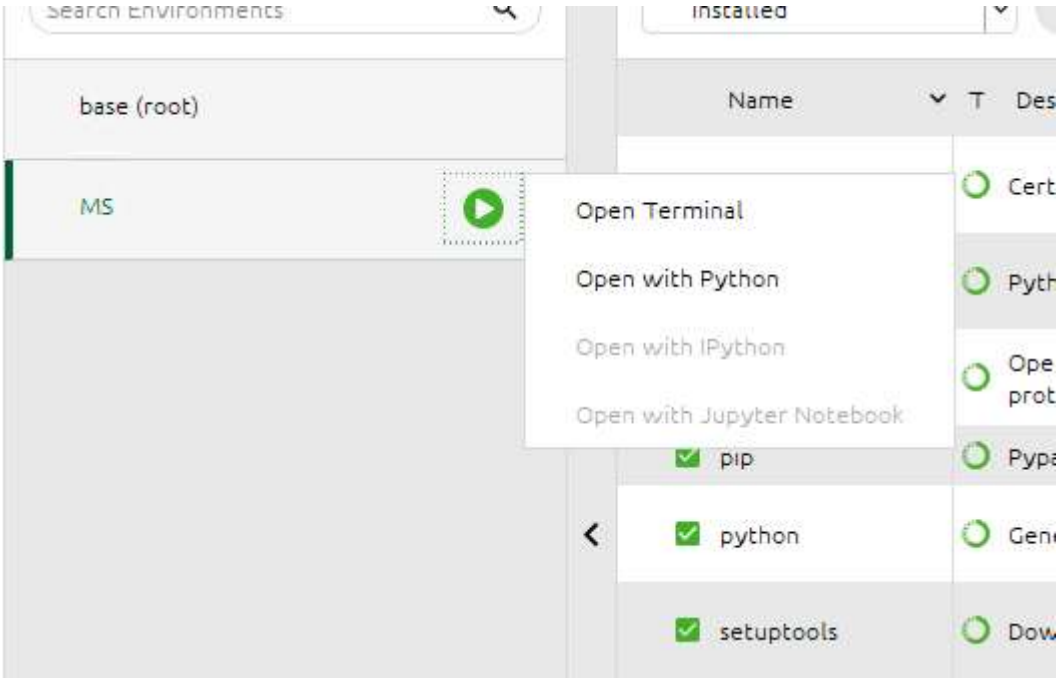
View created virtual environments on Anaconda Navigator.

Figure 1-10 Virtual environments created on Anaconda Navigator



Click the triangle to the right of the environment name to open the CLI of the virtual environment.

Figure 1-11 Opening the virtual environment. Terminal



Now you can install the modules in the current environment without starting the virtual environment.

1.2.5 Deleting a Virtual Environment

Run the **conda remove -n** *environment name* **--all** command in the CLI and type **y** when asked whether to proceed to delete the environment, as shown below:

Figure 1-12 Deleting a virtual environment

```
C:\Users\Administrator>conda remove -n MS --all

Remove all packages in environment C:\Users\Administrator\anaconda3\envs\MS:


## Package Plan ##

  environment location: C:\Users\Administrator\anaconda3\envs\MS


The following packages will be REMOVED:

  ca-certificates-2024.12.31-haa95532_0
  certifi-2022.12.7-py37haa95532_0
  openssl-1.1.1w-h2bbff1b_0
  pip-22.3.1-py37haa95532_0
  python-3.7.16-h6244533_0
  setuptools-65.6.3-py37haa95532_0
  sqlite-3.45.3-h2bbff1b_0
  vc-14.40-haa95532_2
  vs2015_runtime-14.42.34433-h9531ae6_2
  wheel-0.38.4-py37haa95532_0
  wincertstore-0.2-py37haa95532_2


Proceed ([y]/n)? _
```

1.3 Installing Tool Libraries

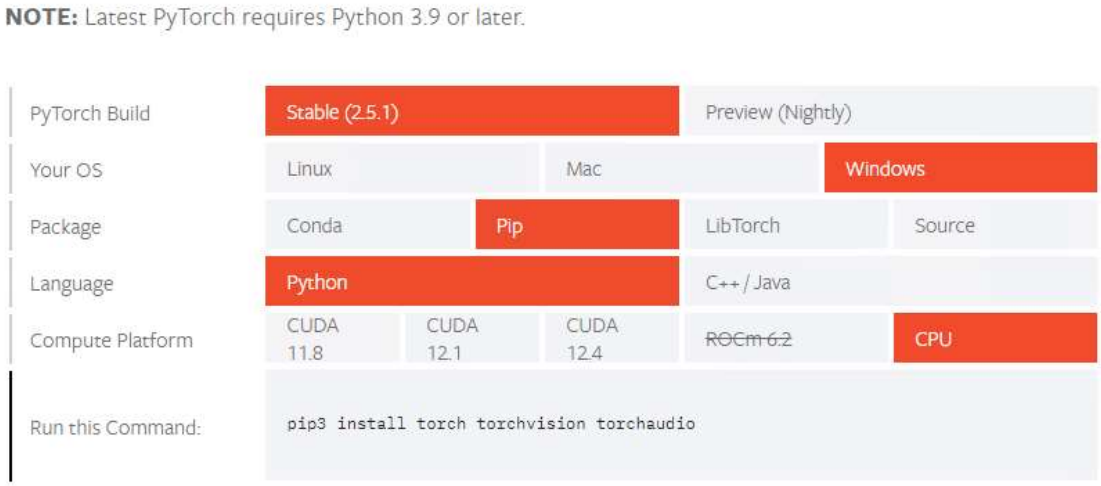
1.3.1 Installing PyTorch

Create a virtual environment PyTorch and activate it. For details about how to create a virtual environment, see the preceding steps.

Step 1 Install PyTorch.

Visit the PyTorch official website https://pytorch.org/get-started/locally/ to obtain the installation command.

Figure 1-13 PyTorch installation command



Select options as required.

Install Pytorch in the virtual environment.

Figure 1-14 Installing PyTorch



If a success message is displayed, the installation is complete.

Step 2 Check whether PyTorch is installed successfully.

Run the following command in the CLI in the PyTorch virtual environment to check whether the installation is successful:

```
python -c "import torch;print(torch.__version__)"
```

If the version number is displayed, the installation is successful.

```
2.5.1+cpu
```

1.3.2 Installing Other Tool Libraries

Activate the corresponding virtual environment and use pip or conda to install the tool library, for example, easydict required in the experiment.

```
pip install easydict
```

2 Ubuntu Lab Environment Configuration

2.1 Installing Anaconda

2.1.1 Procedure

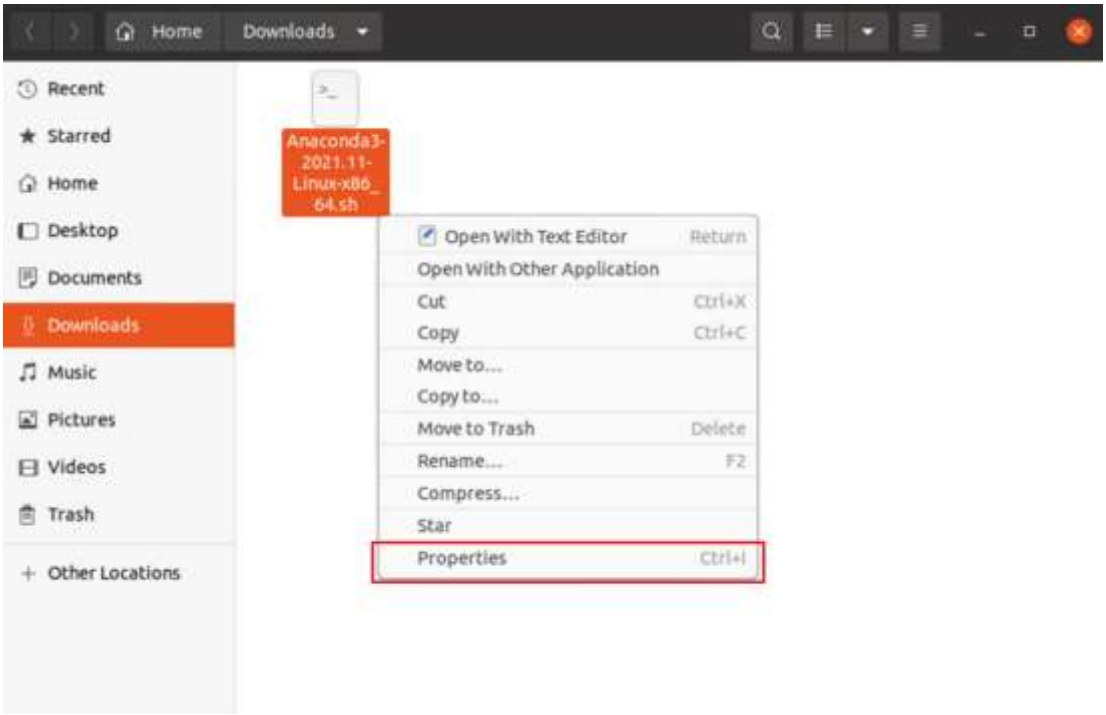
Step 1 Download Anaconda.

Log in to the Anaconda official website https://repo.anaconda.com/archive/ to download the installation package. In this example, the Linux installation package is selected. For Linux 64-bit users, the **Anaconda3-20xx.xx-Linux-x86_64.exe** package is recommended. (This example describes how to install Anaconda. You can also use Miniconda, and the installation procedure is similar. Miniconda is a lightweight version of Anaconda. By default, it contains only Python and Conda. You can use pip or conda to install the Miniconda package.)

Step 2 Install Anaconda.

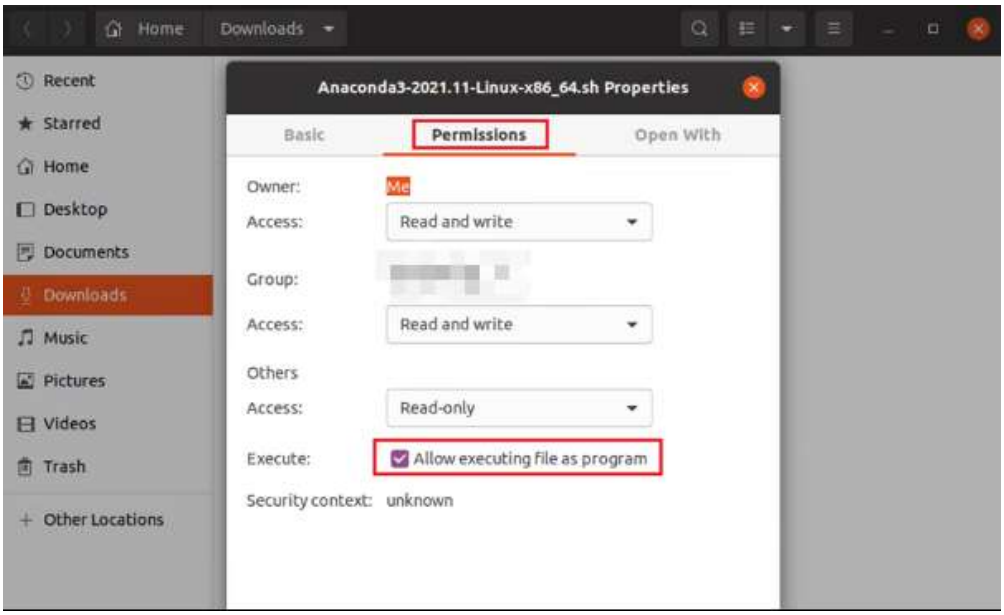
Right-click the Anaconda3-xxxx.xx-Linux-x86_64.sh file and choose Properties.

Figure 2-1 Opening the file properties



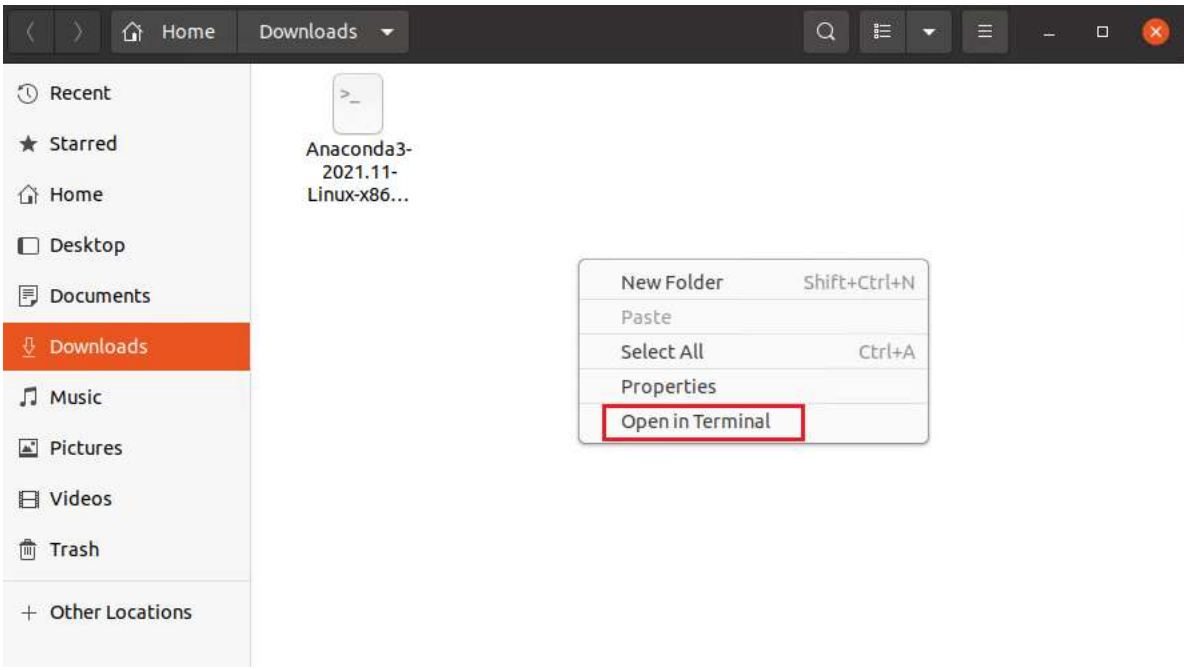
Click the **Permissions** tab and select the check box for **Execute**.

Figure 2-2 Selecting Allow executing file as program



Right-click the blank area in the file location area and choose **Open in Terminal**.

Figure 2-3 Opening the terminal in the current directory

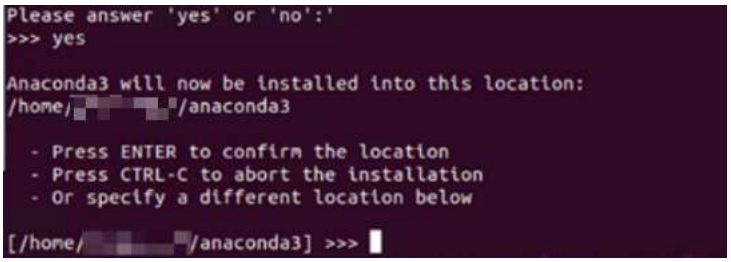


Enter the following command in the terminal to start the installation:

```
./Anaconda3-2021.11-Linux-x86_64..sh
```

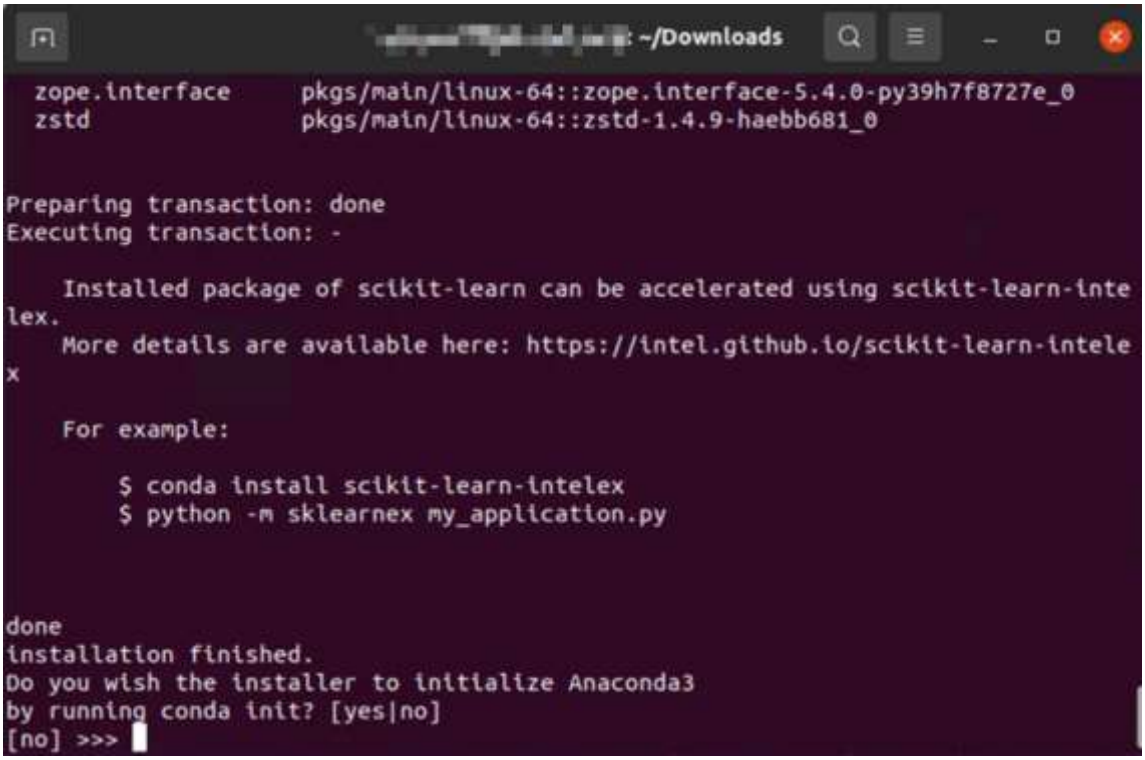
Press **Enter** to read the installation agreement and enter **yes** when asked whether you agree. The default choice is **no**. Press **Enter** to install Anaconda3 in the user directory by default. You can also specify the installation path in the Linux system.

Figure 2-4 Anaconda3 installation path



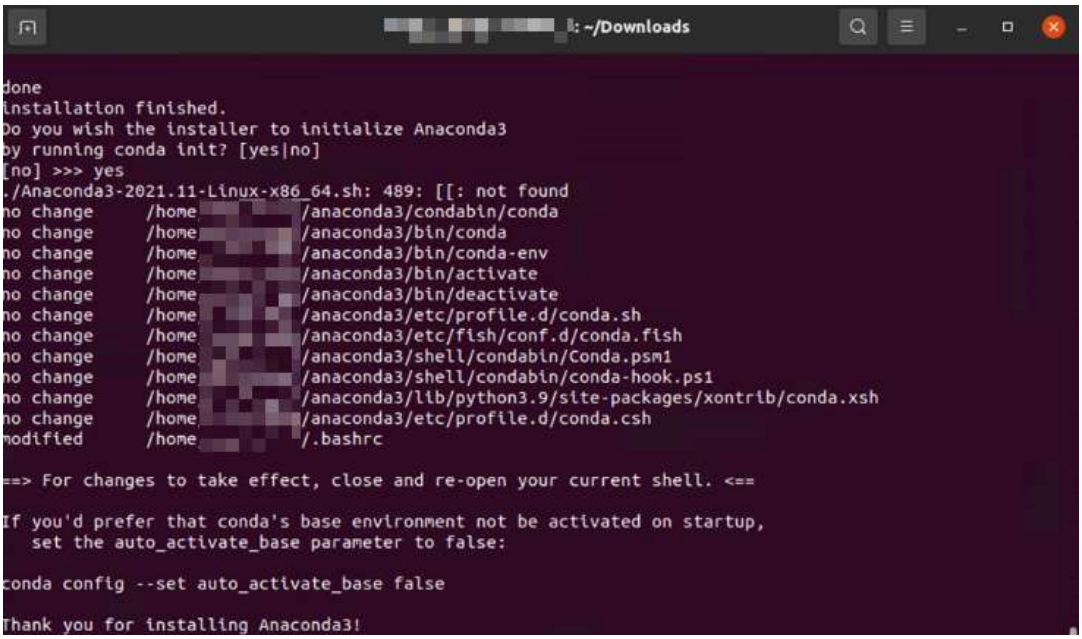
After the installation is complete, enter **yes** to initialize Anaconda3.

Figure 2-5 Initializing Anaconda3



Anaconda3 is installed successfully. Exit the current terminal and create a terminal window to re-initialize the command-line environmental variable.

Figure 2-6 Anaconda3 installed



2.2 1.4 Installing Tool Libraries

2.2.1 Installing PyTorch

Step 1 Create a virtual environment.

Create a PyTorch virtual environment.

Step 2 Activate the virtual environment.

Activate the PyTorch virtual environment.

Step 3 Install PyTorch.

Enter the following command in the terminal:

```
pip3 install torch torchvision torchaudio --index-url https://download.pytorch.org/whl/cpu
```

The installation is successful if a success message is displayed.

Step 4 Check whether PyTorch is installed successfully.

Run the following command in the CLI in the PyTorch virtual environment to check whether the installation is successful:

```
python -c "import torch;print(torch.__version__)"
```

If the version number is displayed, the installation is successful.

```
2.5.1+cpu
```

3 Using Huawei Cloud

3.1 Overview

Huawei Cloud provides enriched cloud resources and services to meet the requirements of developers. The following describes how to apply for Huawei Cloud resources, including registering an account, performing identity authentication, and obtaining the access key (AK) and secret access key (SK). This chapter also describes main Huawei Cloud services.

Upon completion of this task, you will be able to:

- Understand the AI development environment.
- Apply for an AI development environment.
- Test the functions in the environment you have set up.

3.2 Huawei Cloud Account Application and Identity Authentication

1. The account is used to apply for services required for the experiments on the Huawei Cloud website.

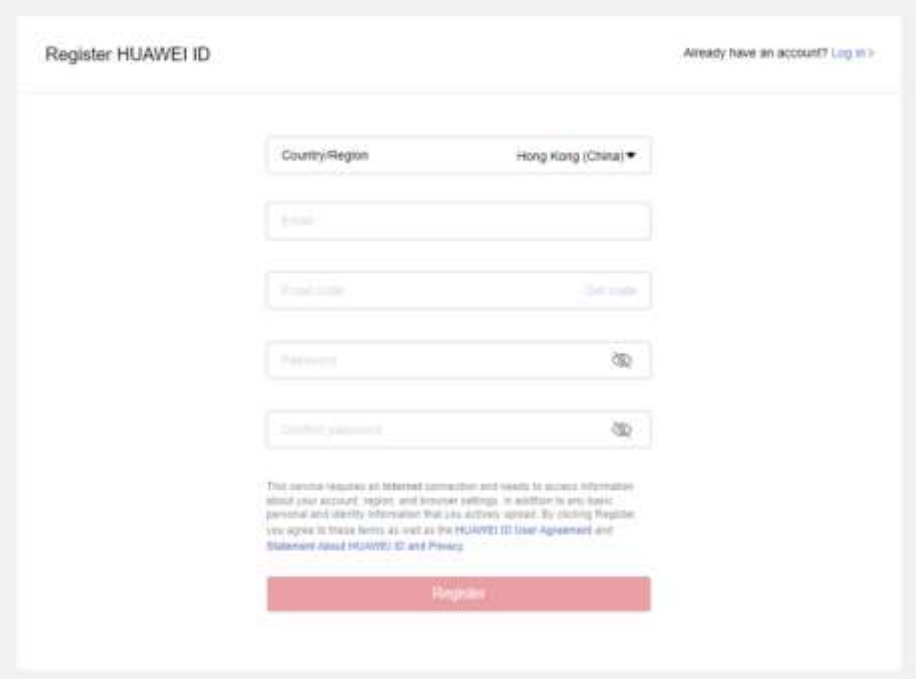
2. The account is used to experience Huawei Cloud services, including deep learning, machine learning, image recognition, speech recognition, and natural language processing.

3.2.1 Procedure

Step 1 Visit the Huawei Cloud official website and register an account.

Visit http://www.huaweicloud.com/ and click **Sign Up** in the upper right corner of the home page.

Figure 3-1 Register HUAWEI ID



Click **Register**. Click **Identity Authentication**. The identity authentication page is displayed.

Step 2 Perform identity authentication.

Click **Individual Authentication**. The individual authentication page is displayed.

3.3 Obtaining the Access Key and Secret Access Key

3.3.1 Overview

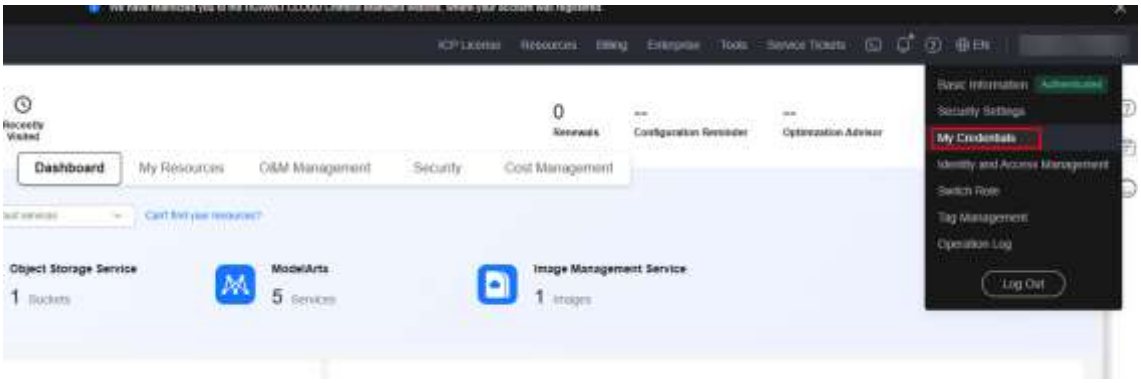
The access key (AK) and secret access key (SK) are your identity codes for using Huawei Cloud services. When using certain Huawei Cloud services for the first time, you are required to enter the AK and SK.

3.3.2 Generating an AK and SK

Step 1 Click **My Credentials**.

Log in with your Huawei Cloud account, click the username in the upper right corner of the page, and choose **My Credentials** from the drop-down list.

Figure 3-2 Entry to My Credentials



Step 2 Add an access key.

On the **My Credentials** page, click **Access Keys** on the left. The **Access Keys** page is displayed. On the **Access Keys** page, click **Create Access Key** to add an access key. Each access key can be downloaded only once.

Figure 3-3 Adding an access key



Note: After an access key is added, a file containing the access key is automatically downloaded. Keep the file secure.

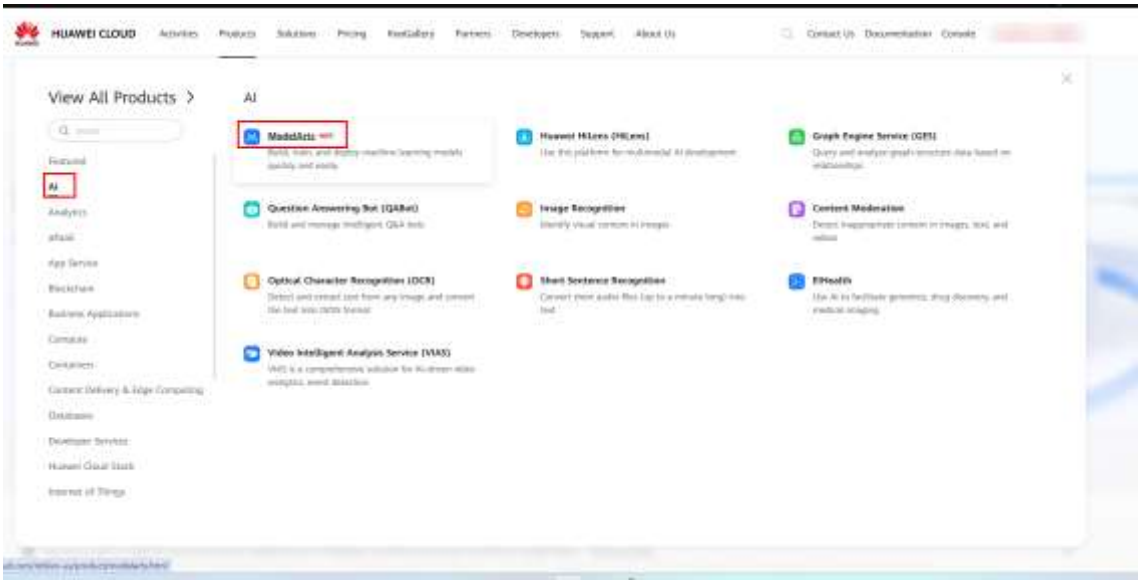
If you do not want to use the AK or SK authentication mode, disable or delete it on this page.

3.4 Purchasing Huawei Cloud Resources-ModelArts

3.4.1 ModelArts

As a one-stop development platform for AI developers, ModelArts provides massive data preprocessing, semi-automated data labeling, distributed training, automated model building, and on-demand model deployment on the device, edge, and cloud for machine learning and deep learning, helping AI developers build and deploy models quickly and manage the AI development lifecycle.

Figure 3-4 ModelArts entry



Access ModelArts, choose **Development Workspace** > **Notebook** on the left, and create a development environment based on CPU.

Figure 3-5 Creating a development environment



Figure 3-6 Create Notebook (1)

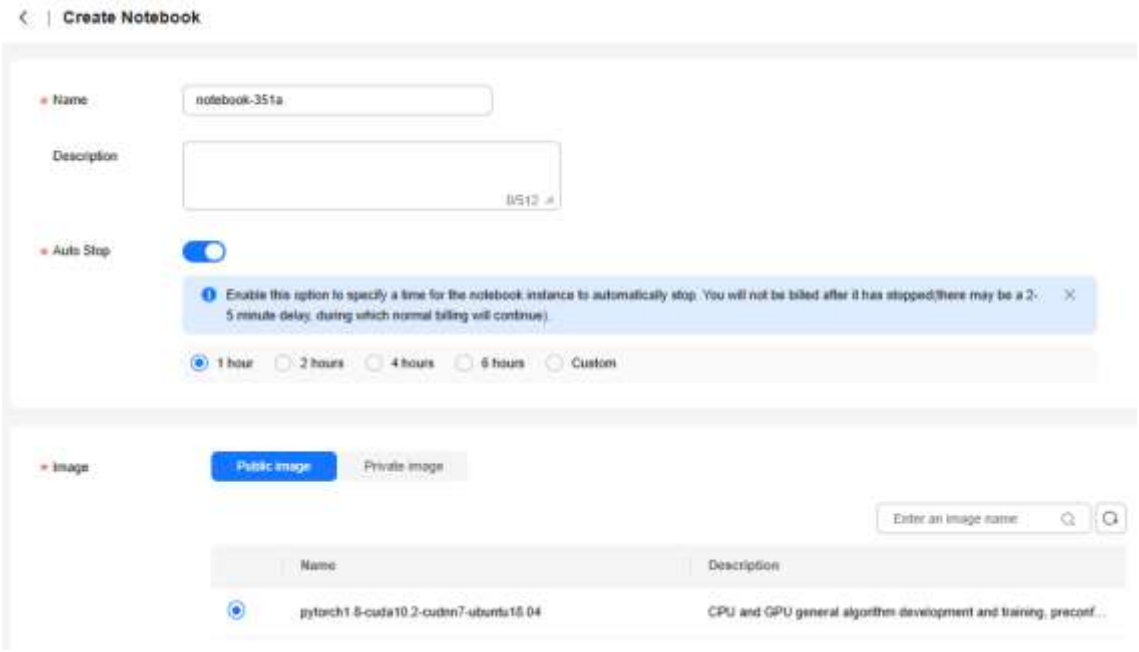
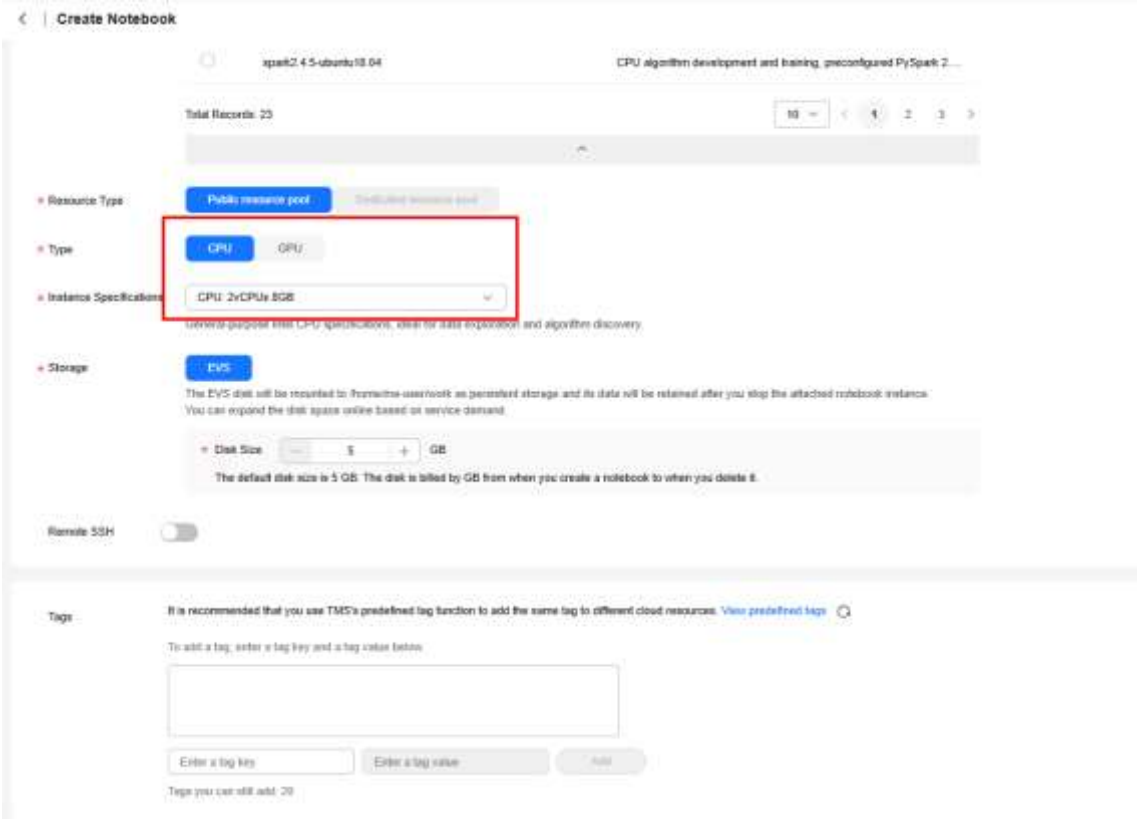


Figure 3-7 Create Notebook (2)

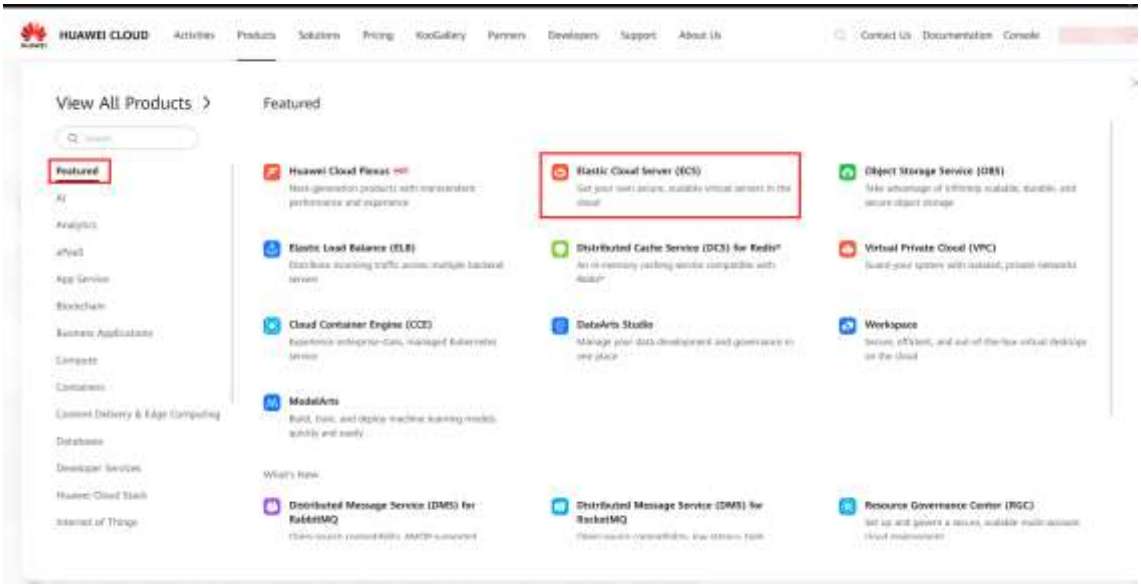


Open the created notebook

3.5 Purchasing Huawei Cloud Resources-ECS

Step 1 Log in to Huawei Cloud and find **ECS**.

Figure 3-8 ECS



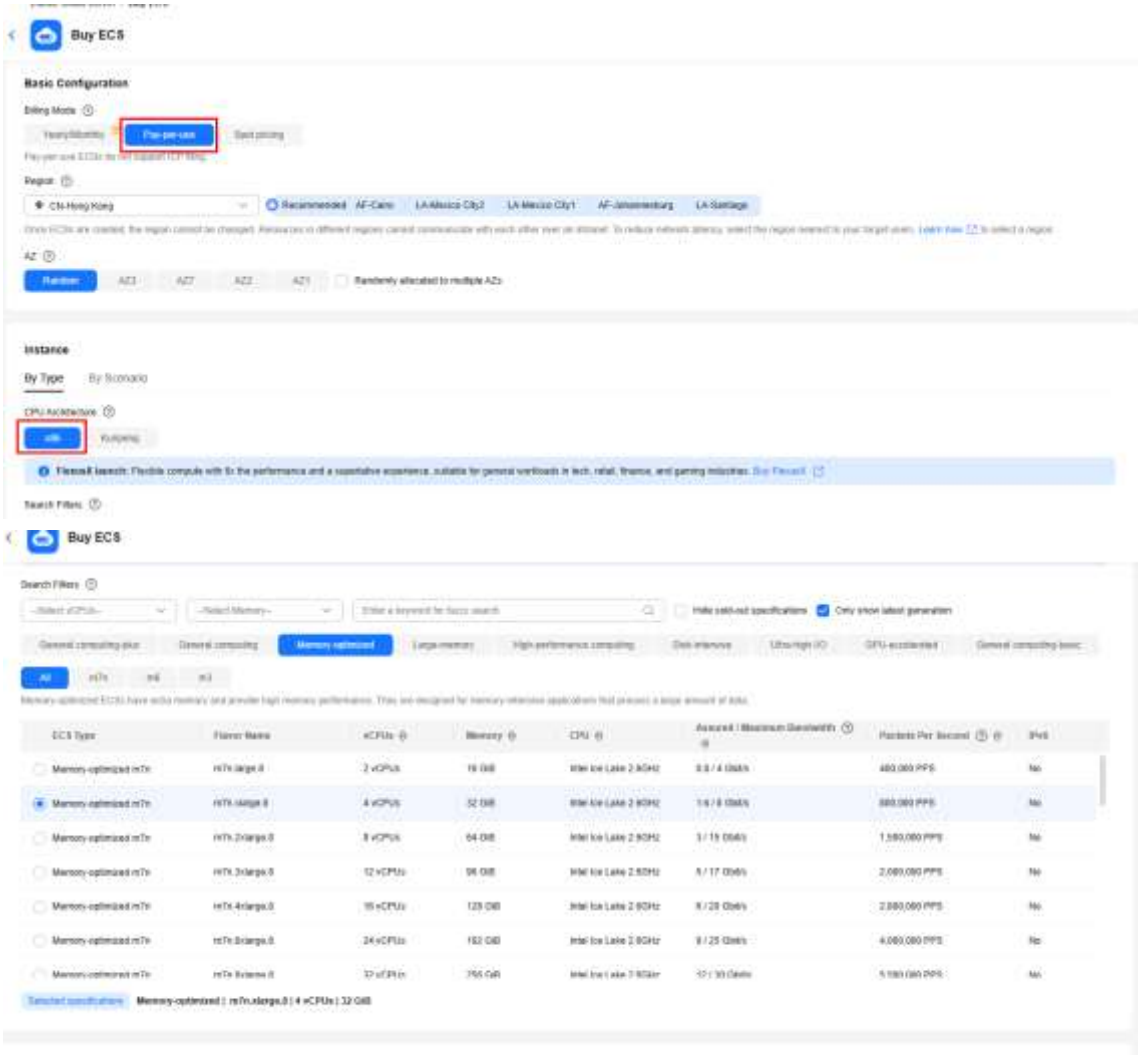
Step 2 Click **Buy**. The product purchase page is displayed.

Figure 3-9 Purchase page



Step 3 Select **Pay-per-use**, **X86**, and **Memory-optimized** (the experiment has a high requirement on memory capacity.)

Figure 3-10 Configuration selection



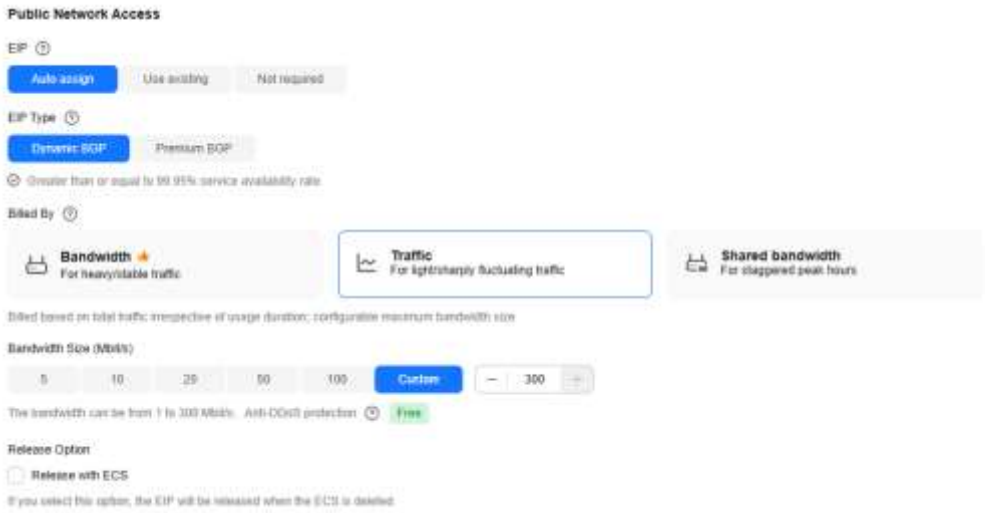
Step 4 Select the instance with a 32 GiB memory, select **Huawei Cloud EulerOS**, and click **Next**.

Figure 3-11 Selecting the system



Step 5 On the **Public Network Access** page, select **Traffic** for **Billed By**, set **Bandwidth Size (Mbit/s)** to 300, retain the default settings for other parameters, and purchase the instance. Experiments only generate inbound traffic, so billing is not required.

Figure 3-12 EIP



Step 6 After purchasing the instance, log in to the ECS through CloudShell. You can also use software such as Xshell to log in if it is installed on your computer.